

COGITO • SCRIBO

架构解析

我思故我写 • 姊妹卷

七套核心系统的工程实现

wUwproject • CC BY-SA 4.0 • 免费公开

arch-v1.3.0 • 2026 年 9 月

目录

版权页 ----- 1

 书名 ----- 1

 作者 ----- 1

 协议（显式声明） ----- 1

 收录与规模 ----- 1

 版本 ----- 1

 与母书的关系 ----- 2

导读：方法论 ↔ 架构映射 ----- 3

 一、映射表 ----- 3

 二、八系统关系总览 ----- 4

 三、实验性说明 ----- 5

 四、阅读路径 ----- 6

01 | skill-standardization 架构与规范体系文档 ----- 7

 一、系统概览 ----- 7

 二、完整审查规则体系（R-01 ~ R-26） ----- 12

 三、核心设计原则 ----- 17

 四、误报分类机制（v2.94.0 重写） ----- 18

 五、审计后自动修复体系（fix.py） ----- 19

 六、关键流程 ----- 22

 七、关键版本变更摘要（v2.82.0 → v2.95.0） ----- 23

02 | semantic-split 架构与规范体系文档 ----- 24

 一、系统概览 ----- 24

 二、三管线递进调度系统 ----- 26

 三、10 道门禁钩子系统 ----- 32

 四、自增强闭环 ----- 34

 五、渐进加载决策树 ----- 35

 六、JSON 模板管理体系 ----- 37

七、模型管理	39
八、核心设计原则	41
九、交互方式	42
十、外部依赖	44
十一、版本历史	45
十二、数据流全景	46
03 activity-duration-estimation 架构与规范体系文档	48
一、系统概览	48
二、七主子技能体系	50
三、全流程编排	55
四、核心设计原则	56
五、配置体系	57
六、配置体系	57
七、风险维度体系	58
八、完整测试	58
04 RAG Assistant 架构文档	60
一、系统概览	60
二、三层架构	66
三、组件详解	68
四、记忆系统 — memory.py	78
五、外部接口	81
六、流程详解	85
七、部署与启动流程	91
八、依赖与存储	93
九、安全与隐私	95
十、版本演进要点	95
05 Structured Writer 架构文档	98
一、系统概览	98
二、后端架构（v3.1.0b1 定稿：LM Studio 统一管理）	100
三、三层架构	101

四、模板系统	102
五、组件详解	104
六、外部接口	108
七、UI 布局	111
八、写作管线	112
九、版本演进要点	114
06 podcast-maker 架构与规范体系文档	117
一、系统概览	117
二、三层架构	119
三、模块划分	121
四、循环体系：大环套小环	126
五、核心机制	137
六、配置体系	141
七、外部接口	142
八、依赖与存储	145
九、边界与不做什么	146
十、版本演进要点	147
07 Orchestrator 架构文档	153
一、系统概览	153
二、双入口架构	154
三、Web UI 架构	156
四、执行引擎（真执行）	158
五、数据模型	160
六、LLM 客户端	160
七、工具系统（16 个）	161
八、配置系统	162
九、CLI 接口	162
十、演变路线	163
十一、目录结构（agent/Orchestrator/）	163
08 silprespec-orchestrator 架构文档	165

一、系统概览	165
二、目录结构	167
三、核心编排架构	168
四、三层 loop 架构	171
五、前置规范 — 14 种穷举组合	172
六、原子库 (atoms.py)	174
七、工具接口规范 (tool_registry.py)	176
八、组件详解	177
九、进度地图 (progress_map.py)	179
十、Web UI 架构	180
十一、LLM 客户端 (llm_client.py)	181
十二、配置系统	181
十三、CLI 接口	182
十四、数据流总览	183
09 技能编排器到 agent 编排器——编排对象迁移与“圈”的 z 轴	185
一、两篇并列文档之间，空着一条变化线	185
二、迁移了什么：对象、通道与 LLM 的在场位置	185
三、表象逼近 ReAct：silprespec 的"自主"长什么样	186
四、“圈”的 z 轴：人工编排到 LLM 自主编排的表象	187
五、这条变化线在母书中的位置	189
六、结语：变的是对象与通道，不变的是枚举内核与决策在人	190
延伸阅读	191
10 施工层级——画地图定位、功能解耦与统一各级推动点位	193
一、引子：蠢问题不发生在结果上，主要来自缺失过程控制	193
二、画地图定位：一张动态的、多文件的地图	193
三、功能层解耦：每个功能的迭代只属于它自己	195
四、统一各级推动点位：耦合只发生在这一层	196
五、总览：一张地图在成品之外，两个结构层在成品之中	198
六、结语：逻辑和纪律是给 LLM 的，不靠它的八爪鱼推理	200
延伸阅读	202

结语：结构是方法论的落点

203

这本册子收录了什么路

203

架构文档的特殊性

204

最后

204

附录 A 统一术语表

205

一、承自母书的术语（定义见母书附录 A）

205

二、架构册特有术语

206

附录 B 运行速查

210

一、技能（宿主平台加载，无端口）

210

二、编排器（独立进程 + Web UI）

210

三、智能体（独立 HTTP 服务，maby_agent 仓库）

211

四、调用关系速记

211

五、收束篇（架构 09/10）

211

版权页

书名

《我思故我写 · 架构解析——八套核心系统的工程实现》

前置规范编排器、RAG 智能体、结构化写作引擎、播客生成智能体……本书收录 wUwproject 八套核心技能与智能体的架构设计文档——它们是《我思故我写》方法论在真实工程里的落地形态。

作者

wUwproject

协议（显式声明）

本书整体采用创作共用署名-相同方式共享 4.0 国际许可协议（CC BY-SA 4.0）——可自由复制、分发、演绎，须署名 wUwproject 并按相同协议共享；各系统代码实现为 Apache 2.0。

收录与规模

- **收录：**架构 01-10（十篇架构文档），正文对应八套核心系统——架构 09/10 为演进收束篇，非新系统
- **规模：**约 7.7 万字（出版折算口径：汉字 + 英文单词 + 数字串，与母书一致），15 章（版权页 + 导读 + 架构 01-10 + 结语 + 附录 A/B）
- **核对同步：**各篇版本号跟随对应工具/智能体的真实版本，成书前与真实代码逐节核对

版本

- 2026 年 8 月 · 初版（arch-v1.0.0）：收录 7 篇架构文档（5 篇成熟 + 2 篇实验性），全部

与对应工具/智能体的真实代码核对同步

- 2026 年 9 月 · arch-v1.0.1: 入书清洗管线化, 章题对齐母书族系格式 (NN | 标题)
- 2026 年 9 月 · arch-v1.0.2: 立册子规范 (STRUCTURE_GUIDE 与成册说明)
- 2026 年 9 月 · arch-v1.1.0: 新增架构 08 《技能编排器到 agent 编排器——编排对象迁移与"圈"的 z 轴》, 正文 8 篇、系统仍七套
- 2026 年 9 月 · arch-v1.1.1: 架构 08 文意修订, 立 z 轴正解并新增循环 / 自主编排 / 填槽即决策三组术语
- 2026 年 9 月 · arch-v1.1.2: 印刷字体根治——思源双字体 (黑体 + 等宽) 全元素覆盖, 杜绝系统回退字体混入
- 2026 年 9 月 · arch-v1.1.3: 表格 GFM 语义修复与打印溢出根治, 全书 h1+h2 双级目录 103 条
- 2026 年 9 月 · arch-v1.1.5: 架构 04 (rag-assistant) 随代码 v2.4.1 全文复核, 新增前置分类 (§ 1.4) 与 Ranker 后插件点位 (§ 3.10) 两节
- 2026 年 9 月 · arch-v1.2.0: 骨架对齐族系规范——版权页小节化、版本日志一行式、附录归位 book/appendix/、PDF 管线归位 build/PDF/ (与排版书同批结构统一)
- 2026 年 9 月 · arch-v1.3.0: 新增结语 (99_结语.md, 正文后、附录前装配) ——三册结语标配补齐, 全书由 12 章扩至 13 章、约 4.9 万字
- 2026 年 10 月 · arch-v1.4.0: 新增架构 06 (podcast-maker) 与架构 10 (施工层级收束篇——画地图定位、功能解耦与统一各级推动点位), 架构 06-08 顺延为 07-09, 全书由 13 章扩至 15 章、约 7.7 万字

与母书的关系

- 母书回答**为什么** (方法论、原则、边界); 本书回答**怎么做** (工程实现、模块划分、代码架构)
- 母书附录 B 工具索引是索引, 本书是这些工具的深度展开
- 两册独立版本: 母书 v1.2.0 完结不再逐版同步; 本书随工具迭代独立更新 (各篇版本号跟随对应工具的真实版本)

导读：方法论 ↔ 架构映射

母书《我思故我写》回答"为什么"——前置规范为什么大于后置验证、穷举为什么是宿命、槽位为什么是减法。本书回答"怎么做"——这些原则如何落成八套真实运行的核心系统。 每篇架构文档开头都写着同一句话：**基于"我思故我写"方法论**。本导读给出映射表——读架构时可以回到母书对应篇目看原理，读母书时可以来这里看落地。

一、映射表

架构篇	系统	对应母书篇目	落地的核心原则
架构 01	skill-standardization（标准化审计引擎，v2.103.0）	03 递归自举的元验证	引擎审计别的技能，自身配置纳入同一审计体系（方法论约束自己）
架构 02	semantic-split（语义拆分，v3.1.1）	09f 名词不解释就等于没有名词	触发双清单（正向触发 + 反向排除）——槽位四维展开的工程形态
架构 03	activity-duration-estimation（活动历时估算，v1.11.7）	07 四条原则的元逻辑	三点估算/蒙特卡洛的工具化；配置推动（09b）
架构 04	rag-assistant（RAG 智能体，v2.4.1）	06 领域智能体 + 09a 探测边界 + 09c 语义免疫	LLM 分词穷举组合检索 + PY 门禁校验；top-N 多 KB 路由；前置分类两级收敛（三槽位归类 × 查询类型场景）
架构 05	structured-writer（结构化写作引擎，v3.1.14）	09b 配置推动 + 09d 粒度 + 09e 提示词拉扯	配置推动的穷举一致性（模板 meta 数组 = 统一推动点位）；引文回验（前置规范，不依赖模型自觉）

架构篇	系统	对应母书篇目	落地的核心原则
架构 06	podcast-maker（播客生成智能体，v1.4.0）	08a 前置规范 + 09b 统一推动点位	判与写分离、环有界；四处唯一定义处（路径/配置/换算/定位）；音色体检三判据合验（验收与检测同尺）
架构 07	orchestrator（编排器，v2.8.1，实验性）	05 链驱动 Pipeline 编排	固化 vs 调用权衡；预定义技能链 + LLM 只做两头
架构 08	silprespec-orchestrator（前置规范编排器，v0.1.0，实验性）	08a 三态谱系 + 08c 槽位减法 + 09b 配置推动	14 种穷举原子化组合，PY 确定性查表，LLM 填空执行
架构 09	演进收束篇——编排对象迁移（以架构 07/08 为标本，非新系统）	05 链驱动 Pipeline 编排 + 08 系有限决策/前置规范	迁移的是对象与通道；表象逼近 ReAct、内核仍有限枚举、决策在人；"圈"沿拟人 z 轴上移但有界
架构 10	施工层级收束篇——画地图定位、功能解耦与统一各级推动点位（以架构 05/06 为标本，非新系统）	08a 前置规范 + 09b 统一推动点位	画地图定位（动态多文件的地图，三方对比即进度管理，成品之外）；功能层解耦（失败半径收敛）；统一各级推动点位（耦合只在此层）；纪律在人

二、八系统关系总览

8 个系统不是并列的——它们分三层，调用关系完全不同：

【技能层】

宿主平台技能目录 • SKILL.md + scripts/ • 无独立进程

skill-standardization（审计引擎，元层——审计其他技能）

semantic-split • activity-duration-estimation



关键区分——两代编排器调的对象不同：

- **第 I 代 orchestrator 调技能：** subprocess 真跑 skill 脚本（可执行过滤，32 个技能入池），中间是死的、LLM 只做两头
- **第 II 代 silprespec-orchestrator 调智能体：** HTTP POST 调智能体 API（rag-assistant、structured-writer），LLM 填空生成工具输入，交付给智能体走内部流程——编排器不替智能体干活

技能与智能体是两类完全不同的存在：**技能**是跨平台分发的扩展单元（SKILL.md 声明 + scripts 脚本，无独立进程，由任意宿主平台加载，也可被编排器以 subprocess 调用）；**智能体**是独立 HTTP 服务（自带 Web UI 与外部 API，自己起进程自己管端口，被编排器以 API 调用）。运行方式见附录 B 运行速查。

三、实验性说明

架构 07、08 标注**实验性**：系统代码已实现（orchestrator v2.8.1、silprespec-orchestrator v0.1.0——14 种穷举组合、六类输入分类、ProgressMap 进度地图等机制均已落地），但**端到**

端的实验验证仍在进行中，架构可能随实验结果调整。它们代表方法论当前正在探索的方向——从"单系统内的前置规范"走向"多系统编排层的前置规范"。实验收敛后，这两篇会随工具迭代更新并去掉实验性标注；母书如出后续版本，亦将收录其沉淀出的方法论结论。

架构 09（演进收束篇）不新增系统，而是把 07/08 之间的变化线立成专文：**为什么从编排技能走到编排智能体**——迁移的是编排对象与调用通道（subprocess 跑技能 → HTTP 调智能体 API），表象逼近 ReAct 而内核仍是有限枚举、决策依旧在人；同一个"圈"沿拟人 z 轴上移了一个坐标，但未越过"决策让渡"的临界线。该篇是对本导读两代定位的展开论述，亦是在架构册侧先行兑现"实验性结论待沉淀"的承诺。

架构 10（施工层级收束篇）同样不新增系统，把多套系统共同遵守的**施工层级**立成专文：**这套项目为什么管得住**——三根支柱：画地图定位（一张动态的、多文件的施工地图，从一份施工公约长起，每次改动方案完整落地、与更新日志互为参照，施工现场/施工规划/施工记录三方对比即进度管理；地图是成品之外的部分，成品里是它被执行后留下的印痕）、功能层解耦（每个功能的迭代只属于它自己，最短的木板不是系统崩溃的节点）、统一各级推动点位（功能的耦合层与多功能调度层，耦合只发生在这一层，单功能全绿而整体崩溃只查通信规范与调度顺序）。以架构 05（structured-writer）与架构 06（podcast-maker）的真实事故为标本，与架构 09 同源收束：决策在人是运行判据，纪律在人是过程判据。

四、阅读路径

- **只看某一个系统**：直接跳对应架构篇，每篇自包含（系统概览 → 模块划分 → 核心机制 → 边界）
- **想理解两代编排器为什么长这样**：读架构 09（演进收束篇）——不重复系统机制，专讲 07 → 08 的变化线（迁移了什么 / 什么没变 / "圈"的 z 轴判据）
- **想理解这套项目为什么管得住**：读架构 10（施工层级收束篇）——画地图定位 / 功能解耦 / 统一各级推动点位（以架构 05/06 的事故为标本）
- **对照母书读**：先读母书对应篇目看原理，再来本文看落地（映射表见上）
- **想理解方法论全貌**：先读母书第 I 部（01-03 与本书架构 01-03 直接对应），再回本书

01 | skill-standardization 架构与规范体系文档

一、系统概览

skill-standardization 是一个 **Skill 全生命周期标准化管理工具集**，围绕以下闭环运行：

规范定义 (spec/*.json)

→ 构建器 (skill_builder: create / update / refactor)

→ 审查器 (skill_audit: R-01 ~ R-26)

→ 修复器 (fix.py: 35+ 自动修复函数 (含 code_block_markers/list_mixing/code_block_lang/section_completeness/error_handling_faq))

→ 验证 (--verify 铁律阻断 + --classify --category --subtype 误判分类)

1.1 三层架构

层	组件	职责
表现层	SKILL.md + references/*.md + CLI	人类可读的文档和命令行交互
业务层	skill_builder / skill_audit / fix.py / safe_io	创建/更新/改造/审查/修复/安全写入的核心逻辑
数据层	spec/*.json	按需加载的标准化规范定义；数据存储在 skills/.standardization//

1.2 目录结构

```
skill-standardization/
├── SKILL.md                # 主文件 (≤230行, 渐进式入口)
├── _meta.json              # 7 字段元数据
                             (name/version/description/author/tags/data_dir/triggers)
├── references/             # 渐进式文档
│   ├── guide.md           # 完整使用教程
│   ├── architecture.md    # 架构设计 (本文件)
│   ├── reference.md        # API/命令参考手册
│   ├── rules.md           # 铁律 1~9 与完整规则说明
│   ├── blueprint_flow.md  # 蓝皮书扫描流程定义
│   ├── antipatterns.md    # 反模式速查
│   ├── data_dir_map.md    # 数据目录路径引用对照表
│   ├── examples.md        # 使用示例
│   ├── faq.md             # 常见问题
│   ├── changelog.md       # 版本更新日志
│   ├── permissions.md     # 权限声明
│   └── LICENSE.md         # MIT 许可证 (R-26 要求)
├── scripts/               # 核心脚本
│   ├── skill_audit/       # 审计引擎包 (重构后统一入口)
│   │   ├── __init__.py    # 主入口 + cmd_xxx() + audit_skill() +
│   │   └── _run_audit_loop
│   │       ├── __main__.py # python -m 支持
│   │       └── fix.py      # 自动修复函数 (35+ fix key, 含新增
code_block_markers 等5个)
│   ├── structure_checker.py # R-06~R-09, R-18~R-25 正文结构 + 质量 +
R-26
│   ├── artifact_checker.py  # R-11~R-12 产出物 + 数据目录 (v2.101.8:
支持 pathlib 推导式放行)
│   ├── data_dir_checker.py  # R-22 数据目录合规
│   ├── consistency_checker.py # 一致性审查 (含嵌套目录树路径重建)
│   ├── _path_detector.py    # 共享路径文件检测 (v2.101.8: 优先改选
_paths.py)
│   ├── _tree_scanner.py     # 目录树扫描器 (R-23 辅助)
│   ├── progress_manager.py  # 进度管理器
│   └── utils.py             # 常量定义 (RULES 列表、关键词映射等)
```

```

|—— safe_io.py          # 安全文件写入（原子写入 + 备份 + Windows 重试）
|—— cleanup_manager.py   # Manifest 驱动清理（备份注册 + 收尾清理）
|—— permission_checker.py # 权限检查器（AST 扫描风险操作）
|—— skill_inspector.py   # 结构扫描器（输出技能蓝皮书）
|—— spec/               # 规范定义（JSON Schema）
    |—— body.json         # 正文章节结构规范 v2.6.0
    |—— ...

```

变化说明（v2.82.0 → v2.95.0）：

-  **新增** `--category` 误判类别强制参数（`engine_mistake` / `engine_cant_judge`）
-  **新增** 模式门禁 `--mode` 必传（移除"不传则不校验"向后兼容）
-  **新增** 修复循环 `sys.exit(2)` 强制阻断（不再 `break` 放行）
-  **修复** `body.json` 合法化（移除冗余顶层键，`section_synonyms` 现在正确加载）
-  **修复** C-11 非标章节三层优先级 instruction（重命名→归并子节→拆分）
-  **新增** `section_names` 加入 `_llm_only_fix_keys`
-  **修复** 一致性审查嵌套目录树路径重建（旧版只提取扁平文件名）
-  **修复** C-17/C-18 OMP 硬编码泛化（`0=3/M=6/P=15` → 通用数值参数检查）
-  **修复** `.verify_fp.json` 格式升级（纯 ID 列表 → `{id: {category, reason}}` 字典）
-  **已删除** `body.json` 中冗余顶层键（`"触发条件"/"核心能力"/"工作流程"` 三组重复定义导致 JSON 非法）

1.3 三层章节体系（section_tiers）

SKILL.md 的 `##` 章节分为三个层级，决定其存留行为和拆分优先级：

层级	包含章节	行为
① must_have	H1 / 约束 / 触发条件 / 核心能力 / 工作流程	永远留在 SKILL.md，必须有标题 + 内容
② whitelist.optional_progressive	快速开始 / 强制约束 / 铁律 / 规范 / 反模式 / FAQ / 配置 / API / 示例 / 限制 / 数据目录说明 / 权限说明 / 临时文件与备份管理 / 注意事项	可留，超230行时优先拆到 references/

层级	包含章节	行为
②' whitelist.al ways_progr essive	版本日志 / 更新日志 / Changelog	强制在 references/, SKILL.md 只能有引用 (R-24)
③ nonstandar d	不在①②的所有H2	LLM 按三层优先级处理：内容匹配 must_have → 重命名；部分匹配 → ### 子节；完全非标 → 拆分 references/

非标章节处理优先级（v2.95.0 C-11 增强）：

第1优先 — 内容属于 must_have 章节职责范围？改标题为标准名
第2优先 — 内容属于 whitelist 章节职责范围？改标题为对应标准名
第3优先 — 与某标准章节部分相关？降级为 ### 子节归入该章节
第4优先 — 完全不匹配？拆分到 references/<slug>.md
⚠ must_have 章节不可空不可删
⚠ always_progressive 章节不可出现在 SKILL.md 正文

1.4 能力与限制章节

能力	说明	限制
审计现有 skill	R-01~R-26 全量检查，输出 PASS/WARN/FAIL 逐条明细及上下文行	仅检查 SKILL.md + _meta.json + scripts/ 文件结构和代码静态分析，不检查 Python 运行时行为
创建新 skill	从模板生成标准骨架（含目录结构预览）	只生成结构模板和占位符，功能代码需要手动填充
改造非标 skill	自动迁移文件到正确位置	不处理跨技能依赖、不自动生成功能代码
批量审计	<code>--audit-all</code> 参数扫描多个 skill	仅支持一级子目录（不支持嵌套）

能力	说明	限制
自动修复	<code>--fix</code> 自动修正格式/结构/路径/生成类问题，覆盖 R-01~R-26 共 20+ 条规则	仅修格式/结构/路径/生成类问题， 不修复代码逻辑错误。 修复后需运行 <code>--verify</code> + <code>--show-fix</code> 两阶段验证确认
权限安全扫描	自动检测脚本中的删除/网络/subprocess 调用	基于 AST 静态分析，无法检测动态执行

1.5 六种执行模式（v2.95.0）

模式	命令	作用	风险等级
audit	<code>python -m scripts.skill_audit audit</code> <code>--confirmed --mode audit</code>	独立全量审计	● 只读
create	<code>python -m scripts.skill_audit create --confirmed --mode create</code>	从模板创建标准的 skill 骨架	● 无害
update	<code>python -m scripts.skill_audit update</code> <code>--confirmed --mode update</code>	增量检查 + 可选修复	● 轻度修改
refactor	<code>python -m scripts.skill_audit refactor</code> <code>--confirmed --mode refactor</code>	全流程改造（蓝皮书→备份→审计→修复→验证→bump→清理）	● 有备份保障
bump	<code>python -m scripts.skill_audit bump</code> <code>--type fix --desc "..."</code> <code>confirmed --mode bump</code>	版本号三端同步升级	● 无害

模式	命令	作用	风险等级
read only	<code>python -m scripts.skill_audit rules --confirmed --mode readonly</code>	只读查询（列出规则/模板）	● 只读

语义门禁 + 模式-命令映射锁（v2.94.0+）：所有模式入口必须传 `--confirmed` + `--mode` 参数。`--mode` 值必须与子命令一致（如 `audit --mode audit`），不一致则 `exit(1)` 阻断。不再有“不传则不校验”的向后兼容。

二、完整审查规则体系（R-01 ~ R-26）

2.1 类别 A：Frontmatter 结构（R-01 ~ R-05）

规则	严重程度	检查内容	通过条件
R-01	ERR OR	YAML frontmatter 存在性 + 11 required + 2 conditional 字段 + <code>_meta.json</code> 7 字段完整性	文件以 <code>---</code> 开头并包含闭合 <code>---</code> ；11 required + 2 conditional 字段分层检查； <code>_meta.json</code> 含 7 标准字段
R-02	ERR OR	<code>name</code> 字段	frontmatter 含 <code>name:</code> ，值非空，且与目录名一致
R-03	ERR OR	<code>version</code> 字段（SemVer + 变更语义规则）	值符合纯数字 x.y.z 格式（禁止 v 前缀）
R-04	ERR OR	<code>description</code> 字段	含 <code>description:</code> ，值非空
R-05	WARN	<code>name</code> = 目录名	frontmatter 的 <code>name</code> 与所在目录名一致

2.2 类别 B：正文结构（R-06 ~ R-10）

规则	严重度	检查内容	通过条件
R-06	WARN	一级标题	正文包含 # 开头的 H1 标题
R-07	ERROR	触发条件章节	含触发场景章节，≥3 个触发词，≥1 个否定条件
R-08	WARN	核心能力章节	含核心能力/功能章节
R-09	WARN	工作流程章节	含工作流程/步骤章节
R-10	ERROR	版本三端一致性 + 共享字段同步	SKILL.md version == _meta.json version == changelog 最新版本号

2.3 类别 C：产出物与数据目录（R-11 ~ R-12）

规则	严重度	检查内容	通过条件
R-11	ERROR	产出物路径 + 风险检测	产出物符合 skills/.standardization// 规范
R-12	ERROR	外部数据目录 + 风险检测	路径符合规范，_meta.json 含 data_dir

2.4 类别 D：安全与权限（R-13 ~ R-17）

规则	严重度	检查内容
R-13	WARN	敏感信息访问声明
R-14	WARN	关键位置写入声明
R-15	ERROR	高权限操作风险说明
R-16	WARN	权限权重说明
R-17	ERROR	渐进加载引用（强制）

2.5 类别 E：质量规范（R-18 ~ R-21）

规则	严重度	检查内容
R-18	WARN	反模式具体性（≥2条，含错误做法/正确做法标记）
R-19	WARN	FAQ 有意义性（≥3对 Q&A）
R-20	WARN	写作规范（术语统一、无模糊词、中英文混排空格）
R-21	WARN	渐进式加载说明（固定模板句）

2.6 类别 F：合规与维护（R-22 ~ R-24）

规则	严重度	检查内容
R-22	WARN	数据目录合规
R-23	WARN	文档-代码一致性（含嵌套目录树路径匹配，v2.95.0）
R-24	WARN	更新日志渐进加载

2.7 类别 G：文档写作格式（R-25，含 C-01~C-19 十九项子检查）

编号	级别	检查项	说明
C-01	ERROR	H1 标题格式	必须为 # <技能名>，不得含版本号
C-02	WARN	标题层级	限制在 ## 和 ###
C-03	WARN	表格使用	结构化信息应使用表格展示
C-04	WARN	引用块使用	提示/注意/警告使用 > 引用块
C-05	WARN	列表区分	有序→步骤，无序→选项

编号	级别	检查项	说明
C-06	WAR N	加粗使用	关键术语/约束用 **加粗**
C-07	WAR N	语言标识	代码块应带语言标识
C-08	WAR N	Checklist	操作前自检使用 - []
C-09	WAR N	渐进引用	统一使用 → 详见 references/
C-10	WAR N	空行规范	frontmatter 闭合后 ≤2 空行
C-11	WAR N	章节顺位 + 非标章节处理	section_order 一致性 + 三层优先级 instruction (v2.95.0)
C-12	WAR N	格式合规	章节格式与 content_format 定义一致
C-13	WAR N	渐进式索引表完整性	所有 references/ 文件是否在索引表中列出
C-14	WAR N	workflow 步骤完整性	workflow 步骤是否覆盖实际代码功能
C-15	WAR N	内容冗余检测	索引表的引用 vs 正文直接的引用
C-16	WAR N	references/ 文档过时	磁盘存在但 SKILL.md 未引用
C-17	WAR N	使用示例质量	示例是否包含输入→输出的完整交互 (v2.95.0 泛化修复, 移除 OMP/CPM 硬编码)
C-18	WAR N	能力边界质量	是否量化阈值、说明参数约束、环境要求

编号	级别	检查项	说明
C-19	WARN	错误处理质量	FAQ 是否包含错误修复指导

C-17/C-18 修复建议泛化 (v2.94.0)：原检查逻辑硬编码了 activity-duration-estimation 的 OMP 术语（O=3/M=6/P=15、CPM分析），现改为通用数值参数检查。所有技能不再收到领域不相关的修复建议。

2.8 类别 H：文档声明规范（R-26，含 C-01~C-08 八项子检查）

编号	级别	检查项
C-01	ERROR	references/LICENSE.md 存在
C-02	ERROR	SKILL.md 正文无独立 license 章节
C-03	ERROR	根目录无 LICENSE 文件
C-04	WARN	scripts/ 下无 LICENSE 文件
C-05	WARN	渐进式索引表含 LICENSE.md 引用
C-06	ERROR	references/LICENSE.md 非空
C-07	ERROR	根目录无 README.md
C-08	ERROR	SKILL.md 正文 README 章节拆分

2.9 规则汇总统计

类别	包含规则	ERROR	WARN	目的
A. Frontmatter	R-01~R-05	4	1	技能身份标识 + _meta.json
B. 正文结构	R-06~R-10	3	2	文档结构和质量
C. 产出物与目录	R-11~R-12	2	0	目录隔离和数据安全
D. 安全与权限	R-13~R-17	2	3	权限声明和风险控制

类别	包含规则	ERROR	WARN	目的
E. 质量规范	R-18~R-21	0	4	内容质量和可读性
F. 合规与维护	R-22~R-24	0	3	长期维护一致性
G. 写作格式	R-25	0	1	文档排版建议（19 项子检查）
H. 文档声明	R-26	0	1	LICENSE + README 声明
合计	R-01~R-26	11	15	

三、核心设计原则

D1: 零外部依赖

D2: 铁律验证阻断模式

D3: 信息零遗漏

D4: 渐进式加载

D5: 模板驱动

D6: 备份优先 + Inspect 先读全

D7: Manifest 驱动清理

D8: 脚本级强制（非 AI 自觉）

（详见 SKILL.md 独立章节，此处不再重复）

四、误报分类机制（v2.94.0 重写）

这是 v2.94.0 最大变化。旧版 `--classify` 无类别约束，LLM 可随意标记任何 FAIL 为误判。新版引入两类固定类别。

4.1 两级误报分类

LLM 对每条 FAIL 只能选以下两类之一标记为误判：

类别	含义	适用场景示例
<code>engine_mistake</code>	引擎技术性错误	BOM 导致 frontmatter 正则未匹配、注释被当实际操作、概念图路径被当文件引用、专有名词被当缺空格
<code>engine_cannot_judge</code>	引擎语义不足，LLM 确认后放行	<code>__init__.py</code> 无需列文档树、反模式内容格式引擎没认出但 LLM 确认合规、 <code>body.json</code> 模板预期与实际域差异

4.2 命令格式

```
# ✅ 合法
python -m scripts.skill_audit audit <dir> --classify 42 --category engine_mistake --reason "BOM字符" --mode audit --confirmed
python -m scripts.skill_audit audit <dir> --classify C-stale_doc_ref --category engine_mistake --reason "概念图路径" --mode audit --confirmed

# ❌ 拒绝
--classify 42                                # 缺 --category
--classify 42 --category i_dont_like_it      # 非法类别
```

4.3 一致性审查误报

一致性审查的 `C-missing_doc_ref` 和 `C-stale_doc_ref` 复用同一类别机制。v2.95.0 修复了旧版只提取扁平文件名的 bug，嵌套目录树路径现可正确匹配（`references/antipatterns.md` 不再被误判为根目录的 `antipatterns.md`）。

4.4 向后兼容

旧格式 `.verify_fp.json`（纯 ID 列表）被忽略。每次审计和审查都是全新的一次性任务，不继承历史误判。

五、审计后自动修复体系（fix.py）

5.1 fix key 与 LLM 手动修复分界

修复循环根据 fix key 类型自动分流：

路径	条件	行为
auto-fix	fix key 不在 <code>_llm_only_fix_keys</code> 中	Python 自动执行
LLM 手动	fix key 在 <code>_llm_only_fix_keys</code> 中	阻断等待 LLM 修复
LLM 手动	无 fix key	阻断等待 LLM 修复

`_llm_only_fix_keys` (v2.95.0, 新增 `section_names`)：

```
_BLOCKED_FIX_KEYS = {
    "workflow_completeness", # C-14：需要 LLM 读代码写工作流 → 输出3步指引
    "example_quality",       # C-17：需要 LLM 读代码创建示例 → 输出3步指引
    "capability_boundary",    # C-18：需要 LLM 理解能力边界 → 输出3步指引
    "section_names",         # C-11：非标章节归类需 LLM 判断内容语义
}
```

BLOCKED 的 3 个 key（C-14/17/18）被阻断时输出结构化指引：

- 1. 读取 `scripts/` 下的 Python 代码理解功能
- 2. 按 JSON 模板写入结构化数据文件
- 3. 重新 `--fix` 自动渲染为 SKILL.md 章节

5.2 修复循环退出机制（v2.94.0 重写）

旧版：auto-fix 耗尽后 break 退出循环，LLM 可以跳过剩余项。新版：auto-fix 耗尽后剩余项 > 0 → sys.exit(2) + 创建重构锁，强制阻断。

```
# 旧版（可跳过）
if remaining:
    print("剩余 N 项需 LLM 修复")
    _save_html_report()
break # ← 不管 remaining 是否为空都 break

# 新版（强制阻断）
if remaining:
    _save_remaining_llm(skill_dir, remaining)
    _create_refactor_lock(skill_dir)
    sys.exit(2) # ★ 强制退出，LLM 无法继续
else:
    print("双 0 达成")
    break # 只有 0 remaining 才放行
```

5.3 修复函数清单（35+）

函数	对应规则	修复内容
fix_name	R-01	修复 name 字段
fix_description	R-04	修复 description 字段
fix_version	R-03	修复 version 字段
fix_author	R-02	修复 author 字段
fix_h1 / fix_h1_version / fix_h1_position	R-06	H1 标题处理
fix_section_trigger / fix_section_core / fix_section_workflow	R-07~R-09	章节创建

函数	对应规则	修复内容
<code>fix_progressive_loading</code>	R-21	渐进式加载模板句
<code>fix_antipattern_progressive</code>	R-18	反模式文档
<code>fix_faq_progressive</code>	R-19	FAQ 文档
<code>fix_writing_standards</code>	R-20	写作规范
<code>fix_data_dir_compliance</code>	R-22	数据目录声明
<code>fix_split_nonstandard</code>	R-17	非标章节拆分
<code>fix_section_order</code>	R-25 C-11	章节重排
<code>fix_progressive_index_table</code>	C-13	渐进式索引表
<code>fix_reclassify_section(action)</code>	R-17 Phase 3	通用非标归类
<code>fix_frontmatter_fields</code>	R-01	补全 frontmatter
<code>fix_license_compliance</code>	R-26	LICENSE 规范
<code>apply_consistency_fix</code>	一致性审查	outdated_rule_ref 修复
<code>fix_code_block_markers</code>	R-22(写作标准)	缩进代码块→围栏 ``` 块 (v2.101.7 新增)
<code>fix_list_mixing</code>	C-05	同章节混排列表统一为多数方样式 (v2.101.7 新增)
<code>fix_code_block_lang</code>	C-07	裸 ``` 补语言标识 (v2.101.7 新增)
<code>fix_section_completeness</code>	C-12	空章节补充格式线索 (v2.101.7 新增)
<code>fix_faq_error_handling</code>	C-19	创建/补充 FAQ 错误处理章节 (v2.101.7 新增)

六、关键流程

6.1 refactor 模式完整步骤 (v2.101.8)

[1/8] 蓝皮书扫描 → `inspect_skill(skill_dir)`

[2/8] 备份 → `.zip` 到 `.standardization//backup/`

[3/8] 全量审计 → R-01~R-26 全量跑

[4/8] ★★★ 细碎修复循环 ★★★

auto-fix 修复可自动修复项

└─ 停滞检测 (2 轮无变化 → 清除 fix key)

└─ 剩余 LLM 手动项 → `sys.exit(2)` + 重构锁

└─ LLM 修复后 `--continue`

[5/8] 全量审计确认 → 双 0 验证 + `_llm_only_fix_keys` 二次筛

[6/8] 全量一致性审查 + 修复 → 文档-代码一致性

[7/8] bump (feature) + 报告

[8/9] cleanup → `end_session()` 清理临时文件

→  refactor 全流程完成

6.2 `--verify` 二次筛查指令 (v2.94.0 更新)

★★★ LLM 二次筛查指令 ★★★

判断每条 FAIL 是否属于以下两种误报：

【engine_mistake】 引擎技术性错误

→ `--classify ID --category engine_mistake --reason "..."`

【engine_cant_judge】 引擎能力不足，LLM 确认后放行

```
→ --classify ID --category engine_cant_judge --reason "..."
```

【真问题】 以上两类均不满足 → 必须修复，不得标记为误判

```
→ 记下 #ID，运行 --show-fix ID 获取修复指引
```

七、关键版本变更摘要（v2.82.0 → v2.95.0）

版本	核心变更
2.94 .0	--category 误判类别强制 + --mode 必传 + 修复循环 exit(2) 阻断 + body.json 合法化 + C-11 三层 instruction + 一致性审查嵌套目录修复 + OMP 硬编码泛化
2.95 .0	文档同步更新（--classify --category、--mode 必传、section_names 加入 _llm_only_fix_keys、guide.md 命令示例更新）

02 | semantic-split 架构与规范体系文档

一、系统概览

semantic-split 是一个 **语义拆分与智能规划工具集**，将用户自然语言任务描述拆解为结构化步骤输出，围绕以下闭环运行：



1.1 三层架构

层	组件	职责
表现层	SKILL.md + references/*.md + CLI	人类可读的文档、命令行交互
业务层	semantic_pipeline / pipeline_b / pipeline_a / pipeline_c / json_manager / model_manager / setup_env	三管线调度、结构分析、语义匹配、双视角推理、JSON 管理、模型下载、环境安装
数据层	.standardization/semantic-split/data/ (capabilities/ + rules/ + models/ + model_index.json)	能力级 JSON 模板库、规则级 JSON 模板库、下载的嵌入模型、模型索引

1.2 目录结构

semantic-split/

```
├─ SKILL.md                # 主文件 (≤230行, 渐进式入口)
├─ _meta.json              # 7 字段元数据 (v3.1.1)
├─ references/             # 渐进式文档 (14 个文件)
│   └─ split_rules.md      # 语义拆分规则 (块划分、主语映射、边界情况)
│   └─ task_type_defaults.md # 任务类型默认映射表 (5 类 + 通用默认值)
│   └─ constraint_annotation.md # 约束强度标注规则 (●●● + 隐式升级 + 注意力锚定 + 双方案差异)
│   └─ planning_rules.md   # 串并行规划规则 (原子步骤、关联热度、里程碑、双视角整合、WP 分解)
│   └─ json_schema.md      # 能力级/规则级 JSON 结构规范 (字段表、示例、存储结构)
│   └─ loading_decision_tree.md # 渐进加载决策树 (规则级 → 能力级 → 模型思考完整分支)
│   └─ examples.md         # 使用示例 (语义拆分 + 模板扫描)
│   └─ changelog.md        # 版本更新日志
│   └─ antipatterns.md     # 反模式指南 (4 条)
│   └─ faq.md              # 常见问题与排错
│   └─ permissions.md      # 权限声明 LOW 风险
│   └─ LICENSE.md          # MIT 许可协议
│   └─ attribution.md      # 第三方组件版权归属
│   └─ automation_tasks.md # 自动任务配置列表
├─ scripts/                # 核心脚本 (7 个 Python)
├─ semantic_pipeline.py    # 三管线统一调度入口 + 10 道门禁钩子系统 + 模板库管理
├─ pipeline_b.py          # Pipeline B — 正则结构分析 (零模型依赖)
├─ pipeline_a.py          # Pipeline A — 语义匹配 (bge-small + bge-reranker)
├─ pipeline_c.py          # Pipeline C — 双视角推理上下文构建 + WP 分解
├─ json_manager.py        # JSON 管理 CLI (scan / categorize / generalize / rule-gen / list / create / validate / info)
├─ model_manager.py        # 模型管理 (多源下载 / 校验 / 索引)
└─ setup_env.py           # 环境检测与 pip 依赖安装
```

1.3 数据目录结构

```
skills/.standardization/semantic-split/data/
├── capabilities/                                # 能力级 JSON 模板（自增强闭环自动生成）
│   ├── make_product_ppt_v1.json # 示例：制作产品介绍 PPT 模板
│   └── ...
├── rules/                                       # 规则级 JSON 模板（由同类能力级 ≥5 份凝练生成）
│   └── ...
├── models/                                     # 下载的嵌入模型（model_manager.py 管理）
│   ├── BAAI_bge-small-zh-v1.5/ # 130MB 轻量中文嵌入
│   └── BAAI_bge-reranker-base/ # 556MB 中文 rerank
├── model_index.json                           # 模型路径索引
├── outputs/                                   # 输出缓存
└── .structure_examples.json
```

二、三管线递进调度系统

2.1 Pipeline B：正则结构分析（Type 管线）

核心文件： `pipeline_b.py` | 参考： `references/split_rules.md` + `references/constraint_annotation.md`

零模型依赖：纯 Python `re` 模块实现，无 pip 安装需求。这是三管线中唯一没有外部依赖的管线。

功能	函数	输出	方法
主语识别	<code>_regex_subjects()</code> / <code>extract_subjects()</code>	主语列表（用户/执行者/第三方）	SUBJECT_MAP 查表 + 去重保序
分块	<code>_regex_blocks()</code> / <code>extract_blocks()</code>	语义块列表（每块含主语推断）	以 <code>。！？\n</code> 分句核
约束标注	<code>_regex_constraints()</code> + <code>extract_constraints_and_attention()</code>	约束列表（等级/关键词/领域）	CRITICAL_KW / SOFT_KW / EXAMPLE_KW 正则组

功能	函数	输出	方法
注意力锚定	<code>extract_attention_anchor()</code>	锚定字典 (critical / core / entity / example / resistance)	CORE_VERB_KW + 数量/时间正则
5W2H 提取	<code>_regex_5w2h()</code> / <code>extract_5w2h()</code>	七维字典 (Why/What/Who/Where/When/How/How much + 覆盖率)	7 组独立正则规则
隐式约束升级	<code>_regex_detect_implicit_upgrade()</code>	升级后约束列表 (组织规范/ 法律合规/安全底线)	5 领域正则组匹配
WP 分解	<code>wps_decompose()</code>	工作包列表 (含耗时估算)	ESTIMATE_MAP 查表 + 逗号/顿号拆分子任务

5W2H 七维度正则提取规则

维度	正则关键词	示例命中
Why	为了、目的是、目标是、旨在、想要、想	"为了向客户展示产品"
What	CORE_VERB_KW 动词组	"制作PPT"、"写报告"
Who	SUBJECT_MAP 主语表	"我"→用户、"你"→执行者
When	DATE_KW 时间词正则	"下周"、"周五前"
Where	在\s*({2,10}?) (?:里上中处方)	"在公司"、"在会议上"
How	HOW_KW + 工具提取	"用公司模板"、"通过 ChatGPT"
How much	AMOUNT_KW 数量词	"20页"、"3天"

约束关键词表

强度	正则组	关键词
● 硬约束	CRITICAL_KW	必须、只能、截止、指定、强制、不允许、不得、禁止、一定
● 软约束	SOFT_KW	最好、尽量、如果、建议、通常、希望、可以、不妨
● 示例	EXAMPLE_KW	比如、例如、像、假如、譬如
🚧 阻力	RESISTANCE_KW	但是、不过、担心、怕、难、卡、然而、但

隐式约束升级领域

领域	正则组	升级规则
组织规范	公司模板、品牌、审批、规范、章程、制度、标准、格式、命名	● → ● ●
法律合规	合同、数据保护、审计、合规、法律、条款、协议、版权、隐私	● → ●
安全底线	权限、加密、安全、密码、认证、防火墙、隔离	● → ●
协作依赖	接口、API、交付、依赖、对接、上下游、联调	● + 影响下游标注
个人偏好	颜色、字体、风格、布局、喜欢、偏好	● 维持

Pipeline B 产出：完整的结构化分析结果字典，包含 subjects / blocks / constraints_attention / five_w2h / attention 五大部件，作为 Pipeline C 推理上下文的输入。

2.2 Pipeline A：语义匹配管线（matching pipeline）

核心文件： pipeline_a.py | **参考：** references/json_schema.md + references/loading_decision_tree.md

三层递进匹配：正则 → embedding（bge-small）→ BERT rerank（bge-reranker），逐层升级，在精度和成本间取得平衡。

三层匹配架构

层	方法	模型	条件	最快路径
① 正则层	关键词子串匹配 (tags / name / description / steps)	无	score ≥ 0.8 直接通过	最快 (零模型加载)
② 嵌入层	bge-small-zh-v1.5 编码 → 余弦相似度	92M B	score ≥ 0.6 作为候选	加载需 ~10 秒
③ Rerank 层	bge-reranker-base CrossEncoder 重排序	1.1G B	score ≥ 0.5 最终确认	加载需 ~20 秒

匹配函数详解

函数	职责	输入	输出
<code>match_json(query, json_entries)</code>	主入口：三层递进匹配	用户 query + JSON 条目列表	按分数降序的匹配结果列表
<code>_regex_match(keywords, json_data)</code>	正则层：6 维子串匹配	拆分后的关键词	0.0-1.0 分数
<code>_embedding_match(query, json_data)</code>	嵌入层：余弦相似度	query + JSON 字符串	0.0-1.0 分数
<code>_rerank_match(query, candidates)</code>	Rerank 层：CrossEncoder top-10 重排	query + 候选列表	带分数重排序结果
<code>classify_constraint(text)</code>	约束强度三分类 (正则 + embedding 补充)	单句文本	{level, method, ...}
<code>generalize_actions(actions)</code>	操作通用化 (替换具体为占位符)	操作列表	通用化后的操作列表
<code>condense_rules(all_steps, threshold)</code>	步骤聚类压缩 (正则 → embedding 递进)	步骤列表	聚类后的 condensed_steps

函数	职责	输入	输出
<code>should_load_json(query, json_entry)</code>	渐进加载命中判定（三层递进）	query + 单 JSON 条目	{hit, score, layer}

正则层匹配权重分配

匹配维度	权重	条件
tags 命中	+0.15/条	kw in tag or tag in kw
name 命中	+0.20/条	kw in name
description 命中	+0.10/条	kw in desc
steps 命中	+0.05/步	kw in step(name+action)
最高分	1.0	超过即截断

模型懒加载策略

```
# pipeline_a.py 中的懒加载模式
_EMBEDDER = None
_RERANKER = None

def _load_embedder():
    # 仅在 embedding 层被需要时加载
    # CPU 推理, 设置 CUDA_VISIBLE_DEVICES=-1 禁用 GPU
    # 模型路径从 MODELS_DIR 自动查找

def _load_reranker():
    # 仅在 rerank 层被需要时加载
    # 同 CPU 推理, CrossEncoder 实例化
```

模型查找路径: `MODELS_DIR / "BAAI_bge-small-zh-v1.5"` → 别名匹配 (`bge-small-zh-v1.5`) → 返回 None

Pipeline A 产出: json_matches 匹配结果列表 + constraint 约束分类 + pipeline_layers 使用记录 (regex / embedding / rerank)。

2.3 Pipeline C：双视角推理管线（reasoning pipeline）

核心文件： `pipeline_c.py` | **参考：** `references/planning_rules.md`（第六节：双视角推理与整合规则）

工作方式： Python 构建推理上下文 → 智能体原生推理（LLM） → 步骤回填。Pipeline C 本身不执行推理，而是构建结构化的推理上下文引导智能体完成三阶段思考。

三阶段双视角推理

[阶段1] 聚焦推理（focus_reasoning）

职责：生成保守、安全、可执行的聚焦方案

规则：

- 每个 5W2H 维度取最窄/最保守/最直接的值
- 严格遵守所有 ● 硬约束
- 只使用已验证的低成本方法
- 每个步骤 ≤30 分钟可完成
- 输出 JSON 步骤列表（含

`name/action/milestone/depends_on/parallel_group/dependency_heat`）

[阶段2] 发散推理（divergent_reasoning）

职责：生成创新、大胆的发散方案

规则：

- 每个 5W2H 维度取最宽/最大胆/最间接的值
- ● 硬约束可轻微突破，但必须标注风险并提供备用方案
- 引入至少一个非惯用工具或方法
- 包含"如果无限资源会怎么做"变体
- 输出 JSON 步骤列表 + 创新点列表

[阶段3] 整合推理（integration_reasoning）

职责：将聚焦方案（骨架）+ 发散方案（创新点）整合为单一方案

规则：

- 以聚焦方案为骨架（步骤顺序 = 聚焦方案）
- 发散创新点通过语义相似度嵌入对应聚焦步骤
- 所有创新步骤标注「🌟 增强，来源：发散方案」
- 涉及 ● 硬约束突破的标注 ⚠️ 风险
- 输出单一的整合后 JSON 步骤列表

推理上下文构建

`build_reasoning_context()` 是 Pipeline C 的核心函数，构建包含以下内容的推理上下文：

上下文组件	来源	用途
5W2H 任务描述	Pipeline B <code>five_w2h</code>	提供七维任务全貌
约束清单	Pipeline B <code>constraints_attention</code>	提供 ● ● ● 约束优先级
结构分析增强	Pipeline B 动词/实体/NER 提取	增强 LLM 思考深度
模板参考	Pipeline A <code>matches</code> （命中 ≥ 0.6 才提供）	few-shot 参考，提升推理质量
聚焦 Prompt	<code>FOCUS_SYSTEM</code> 常数	引导保守思考
发散 Prompt	<code>DIVERGENT_SYSTEM</code> 常数	引导创新思考
整合 Prompt	<code>INTEGRATION_SYSTEM</code> 常数	引导双方案整合

步骤解析（降级策略）

`parse_steps_from_agent()` 提供三级降级解析：

级别	尝试	方法
1	直接 JSON 解析	<code>json.loads()</code> → list 或 dict 提取
2	代码块提取	从 ``json`` 区块提取
3	空列表回退	返回 []

三、10 道门禁钩子系统

核心文件： `semantic_pipeline.py` （第 37-57 行）

10 道钩子覆盖从输入校验到 WP 分解的完整流程，每道钩子独立记录状态和详情。所有钩子通过后，最终输出才被视为完整。

3.1 钩子定义与触发时机

[钩子1] input_valid → 输入校验通过（非空、≤2000 字）

[钩子2] b_pipeline_done → Pipeline B 结构分析完成

[钩子3] a_scan_done → Pipeline A 模板扫描完成（命中/未命中）

[钩子4] decision_made → 渐进加载决策（命中模板 / 未命中走语义）

[钩子5] llm_generated → 智能体推理启动/跳过

[钩子6] focus_reasoning → 聚焦方案推理（双视角第一步）

[钩子7] divergent_reasoning → 发散方案推理（双视角第二步）

[钩子8] integration_reasoning → 整合方案推理（双视角第三步）

[钩子9] template_saved → 自动保存能力级 JSON 到 capabilities/

[钩子10] wp_done → WP 工作包分解完成

3.2 钩子状态机

```
HOOK_STATUS = {  
    "input_valid": False,          # 钩子1  
    "b_pipeline_done": False,      # 钩子2  
    "a_scan_done": False,          # 钩子3  
    "decision_made": False,        # 钩子4  
    "llm_generated": False,        # 钩子5  
    "focus_reasoning": False,     # 钩子6  
    "divergent_reasoning": False,  # 钩子7  
    "integration_reasoning": False, # 钩子8  
    "template_saved": False,       # 钩子9  
    "wp_done": False,              # 钩子10  
}
```

所有钩子初始化为 `False`，每个钩子在其对应操作完成后通过 `_hook()` 函数标记为 `True`。`_hook()` 同时写入 `HOOK_LOG` 列表，记录钩子名称、状态标记和详情文本。

3.3 三种钩子行为模式

模式	对应钩子	行为
阻断	input_valid	空输入直接返回 error，不走管线
分支	decision_made	命中模板 → 跳管线 C；未命中 → 走 Pipeline A 语义 + Pipeline C 推理
强制顺序	钩子 6/7/8	双视角推理子步骤按 focus → diverge → integrate 强制顺序执行

四、自增强闭环

核心机制： semantic_pipeline.py 中的 _save_template() + _scan_matching() + _load_all_capabilities() + match_json()

4.1 闭环流程



4.2 三步闭环机制

步骤	操作	触发条件	产出
① 保存	_save_template() 将当前步骤通用化写入 JSON	Pipeline C 推理完成且产生步骤	capabilities/{task}_v1.json

步骤	操作	触发条件	产出
② 命中	<code>_scan_matching()</code> 扫描模板库	每次处理新任务	匹配结果列表
③ 凝练	<code>json_manager.py rule-gen</code> 将同类能力级压缩为规则级	同类能力级 ≥ 5 份（用户主动触发）	<code>rules/rule_{cat}_v1.json</code>

4.3 模板保存的通用化规则

`_save_template()` 调用 `pipeline_a.generalize_actions()` 将具体操作泛化：

原始	通用化后
"介绍钛合金马扎"	"介绍[产品名称]"
"明天下午3点前交付"	"在[截止时间]前交付"
"导出为微信公众号格式"	"导出为[目标格式/平台]"

通用化步骤保留结构（步骤数量、并行组、milestone、dependency_heat、depends_on），仅替换具体值为占位符。

文件名生成规则

```
task_name = re.sub(r'^(请|帮|帮我|我想|我需要|麻烦)', '', text.strip())[:20]
clean_name = re.sub(r'^[\u4e00-\u9fff\w]', '', task_name)[:16]
# 输出：capabilities/{clean_name}_v1.json
```

五、渐进加载决策树

核心文件：`references/loading_decision_tree.md` | 实现：

```
semantic_pipeline.py + pipeline_a.py should_load_json() +
json_manager.py cmd_scan()
```

5.1 三阶段决策

Phase ①: 规则级扫描

- └ 完全命中 → 用 condensed_steps 生成规划 → 询问用户
- └ 部分命中 → 规则级前N步 + 渐进加载能力级补充
- └ 粗粒度命中 → load_capability_if_detail_needed 加载细节
- └ 未命中 → Phase ②

Phase ②: 能力级扫描

- └ 命中 → 直接复用/拼装 → 询问用户
- └ 未命中 → Phase ③

Phase ③: 模型思考

- └ Pipeline C 双视角推理 → 生成步骤 → 询问用户
- 用户确认执行 → 执行完成后 → _save_template()

5.2 匹配阈值

层级	通过阈值	行为
规则级完全匹配	≥0.6	直接使用 condensed_steps
规则级部分匹配	前 N 步适用	加载规则级前 N 步 + 容量级补充
能力级完全匹配	≥0.6	直接使用能力级 steps
能力级组合匹配	多个拼装覆盖	多个能力级拼装
能力级不命中	<0.6	作为 few-shot 参考喂给智能体

5.3 脚本辅助匹配

```
# ① 扫描规则级
python scripts/json_manager.py scan --keywords <任务关键词> --type rule --top
3
```

② 规则级不命中时，扫描能力级

```
python scripts/json_manager.py scan --keywords <任务关键词> --type capability  
--top 5
```

③ 均不命中时，检查归类统计是否需要凝练规则级

```
python scripts/json_manager.py categorize --threshold 5
```

六、JSON 模板管理体系

核心文件: `json_manager.py` | 参考: `references/json_schema.md`

6.1 双层 JSON 体系

规则级 (rule) —— 上层抽象，粒度粗

- | 含 condensed_steps (压缩步骤) + capability_refs (来源引用)
- | load_capability_if_detail_needed 渐进加载指示
- |
- | 通过 rule-gen 子命令从同类能力级 ≥ 5 份凝练生成
- |

能力级 (capability) —— 原子单元，粒度细

含 steps (完整步骤列表) + generic_params (通用化占位符)

通过 _save_template() 自动保存

6.2 能力级 JSON 结构

```
{  
  "id": "make_product_ppt_v1",  
  "type": "capability",  
  "name": "制作产品介绍 PPT",  
  "version": "1.0.0",  
  "created_at": "2026-05-22",  
  "description": "从零制作一份产品介绍 PPT 的完整步骤",  
  "generic_params": ["[产品名称]", "[核心参数]", "[截止时间]"],  
}
```

```

"steps": [
  {
    "id": "s1", "name": "收集资料",
    "action": "搜索/整理[产品名称]相关资料和[核心参数]",
    "parallel_group": null, "milestone": true,
    "dependency_heat": 0, "depends_on": [],
    "constraint_level": "none", "source": "focus"
  }
],
"tags": ["ppt", "内容制作", "产品介绍"]
}

```

6.3 规则级 JSON 结构

```

{
  "id": "rule_ppt_v1",
  "type": "rule",
  "name": "PPT 类任务规则",
  "version": "1.0.0",
  "created_at": "2026-05-22",
  "description": "统合所有 PPT 相关能力级 json",
  "source_capability_count": 5,
  "capability_refs": [
    { "id": "make_product_ppt_v1", "name": "制作产品介绍 PPT" }
  ],
  "condensed_steps": [
    {
      "id": "r1", "name": "资料与大纲",
      "milestone": true, "parallel_group": null,
      "maps_to": ["make_product_ppt_v1.s1", "make_product_ppt_v1.s2"],
      "load_capability_if_detail_needed": "make_product_ppt_v1"
    }
  ],
  "tags": ["ppt", "规则汇总"]
}

```

6.4 json_manager CLI 子命令

子命令	功能	核心参数	适用场景
scan	关键词扫描匹配	--keywords , --type , --threshold , --top	渐进加载决策的①②阶段
categorize	归类统计（按 tag 分组）	--threshold （默认 5）	判断是否达到规则级凝练阈值
generalize	通用化字段替换	--input , --params , --auto , --output	保存模板前的通用化步骤
rule-gen	生成规则级 JSON 框架	--files / --tag , --output	同类能力级 ≥5 份后凝练
list	列出所有 JSON 文件	--type , --tag , --verbose	模板库概览
create	创建新 JSON 骨架	--type , --name , --output	手动创建新模板
validate	验证 JSON 格式	--file	字段完整性验证
info	显示 JSON 详情	--file	查看单条模板详细信息

七、模型管理

核心文件： model_manager.py | 参考： references/attribution.md

7.1 模型清单

模型	管线	大小	类型	协议
BAAI/bge-small-zh-v1.5	Pipeline A 嵌入层	92MB	embedding	MIT
BAAI/bge-reranker-base	Pipeline A 重排序层	556MB	CrossEncoder	MIT
合计		~648MB	全部商业友好	

Pipeline B 为纯正则实现，零模型依赖。v3.0.1 删除了 Stanza (1.1GB)，从 3 个模型减为 2 个。

7.2 多源递进下载

优先级	源名称	方法	说明
1	modelscope	<code>snapshot_download()</code>	ModelScope 国内镜像（推荐）
2	hf_mirror	<code>snapshot_download()</code> + <code>HF_ENDPOINT</code>	HuggingFace 国内镜像
3	hf_official	<code>snapshot_download()</code>	HuggingFace 官方源
4	hf_direct	<code>hf_hub_download()</code> 逐文件	最稳定，避免子进程死锁

每源最多 2 次重试，下载后自动完整性检查和路径索引。

7.3 模型路径查找

`_find_actual_model_path()` 使用内容感知方式查找模型路径：

```
# 先查 MODELS_DIR (按名称相似度)
for d in os.listdir(MODELS_DIR):
    score = _name_similarity(_normalize(target), _normalize(d))
    if score > best_score and _is_model_dir(dp):
        best_score, best = score, dp

# 未找到则查 HF 缓存
safe_id = f"models--{model_id.replace('/', '--')}"
snap_dir = CACHE_DIR / safe_id / "snapshots"
```

八、核心设计原则

D1: Pipeline B 零外部依赖

Pipeline B（正则结构分析）仅使用 Python 标准库 `re` + `pathlib`，零 `pip install` 需求。5W2H 提取、主语识别、约束标注、注意力锚定全部用正则实现。

目的：最常用的结构分析管线即装即用，无需下载模型。

D2: 三层递进匹配（成本最优）

Pipeline A 的三层匹配（正则 → embedding → rerank）在精度和成本间渐近升级：

正则层（零成本，毫秒级） → 不够再加 embedding（百毫秒级） → 还不够再加 rerank（秒级）

目的：85%+ 的匹配在正则层解决，避免不必要的模型加载。

D3: 自增强闭环

每完成一次任务自动生成能力级 JSON 模板，下次同类任务避免重复推理。多份能力级可凝练为规则级，实现从零到库的知识积累。

目的：系统越用越不用思考，LLM 调用量随使用递减。

D4: 双视角推理强制

Pipeline C 强制走聚焦 → 发散 → 整合三阶段推理，10 道门禁的钩子 6/7/8 分别约束三个阶段。

目的：避免单一视角偏倚，聚焦方案保证可执行性，发散方案提供创新灵感。

D5: 渐进加载

规则级 → 能力级 → 模型思考，只有上层不命中时才降级到下层。命中即用，0 LLM。

目的：LLM 调用量化化，典型任务 80% 可在规则级/能力级层解决。

D6: 松耦合模块化

4 个管线脚本（pipeline_b / pipeline_a / pipeline_c / semantic_pipeline）+ 3 个辅助脚本（json_manager / model_manager / setup_env）各自独立，通过函数导入和 JSON 数据交换耦合。

目的：每个管线可独立升级/替换，不影响其他模块。

D7: JSON 通用化 ≠ 抽象

能力级 JSON 的通用化是字段替换（具体值 → 占位符），不是抽象提炼。步骤粒度、结构、并行关系全部保留。

目的：通用化后仍保持精确的可执行性，不会丢失步骤细节。

D8: 原子写入安全

所有 JSON 文件写入使用 `.tmp` + `os.replace()` 原子操作模式，防止写入过程中断导致文件损坏。

九、交互方式

9.1 主入口命令行

```
# 完整流程（含智能体推理）
python scripts/semantic_pipeline.py --text "帮我用公司模板做一份PPT，下周五前交给客户"

# JSON 格式输出
python scripts/semantic_pipeline.py --text "..." --json

# 完整中间结果（含所有管线数据）
python scripts/semantic_pipeline.py --text "..." --json --full
```

```
# 跳过智能体推理（返回骨架）
python scripts/semantic_pipeline.py --text "..." --skip-llm

# 显示门禁钩子状态
python scripts/semantic_pipeline.py --text "..." --hooks

# 从文件读取输入
python scripts/semantic_pipeline.py --file input.txt --json
```

9.2 JSON 管理命令行

子命令	完整命令示例
scan	<code>python scripts/json_manager.py scan --keywords ppt 制作 --top 5</code>
categorize	<code>python scripts/json_manager.py categorize --threshold 5</code>
generalize	<code>python scripts/json_manager.py generalize --input xxx.json --params "钛合金马扎=[产品名称]"</code>
rule-gen	<code>python scripts/json_manager.py rule-gen --tag ppt --output rule_ppt_v1.json</code>
list	<code>python scripts/json_manager.py list --type capability --verbose</code>
create	<code>python scripts/json_manager.py create --type capability --name make_report_v1</code>
validate	<code>python scripts/json_manager.py validate --file xxx.json</code>
info	<code>python scripts/json_manager.py info --file xxx.json</code>

9.3 模型管理命令行

```
# 下载所有模型（建议仅 Pipeline A）
python scripts/model_manager.py --download-all

# 下载指定模型
python scripts/model_manager.py --download bge-small
```

```
python scripts/model_manager.py --download bge-rerank
```

```
# 列出已下载模型
```

```
python scripts/model_manager.py --list
```

```
# 验证所有模型完整性
```

```
python scripts/model_manager.py --verify-all
```

9.4 环境安装

```
# 国内推荐（阿里云镜像）
```

```
python scripts/setup_env.py --auto-install --mirror aliyun
```

```
# 仅检测
```

```
python scripts/setup_env.py --check-only
```

```
# 清华镜像
```

```
python scripts/setup_env.py --auto-install --mirror tsinghua
```

十、外部依赖

包	用途	必需?	对应管线
sentence-transformers	bge embedding + CrossEncoder	是	Pipeline A
huggingface-hub	模型下载 (hf_hub_download)	是	model_manager
modelscope	ModelScope 国内下载	可选	model_manager
torch	PyTorch 后端 (sentence-transformers 依赖)	推荐	Pipeline A

Pipeline B 零依赖：纯 Python 标准库实现，即装即用。 **Pipeline C 零依赖**：推理上下文构建使用 Python 标准库，推理由智能体原生完成。

十一、版本历史

版本	日期	核心变化
2.1.0	2026-05-22	初始标准化版本：skill-standardization 引擎创建
2.2.0	2026-05-23	R-11 产出物路径修正： <code>data/</code> → <code>.standardization/semantic-split/data/</code>
2.3.0	2026-05-23	DATA_DIR 路径修正 + <code>_meta.json</code> <code>data_dir</code> 字段
2.4.0	2026-05-23	5W2H 维度提取 + 约束标注 + 双视角推理整合
2.4.1	2026-05-23	SKILL.md 瘦身 (315→200行) + 渐进式 MD 体系完善
2.5.0	2026-05-23	标准化改造：章节重排、示例更新、路径修正
2.6.0	2026-06-30	refactor 全流程：0 ERROR 0 WARN
3.0.0	2026-07-01	三管线架构落地： <code>pipeline_a/b/c</code> + <code>semantic_pipeline</code> + <code>model_manager</code> + <code>setup_env</code> + 自增强闭环 + 10 道钩子
3.0.1	2026-07-01	删除 Stanza (1.1GB)，Pipeline B 纯正则化 + 钩子系统 7→10 道
3.0.2	2026-07-01	双视角推理门禁强制化 + 清理所有 spaCy 残留
3.1.0	2026-07-01	修复 <code>pipeline_b/c</code> 死代码 + <code>model_manager</code> 索引错误 + 自增强闭环接入钩子

版本	日期	核心变化
3.1.1	2026-07-01	渐进式索引表恢复完整（R-25 fix 吞列 bug）

十二、数据流全景



| └─ 不命中 → Pipeline C 双视角推理



Pipeline C — 双视角推理（智能体原生）

- | └─ [钩子6] 聚焦方案：保守骨架
- | └─ [钩子7] 发散方案：创新点
- | └─ [钩子8] 整合方案：骨架+创新点融合
- | └─ [钩子9] 保存模板 → capabilities/{task}_v1.json
- | └─ [钩子10] WP分解 → 工作包列表



输出 JSON (steps + wps + hooks + pipeline_summary)

03 | activity-duration-estimation 架构与规范体系文档

一、系统概览

activity-duration-estimation 是一个 **全周期项目管理工具集**，围绕以下闭环运行：



1.1 四层架构

层	组件	职责
表现层	SKILL.md + references/*.md + CLI + HTML 报表 + 设置面板	人类可读的文档、命令行交互、可视化报表和配置
业务层	analysis_engine / wbs_engine / project_docs_engine / economic_analysis_engine / evm_engine / runner	估算、分解、文档、分析的核心逻辑

层	组件	职责
数据层	project_knowledge / economic_knowledge / evm_knowledge (三库隔离) + knowledge_schema	SQLite 知识库 + 标准化接口
配置层	settings_manager / settings_server	运行时设置 + HTML 可视化面板

1.2 目录结构

```

activity-duration-estimation/
├── SKILL.md                # 主文件 (≤230行, 渐进式入口)
├── _meta.json              # 7 字段元数据
├── references/              # 渐进式文档 (15 个文件)
│   ├── methods.md          # 四种估算方法详解
│   ├── wbs-methodology.md  # WBS 分解方法论
│   ├── semantic-analysis.md # 语义分析架构
│   ├── project-docs-methodology.md # 项目文档生成方法论
│   ├── economic-analysis-methodology.md # 经济效益分析方法论
│   ├── evm-methodology.md  # 挣值管理方法论
│   ├── risk-dimensions.md  # 7 类风险维度
│   ├── search-integration.md # 外部知识搜索集成
│   ├── knowledge-interface.md # 知识库接口设计
│   ├── thinking-tools.md   # 12 种思维工具
│   ├── report-template.md  # 报表模板
│   ├── antipatterns.md     # 反模式
│   ├── faq.md              # 常见问题
│   ├── changelog.md        # 版本更新日志
│   └── templates/          # JSON 文档模板
│       ├── 立项申请书.json # 11 节 P0 模板
│       ├── 结项报告书.json  # 10 节 P0 模板
│       ├── 相关方登记册.json # 4 节 P0 模板
│       └── 风险登记册.json   # 5 节 P0 模板
└── scripts/                # 核心脚本 (14 个 Python + 4 个 HTML 模板)
    ├── runner.py           # 全流程编排层: PipelineState +
run_pipeline()
    ├── analysis_engine.py   # 估算核心引擎: CPM / MC / 重叠分析 / 甘特图 SVG
    └── wbs_engine.py        # WBS 分解引擎: 4 方法 / 3 模板 / 多格式输出

```

```

├── project_docs_engine.py # 项目文档生成引擎：双模式 / 12 思维工具
├── economic_analysis_engine.py # 经济效益分析引擎：ROI/NPV/IRR/BCR/PBP
├── evm_engine.py          # 挣值管理引擎：PV/EV/AC/SPI/CPI/EAC
├── knowledge_schema.py    # 知识库标准化接口：YAML+MD 格式 / 字段映射
├── project_knowledge.py   # shared.db 引擎：SQLite+FTS5 查询/写入/外部对
接
├── economic_knowledge.py  # economic.db 引擎：经济效益独立知识库
├── evm_knowledge.py       # evm.db 引擎：挣值管理独立知识库
├── risk_dimensions.py     # 7 类风险维度匹配引擎（D1-D7）
├── settings_manager.py    # 全局设置 CRUD + CLI 交互
├── settings_server.py     # 设置可视化面板 HTTP 服务器
├── full_test.py           # 全功能测试套件（100/100 PASS）
├── templates/             # HTML 报表模板
│   ├── report-template.html # 主报表模板
│   ├── economic-report.html # 经济效益报表模板
│   ├── evm-report.html     # 挣值管理报表模板
│   └── settings.html       # 设置面板模板

```

二、七大子技能体系

⚡ **统一入口：** `runner.py` 提供 `run_full()` 全流程执行，`run_pipeline()` 分阶段执行。子技能可独立运行：`python scripts/analysis_engine.py --wbs "xxx"`、`python scripts/economic_analysis_engine.py --roi "yyy"`

2.1 子技能 A：语义分析与方法推荐

核心文件： `analysis_engine.py`

功能	实现	输出
5W2H 参数提取	正则 + LLM 双通道提取	结构化的 7 维参数表
三级约束标注	约束强度检测（必须 > 建议 > 可选）	约束矩阵
任务类型分类	6 类（软件/建筑/制造/科研/农业/活动）	类型标记
估算方法推荐	任务类型 → 自动匹配最优方法组合	方法建议 + 置信度

外部知识搜索（两阶段流程）：

- 1. LLM 自判：LLM 判断是否需要外部知识补充
- 2. 搜索补充：WebSearch 获取领域参数 → 融入参数表

2.2 子技能 B：WBS 工作分解

核心文件： `wbs_engine.py` | 参考： `references/wbs-methodology.md`

功能	说明
4 种分解方法	交付成果式 / 生命周期式 / 模块组件式 / 职能领域式
3 个参考模板	LLM 自适应填充
100% 规则验证	启发式检查父子节点覆盖度
4 种输出格式	文本树 / Markdown / JSON / SVG 甘特图

关键类：

类/函数	说明
<code>WBSNode</code>	WBS 节点：name / level / description / deliverables / criteria / children
<code>WBSResult</code>	WBS 结果：root_nodes / coverage_ratio / validation_warnings
<code>build_node_tree(tasks, method)</code>	按指定方法构建 WBS 树
<code>validate_coverage(wbs_result)</code>	100% 规则验证

2.3 子技能 C：活动历时估算（四种方法）

核心文件： `analysis_engine.py` | 参考： `references/methods.md`

四种估算方法

方法	公式	特征	适用场景
直接估算	$(O+M+P)/3$	简单平均，适合数据充分	历史数据丰富的任务
β分布/PERT	$(O+4M+P)/6$	加权平均，偏向最可能值	软件、研发类任务
正态分布	以 M 为均值，P-O 为范围	标准差量化不确定性	重复性任务
蒙特卡洛模拟	2000+ 次采样（PERT-Beta / 三角 / 泊松并行）	概率分布 + 置信区间	高风险、高不确定性任务

CPM 关键路径分析

参数	含义
ES / EF	最早开始时间 / 最早结束时间
LS / LF	最晚开始时间 / 最晚结束时间
TF（总浮动）	不影响总工期的最大延迟
FF（自由浮动）	不影响后续任务的最大延迟

四种依赖类型：FS（完成-开始） / SS（开始-开始） / FF（完成-完成） / SF（开始-完成）

任务重叠分析

使用扫描线算法处理并行任务的重叠关系，输出视觉化的甘特图 SVG。

2.4 子技能 D：项目文档生成

核心文件： `project_docs_engine.py` | 参考： `references/project-docs-methodology.md`

双模式生成

模式	行为	适用场景
手动模式	输出空模板（占位符），用户自行填充	仅需框架
混合模式	按章节设 auto / outline / manual	部分内容需人工确认
全自动模式	所有节 auto，LLM 完整填充	快速产出

4 个 P0 模板

模板	节数	视图
立项申请书	11	模板填充 + 思维工具嵌入
结项报告书	10	模板填充 + 自动摘要
相关方登记册	4	表格生成
风险登记册	5	风险表格 + D1-D7 维度匹配

12 个思维工具嵌入

SWOT / SMART / PDCA / RACI / 5W1H / 5 Whys / 鱼骨图 / Pareto / Kano / Gantt / PERT / Decision Matrix

2.5 子技能 E：经济效益分析

核心文件： `economic_analysis_engine.py` | 参考： `references/economic-analysis-methodology.md`

指标	说明	计算公式
ROI	投资回报率	$(\text{收益} - \text{成本}) / \text{成本} \times 100\%$
NPV	净现值	$\sum(CF_t / (1+r)^t) - \text{初始投资}$
IRR	内部收益率	NPV = 0 时的折现率（牛顿迭代法）
BCR	效益成本比	总收益现值 / 总成本现值

指标	说明	计算公式
PBP	回收期	累计现金流转正的年数

输出：自包含 HTML 报表（Chart.js 图表），含逐年现金流表、多折现率对比。

2.6 子技能 F：挣值管理

核心文件： `evm_engine.py` | 参考： `references/evm-methodology.md`

参数	含义	公式
PV	计划值	计划完成工作量 × 预算单价
EV	挣值	实际完成工作量 × 预算单价
AC	实际成本	实际花费
SPI	进度绩效指数	EV / PV
CPI	成本绩效指数	EV / AC
EAC	完工估算	BAC / CPI

双模式：不纠偏模式（ $EAC = AC + (BAC - EV)$ ） / 纠偏模式（ $EAC = AC + (BAC - EV) / CPI$ ）

输出：自包含 HTML 报表（SPI/CPI 趋势图 + 预测曲线）。

2.7 子技能 G：三库隔离知识库

核心文件： `project_knowledge.py` / `economic_knowledge.py` / `evm_knowledge.py` | 参考： `references/knowledge-interface.md`

三库架构

```

activity-duration-estimation/
├── data/
│   ├── shared.db      (project_knowledge: 共享知识库 - SQLite FTS5)
│   ├── economic.db    (economic_knowledge: 经济效益知识库)
│   └── evm.db          (evm_knowledge: 挣值管理知识库)

```

知识条目格式 (YAML + Markdown)

```
---
id: "ade-001"
type: "industry_standard"
source: "PMBOK_6th"
tags: [dur_est, pert, cpm]
---

## 三点估算公式

β分布 (PERT) : ( O + 4M + P ) / 6
```

标准化接口 (knowledge_schema.py):

函数	说明
entry_to_markdown(entry)	知识条目 → Markdown 文档
parse_markdown_entry(md_text)	Markdown → 知识条目
query_knowledge(db_path, query)	跨库查询
skill_registry(query)	自动判断查询目标库

三、全流程编排

3.1 PipelineState

runner.py 的核心状态机:

```
@dataclass
class PipelineState:
    phase: str                # 当前阶段: parse / wbs / estimate / cpm /
    report / doc
    task_description: str     # 原始任务描述
    parsed_params: dict      # 5W2H 解析结果
```

```
wbs_result: Optional[WBSResult]      # WBS 分解结果
estimation_result: Optional[dict]    # 估算结果
cpm_result: Optional[dict]           # CPM 分析结果
report_html: Optional[str]           # HTML 报表
docs: Optional[dict]                 # 项目文档
errors: List[str]                    # 错误日志
```

3.2 6 阶段执行顺序

```
Phase 1: parse      — 语义分析 + 5W2H 提取 + 任务类型分类 + 方法推荐
Phase 2: wbs        — WBS 分解 (4 方法选一 + 100% 规则验证)
Phase 3: estimate   — 四点估算 /  $\beta$ 分布 / 正态 / 蒙特卡洛并行
Phase 4: cpm        — CPM 关键路径 + 重叠分析 + 甘特图 SVG
Phase 5: report     — 自包含 HTML 报表生成
Phase 6: docs       — 项目文档生成 (混合模式概率最大)
```

3.3 LLM 交互点异常

`LLMInteractionRequired` 异常在以下场景被触发，通知 LLM 参与：

- `parse` 阶段：5W2H 参数不完整 → 询问用户补充
- `wbs` 阶段：分解方法不确定 → 询问用户偏好
- `estimate` 阶段：参数缺失 → 询问用户或搜索外部知识
- `docs` 阶段：模板选择 → 询问用户

四、核心设计原则

D1: 零外部依赖

所有脚本仅使用 Python 标准库 + SQLite，零 pip install。 **目的：**降低安装门槛，即装即用。

D2: 子技能隔离

七个子技能各自维护独立代码和知识库，通过 `runner.py` 统一调度。 **目的：**每个子技能可独立使用，也可组合成全流程。

D3: 自包含输出

所有报表（经济效益分析、挣值管理、主报告）均为自包含 HTML，无外部依赖。**目的：**无需部署服务器，双击 HTML 即可查看。

D4: 知识库即文档

`references/knowledge-interface.md` 定义了 YAML+MD 双格式标准，知识条目可读、可写、可导出。**目的：**知识库不仅是存储，也是人类可读的参考文档。

D5: 三阶段文档生成

手动 → 混合 → 全自动，覆盖从"仅框架"到"完整产出"全 spectrum。**目的：**适应不同用户对文档完成度的需求差异。

五、配置体系

`settings_manager.py` 统一管理，JSON 格式持久化，支持 CLI 和 HTML 面板两种修改方式。

配置项	默认值	说明
<code>web_search_mode</code>	auto	外部知识搜索模式（auto / manual）
<code>kb_collect_mode</code>	auto	知识库采集模式（auto / manual）
<code>kb_query_mode</code>	auto	知识库查询模式（auto / manual）
<code>doc_template</code>	null	项目文档模板选择
<code>doc_write_mode</code>	auto	文档写入模式（auto / manual / template）

HTML 面板支持实时修改并保存。

六、配置体系

(本节延续上一节标题的编号，实为同一节)

settings_manager.py 支持以下交互方式：

方式	命令
CLI 查看	<code>python scripts/settings_manager.py --show</code>
CLI 设置	<code>python scripts/settings_manager.py --set web_search_mode>manual</code>
HTML 面板	启动服务 <code>python scripts/settings_server.py</code> → 浏览器打开 localhost 端口

七、风险维度体系

核心文件： `risk_dimensions.py` | 参考： `references/risk-dimensions.md`

7 类风险维度（D1-D7）：

维度	范围	输出
D1 技术风险	技术实现不确定性	概率 × 影响矩阵
D2 进度风险	工期延误可能性	关键路径浮动预警
D3 成本风险	预算超支概率	蒙特卡洛成本分布
D4 资源风险	人力/设备短缺	资源负荷热力图
D5 质量风险	质量不达标	缺陷预测
D6 外部风险	供应链/政策变化	情景树
D7 管理风险	沟通/决策失效	RACI 缺口

八、完整测试

`full_test.py` 是全功能测试套件，100/100 PASS，覆盖：

测试类别	数	说明
WBS 分解	12	4 方法 × 3 场景
估算方法	16	4 方法 × 4 边界条件
CPM 分析	8	4 依赖类型 × 2 拓扑
蒙特卡洛模拟	6	3 分布 × 2 采样规模
经济效益	10	5 指标 × 2 场景
挣值管理	8	2 模式 × 4 场景
项目文档	12	3 模式 × 4 模板
风险维度	8	7 维度 + 组合
知识库接口	10	CRUD × 3 库
边界鲁棒性	10	空/零/超大值

04 | RAG Assistant 架构文档

一、系统概览

RAG Assistant 是一个本地知识库问答智能体，基于 local-rag-builder 技能构建，核心理念是从传统单轮问答升级为 LLM 驱动的组合式检索 + 用户画像自适应交互：



- [插件注入·Ranker 后点位] input_return 插件注入上下文（检索管道含 rerank 完成后、综合回答前，见 § 3.10）
- [LLM 综合回答] 基于完整上下文 + 用户画像 + 3插槽 prompt 生成回答
- [插件副作用] input_output 插件执行（如日志记录）
- [引用门禁] 校验 LLM 回答中的 [n] 引用是否在资料段落范围内

1.1 核心设计原则

原则	说明
LLM 分词 > 规则分词	实体/属性由 LLM 基于语义标注，不依赖关键词规则
穷举 > 猜测	三层切片全查一遍（entity 单独 / entity×attr / rel 语义切片），不预判哪组最优。rel 时多实体两两配对，单实体 attrs 两两配对
有限枚举 > 无界穷举	三槽位骨架限定归类，查询类型场景约束取值——穷举收敛为三元分类下专属场景内的有限枚举（§ 1.4）
原文锚定 > 自由发挥	evidence 强制每个 entity/attr 提供原文出处，三源校验 + NLI 语义二次判断，拒绝 LLM 编造检索词
去重 > 冗余	SM3 国密哈希按内容去重，避免重复上下文浪费 token
自修正 > 静默丢弃	两阶段决策：模式判定单次调用 → 动作校验循环最多 5 次，解析错误/校验拒绝反馈重试，禁止逃逸动作模式，耗尽后禁止 LLM 自由回答
技能完整走 > 绕路	每片独立走 route_query → retrieve_documents → reranker → NLI → build_context 全流程，不改造技能内部逻辑
配置持久化 > 运行时内存	所有配置（LLM / 路由 / 重排序 / NLI / 切片 / prompt 插槽 / 查询类型）写入 rag_config.json，刷新页面不丢
历史隔离 > 上下文污染	第一轮 LLM 决策仅传压缩摘要 + 上一轮问答原文（供 evidence 三源引用），完整历史不进决策，摘要概念禁止用作当前 entities/attrs

原则	说明
用户画像自适应 > 固定 prompt	基于 OCEAN 多维人格 + 语言风格分析的画像系统，自动调整 LLM 交互风格
旁路扩展 > 主体侵入	扩展点只增不改、失败透传；主体（路由/检索/精排）变更必须架构级，不接受补丁式侵入（§ 3.10）

1.2 路由开关行为

三开关体系：

开关	控制	开	关
<code>kb.enabled</code>	多知识库主开关	允许入库/出库路由工作	全部路由失效，全进 default
<code>kb.auto_classify</code>	入库路由	嵌入模型余弦相似度匹配文档 × 各 KB 关键词，路由到最佳 KB	纯关键词匹配，无匹配进 default
<code>router.enabled</code>	出库路由	嵌入模型 × KB 签名关键词做余弦相似度（精排开时），或嵌入模型 × 规则关键词（精排关时）。精排关时不写 KB 签名	纯关键词匹配，不写 KB 签名

路由方法枚举：

路由方法	触发条件
<code>hardcoded</code>	命中硬编码关键词规则
<code>embedding_signature</code>	精排开：嵌入模型 × KB 签名关键词做余弦相似度。v1.7.0b1 支持多向量路由：有分象限签名列表时各做一次 cosine 取最高分，优于单字符串路由
<code>embedding_keyword</code>	精排关：嵌入模型 × 规则关键词（top-30）做余弦相似度
<code>default</code>	无匹配，路由到 default
<code>direct</code>	用户直接指定了知识库

路由方法	触发条件
broadcast	语义回退失败后全量广播所有 KB 检索

top-N 多 KB 路由 (v2.3.0): `route_query()` 从 top-1 (只取最高分 KB) 改为 top-N——收集所有过阈值 KB, 按分数降序取前 N 个, 激活 `retrieve_context` 预留的多 KB 并查循环。跨域问题 (如"量子物理与音乐的关系") 不再漏召回次相关 KB。三层防护: UI 限 1-10 → 代码夹 `[1,10]` → 运行时限实际过阈值 KB 数。配置项 `router.top_n` (默认 1 保持兼容) 与 `router.classify_threshold` (主路由 KB 分数下限, 默认 0.3); 外部 API `POST /api/kb/query` 支持 `top_n` 参数单次临时指定。

1.3 KB 签名生成流程 (v1.7.0b1)

入库 → 文档 chunks

→ 四分法采样 ($N < 200$ 全量 / $200 \leq N < 500$ 全域随机 / $N \geq 500$ 四分+每份随机)

→ 4 象限各自独立计算 BCE 语义质心

→ 每象限取距质心最近的 20 个 chunk

→ 各象限独立 jieba 候选词提取 + 停用词过滤 (含 PDF 分页残留词)

→ 各象限独立 BCE 比对原始关键词排序

→ 四段拼接: `Q1[:20] + Q2[:20] + Q3[:20] + Q4[:20]` → 上限 80 词

→ 签名同时保存为合并字符串 + 4 个分象限子字符串 (多向量路由用)

→ 反哺: `(30 - count(originals)) // 4` 每象限配额

v1.7.0b1 核心改进:

- 四分法从"采样 4 份合 1 质心"改为"4 份各自算质心、各取 20 近邻"
- 签名上限从 12 词扩至 80 词 (不强求, 取实际值)
- 签名同时保存多段子签名 (`signatures` 列表), 路由时逐个 cosine 取最高分
- 反哺从全局排序改为四象限均分: `(30 - originals) // 4`

- 停用词扩展 8 个 PDF 分页残留词：接上、转下页、上一页、下一页、上页、下页、翻页、第几页

1.4 可配置的前置分类 — 三元分类下的专属场景有限枚举

前置分类解决一个问题：任意自然语言问题的检索空间是**无定义穷举**，必须收敛为有限枚举，切片数与检索成本才可预估。收敛分两级，两级正交：

无定义穷举（任意自然语言问题）

↓ 第一级【归类】：三槽位骨架（系统锁定，不可配）

所有查询动作固定为 entities（主体）/ attrs（目的）/ rel（行为）

三槽位 + type / kb / evidence 参数，LLM 只按语义规则填槽（§ 5.3）

= 归类内穷举 —— 穷举有了维度，但每个槽位内仍可无界取值

↓ 第二级【场景】：查询类型（预制 + 用户自定义，可配）

每个场景 = 三槽位取值约束模板（label + example + rules），

规定该类问题 entities 填什么、attrs 填几个/什么域、rel 是否固定

= 用户场景内穷举 —— 场景内槽位取值模式有限

↓ 组合展开器（三层切片）

= 有限枚举 —— 切片数场景内可预估、可预算

第一级：归类。 三槽位骨架就是限定了分类——无论问题多自由，LLM 输出都被固定到 entities/attrs/rel 三维。骨架由系统锁定（动作格式 + 校验器 + 展开器三方联动），变更属协议级/架构级改动。

第二级：场景。 三元分类下每一类槽位仍可无界取值，因此需要场景约束值域。场景不是启发式技巧，而是**用户问题的类型归属**：问题本身是二元对立的，就命中 opposition 场景，按其规则填槽。归类与场景的关系：**几个固定参数（三槽位）就是归类，用户配置（场景增删与取值规则）就是场景**——用户最清楚自己领域内的问题形态，场景内穷举因此收敛为有限枚举。

内置场景（`BUILTIN_QUERY_TYPES`，agent.py）：

key	label	example	entities 规则	attrs 规则	rel 规则
fact	事实查询	"茅台的价格是多少?"	主体/名词, 能被替换为"关于XX"的XX	目的/属性维度, 禁比较意图词与疑问词	留空 (不触发两两配对)
compare	实体对比	"茅台和五粮液酿造工艺异同"	被对比的多个实体, 逗号分隔	对比维度, "比它们的什么方面"	"对比"
opposition	二元对立	"AI是顺着倾向回答还是独立思考"	只填主体, 不把对立面放进来	两个对立面的表述, 逗号分隔	"对比"
analysis	多维度分析	"新能源汽车的市场规模、政策环境和消费者态度"	分析主体, 只填1个	分析维度并列列举	留空 (两主体对比应改用compare)

自定义机制 (query_types , rag_config.json):

环节	说明
存储	rag_config.json 的 query_types 字典; 同 key 覆盖内置, 新增 key 即新增场景
Web 管理	配置页查询类型区 CRUD (/api/config/query_types), 内置类型不可删除, 自定义类型可增删改
场景结构	{key: {label, example, rules: {entities, attrs, rel}, built_in}}, rules 三键对应三槽位填写规则
注入链路	_get_all_query_types() 合并内置+自定义 → _build_type_reference() 渲染进第一轮决策 system prompt 的「查询类型参考」节——LLM 不需要声明类型, 只需参照最匹配场景的规则填槽

设计边界:

- 场景只约束"怎么填槽", 不改变执行管线——组合展开器对全部场景一视同仁地做三层切片
- 场景增删 = 配置级变更, 随时可做; 三槽位骨架变更 = 架构级变更, 需动作格式/校验器/展开器联动

- 场景化收益以 opposition 为例：无场景约束时 LLM 倾向把对立面塞进 entities 导致展开爆炸；场景规则将其收敛为"主体 1 + 对立面 2 (attrs) + rel 固定"，切片数确定
- 实时类需求当前走 type="search" 独立动作通道；如需"实时查询"场景化（场景内混合 rag/search 通道），属用户自定义场景的演进方向

二、三层架构

层	组件	职责
表现层	web_ui.py (port 8765) / RAG 配置页 subprocess (port 8766) / CLI (stdin)	Web 界面、LLM 配置面板、聊天界面、模型管理、知识库配置、CLI 交互
业务层	agent.py / rag_wrapper.py / engine/rag_core.py / engine/router.py / engine/reranker.py	决策循环、组合查询、路由/检索/重排序/NLI、记忆管理、用户画像
基础设施	llm_client.py / engine/config.py / engine/utils.py / data/ / vendor/	LLM 通信、配置管理、数据持久化、内嵌第三方依赖

2.1 完整文件结构

rag-assistant/

```

├─ main.py                # ★ 入口 (4 种模式: web / cli / batch / jsonl)
├─ setup.bat              # Windows 一键启动 + 进程管理
├─ requirements.txt       # 依赖清单
├─ CHANGELOG.md           # 版本更新日志
├─ PROTOCOL.md            # 外部接入协议规范 (HTTP/CLI/文件交互)
├─ llms.txt               # AI 可读的项目自描述文档 (llmstxt.org)
├─ blueprint_rag.json     # PyPI 发布蓝图
├─ README.md              # 项目说明
├─ LICENSE                # Apache 2.0
├─ server.pid             # 运行时 PID (setup.bat 管理进程)
├─ _test_router_score.py  # 路由评分测试脚本
├─ .gitignore

```

```

├─ rag_assistant/        # ★ 智能体核心层
│   └─ __init__.py       # 版本号: 0.9.5

```

└─ agent.py	# Agent 决策循环 (~620 行)
└─ web_ui.py	# Web 界面 (port 8765, ~1300 行)
└─ llm_client.py	# LLM 统一客户端
└─ rag_wrapper.py	# RAG 封装桥接层
└─ search.py	# 联网搜索
└─ memory.py	# 四层记忆 + 用户画像系统
└─ _fix_rag.py	# 破损数据修复工具

└─ engine/	# ★ local-rag-builder 技能核心 (独立副本)
└─ __init__.py	
└─ rag_core.py	# RAG 核心: 检索/嵌入/导入
└─ router.py	# 两级路由: 关键词 + 嵌入 × KB 签名
└─ reranker.py	# 三模式重排序 + FallbackRouter
└─ nli_classifier.py	# NLI 三向分类器
└─ knowledge_base_manager.py	# 知识库 CRUD + 自动分类 + SM3 去重 + ChromaDB 容灾
└─ config.py	# 配置加载/保存/自动修正模型路径
└─ embedding_model_manager.py	# 5 源并行模型下载管理
└─ prompt_manager.py	# 3 插槽 + 预设管理 + 用户画像扩展点
└─ text_splitter.py	# 5 策略 + 5 守卫插件架构
└─ rag_skill.py	# 技能接口
└─ rag_standalone.py	# 独立模式
└─ rag_web_ui.py	# RAG 配置页 (完整前端)
└─ rag_setup_orchestrator.py	# 安装编排
└─ rag_env_setup.py	# 环境检测
└─ utils.py	# 工具函数 + 数据目录管理

└─ vendor/	# ★ 内嵌第三方库 (零 pip 也可在受限环境中运行)
└─ bs4/	# BeautifulSoup4
└─ pypdfium2/	# PDF 解析 (Google PDFium 引擎, v2.4.0 替换 pypdf)
└─ pypdfium2_raw/	# PDFium 预编译二进制 (含 pdfium.dll)
└─ pypdfium2_cfg/	# pypdfium2 配置
└─ markdownify/	# HTML → Markdown
└─ soupsieve/	# CSS 选择器 (bs4 依赖)

└─ data/	# 运行时数据
└─ config/rag_config.json	# 引擎全量配置 (含 llm 子字典、prompt_slots 等)
└─ kb/	
└─ kb_index.json	# 知识库索引
└─ kb_signatures.json	# KB 签名关键词
└─ auto_classify_rules.json	# 自动分类规则
└─ {name}/	# 各知识库 (13 个, ChromaDB SQLite + HNSW)
└─ models/	
└─ model_index.json	# 模型索引
└─ config/rag_config.json	# LLM 与检索配置
└─ memory/	

		compressed_{id}.txt	# LLM 压缩摘要
		kb_gaps.json	# 知识缺口 (最多 200 条)
		user_habits.json	# 用户习惯 + OCEAN 人格画像
		sessions/{id}.txt	# 短期对话
		prompts/	
		custom_presets.json	# 用户自定义 prompt 预设
		custom_prompt_template.txt	
		import_manifest.json	# 待入库文件清单
		imports/	# 浏览器上传临时目录 (入库后自动清理)
		cache/	# 模型下载临时缓存

三、组件详解

3.1 决策循环 — agent.py

核心是 `chat()` → `_decide_with_retry()` 的**两阶段决策 + 动作校验循环**。

两阶段设计：

阶段 1（模式判定）：单次 LLM 调用，不进循环

- └ 显式 <>（或旧版无动作标记） → chat 模式，直接返回
- └ 解析出 query/search/import 或格式错误 → 进入阶段 2

阶段 2（动作校验循环 `_action_validation_loop`）：max_retries=5

- └ LLM 一旦进入动作模式，必须在重试中保持动作模式

——不允许通过"不输出 <>"逃逸校验，逃逸按修正提醒强制回正

- └ 解析错误 / 校验拒绝 → 以【修正提醒】反馈给 LLM 重试
- └ 5 次耗尽 → 禁止 LLM 自由回答（防止捏造引用），返回固定提示"请重新表述你的问题"

第一轮决策消息构建（`_build_first_pass_messages`，不传完整历史对话）：

1. system prompt（动作格式 + 查询类型参考场景规则，见 §1.4）

- 2. 上一轮问题/回答原文（`_get_previous_turns()`，供追问时 entities/attrs/evidence 三源引用）
- 3. 压缩摘要（标注"其中的概念/关键词不得用于当前问题的 entities/attrs"，防历史泄漏）
- 4. 用户画像提示（`prompt_manager.build_persona_context()`）
- 5. 当前消息

_parse_action — 状态机解析器

从 LLM 输出中提取 `<>` 指令，使用**状态机逐字符扫描**（而非正则表达式）：

- 1. **Windows 路径兼容**：`C:\Users\...` 中的反斜杠不被当作转义前缀，`\U` 不被解释为 Unicode 序列
- 2. **文件名含引号**：仅 `\"` 和 `\\` 视为转义，其他 `\x` 保持字面量
- 3. **格式错误捕获**：`'`、`<>` 等非标准写法返回错误原因进入修正循环，不静默丢弃

返回格式：

- `(None, None)` — 无动作标记（chat 模式）
- `(None, "原因")` — 有 `<` 但格式错误
- `( {... }, None )` — 解析成功

_validate_action — 动作校验

type 必须是 `query / search / import / chat`；chat 无需额外校验，其余逐类校验：

类型	校验项
<code>query</code>	entities/attrs 必填且各为 单概念 （禁 <code>/</code> 、 <code> </code> 、 <code>、</code> 、 <code>•</code> 拼接分隔符）； evidence 必填 （见下）；kb 若填必须为用户原话提及的名称
<code>import</code>	关键词门禁 ：用户没说导入/入库等，直接拒绝 import 指令；path 非 MANIFEST 时必须存在（支持逗号分隔多路径）；kb 必须用户原话提及 且真实存在 （查 list_kbs）
<code>search</code>	query 必填

evidence 原文锚定校验：

- 格式：JSON 字典，key 与 entities/attrs 中的写法**精确一致**（LLM 多塞的额外 key 自动忽略），value 必须是**原文单个连续子串**（禁拼接分隔符，尾部省略号自动清洗）
- 三源校验：value 须在**当前消息 / 上一轮问题 / 上一轮回答**之一中存在
- 语义二次判断：硬编码校验不通过且 `nli.output_enabled` 开启时，走 NLI `verify(key, value)`（key/value 互含子串自动通过，否则模型判断），降低错误拒绝率
- 拒绝信息附带 key 所在源的**字符级切片定位参考**（前后 30/80 字符），引导 LLM 对照原文修正

组合查询 — 三层切片展开

当 LLM 输出 `type="query"` 时触发。展开器做三层切片（示例为 opposition 场景填槽）：

```
# LLM 输出示例
<<ACTION type="query" entities="AI" attrs="迎合,独立思考" rel="对比"
    evidence='{"AI":"AI","迎合":"顺着倾向回答","独立思考":"独立思考"}'>>

# 三层切片：
slices = [
    "AI",                                # 第一层: entity 单独（实体宽泛检索）
    "AI 迎合",                           # 第二层: entity × attr（事实块检索）
    "AI 独立思考",
    "AI 迎合 独立思考 对比",            # 第三层: rel 语义切片（单实体 → attrs 两两配对
+ rel)
]

# rel 语义层的两种形态：
# 多实体 (≥2) : itertools.combinations 两两配对 → "e1 e2 rel" / "e1 e2 attr
rel"
# 单实体: attrs 两两配对 → "e a1 a2 rel"
# 过滤：比较意图词（异同/区别/对比…）与疑问词（为什么/怎么/如何…）从 attrs 剔除，不自动设
rel
```

切片展开后各片独立走 `rag.query(slice, include_header=True)` 完整流程，结果按 SM3 内容哈希去重合并；同时回取各命中文档的**源文档头部块**（`headers`），以【文档: 源文件】形式拼在语义检索结果之前，补足头部上下文。

get_embeddings() 缓存优化（v0.8.5）：组合查询共享同一个嵌入模型实例，避免每片重复加载（18 次 → 1 次）。

3.2 LLM 客户端 — `llm_client.py`

支持双后端，统一返回 `{text, reasoning, raw}`。支持流式响应：

后端	协议	接口	模型参数	默认端口
LM Studio	HTTP (OpenAI 兼容)	<code>/v1/chat/completions</code>	<code>max_tokens</code> / <code>temperature</code>	1234
Ollama	HTTP (Ollama API)	<code>/api/chat</code>	<code>num_predict</code> / <code>temperature</code>	11434

模型发现：调用 `/v1/models` (LM Studio) 或 `/api/tags` (Ollama) 列出可用模型。

健康检查：`check_health()` 通过简化 API 调用测试后端连通性。

3.3 Web 界面 — `web_ui.py`

基于 Python `http.server` 的单文件 Web 界面 (~1300 行)，无外部框架依赖。端口自动分配 (`_find_ports()` 查找 2 个可用端口)。

前端：内嵌单页 HTML + JS，使用 marked CDN (`cdn.jsdelivr.net/npm/marked/marked.min.js`) 渲染 Markdown。

端点	方法	说明
<code>/</code> 或 <code>/index.html</code>	GET	主页面 (配置 Tab + 对话 Tab)
<code>/api/config</code>	GET	获取完整配置
<code>/api/kbs</code>	GET	知识库列表
<code>/api/llm/models?backend=xxx</code>	GET	扫描模型列表
<code>/api/llm/test</code>	GET	测试 LLM 连接
<code>/api/config/llm</code>	GET/P OST	获取/更新 LLM 配置 (backend/model/timeout/maxtokens)

端点	方法	说明
/api/config/search	GET/POST	联网搜索配置
/api/config/memory	GET/POST	记忆参数配置
/api/config/query_types	GET/POST	查询类型场景 CRUD（内置类型不可删除，自定义同 key 覆盖，见 § 1.4）
/api/chat	GET/POST	Agent 决策循环聊天
/api/chat/history	GET	聊天历史持久化（v0.8.4）
/api/agent/query	GET/POST	直接 RAG 查询（绕过 Agent 决策）
/api/agent/import	POST	导入文档（3 种模式：path/text/content）
/api/agent/upload-files	POST	浏览器上传文件到服务器临时目录
/api/agent/gaps	GET	知识缺口列表
/api/memory/reset	GET	重置对话
/api/memory/compress	GET/POST	压缩上下文
/api/memory/clear-context	GET/POST	清除上下文
/api/memory/inject	POST	注入系统通知
/api/session/new /api/session/list	GET/POST	会话新建 / 列表（含归档会话与首条消息预览）
/api/session/switch /api/session/archive	GET/POST	会话切换 / 归档（压缩摘要同步归档）
/api/session/restore /api/session/delete	GET/POST	会话恢复 / 永久删除

端点	方法	说明
/api/search/toggle	POST	联网搜索开关
/api/availability-status	GET	模型下载探测状态 (v0.9.0)
/api/plugins	GET	插件列表 (v2.1.0)
/api/plugins/toggle	POST	启用/禁用插件 (v2.1.0)
/api/plugins/config	POST	打开插件配置界面 (v2.1.0)
/api/plugins/refresh	POST	重新扫描插件目录 (v2.1.0)
/api/plugins/generate	POST	AI 插件生成器 (v2.1.0)

关键交互细节:

- loadModels() 在页面加载后 500ms 触发, 填充模型下拉框
- 配置保存后立即同步到 self.agent.llm.* 运行时实例
- llm_max_tokens 和 llm_timeout 持久化到 rag_config.json
- 路由/reranker/NLI toggle 无已下载模型时灰化 + 红色提示文字
- 网络探测结果实时增量更新 ● / ●

文件上传流程: 点击文件选择按钮 → 文件以 base64 二进制上传到服务器 data/imports/ 目录并记录到 import_manifest.json, 同时聊天框出现系统通知。用户输入"入库"后 LLM 发出 path="MANIFEST" 指令, 系统读取清单逐个走完整导入管线。

PDF 导入 (v2.4.0 引擎迁移 pypdf → pypdfium2, 三层 OCR 判断):

- 多页 PDF 合并全部页内容后切分 ("\n\n".join(d.page_content for d in docs))
- PDF 引擎: pypdfium2 (Google PDFium), 从根源消除 pypdf 的 ToUnicode CMap 形似字缺陷 (Iethods → Methods 类)
- 三层 OCR 判断 (v2.4.0 重构, 替代旧版两层判断):
 1. **二进制类型判断:** 读 PDF 二进制检测 /Font 与 /Image —— 无 /Font 直接 OCR (无文本层); 有 /Image 且无文本页占比 > 50% 直接 OCR (混合版扫描件)
 2. **信号 2 乱码检测** (逐页): 英文词间距丢失 (alpha > 0.5 且 max_run > 30)
 3. **信号 4 乱码检测** (逐页): 中文常用字覆盖率 < 50% (_COMMON_CJK ~200 高频字表)
- 乱码页占比 > 10% → 整篇 OCR; 乱码 chunk 不剔除, 让检索自然降权
- 英文正常 PDF 不走 OCR

3.4 RAG 封装层 — rag_wrapper.py

将 local-rag-builder 的技能接口包装为 Agent 可调用的形式：

```
rag.query(question, kb_name=None, include_header=False, ...)
    → retrieve_context(question, kb_name, ...)
        → route_query(question) # 路由：两级（关键词 + 嵌入 × KB签名）
            → retrieve_documents(question, kb) # 检索：取 top-K chunk (HNSW 自动修复)
                → reranker.rerank(docs) # 可选：精排 (model/rule/hybrid)
                    → nli_classifier.classify(docs) # 可选：NLI 三向标注
                        → build_context(docs) # 构建上下文（含 NLI 标签）
                            → return {context, docs, kb, headers, has_context}
```

3.5 搜索模块 — search.py

引擎	方式	API Key
DuckDuckGo	requests.get(html.duckduckgo.com/html/)	无需 Key
Tavily	POST api.tavily.com/search	需配置 Key
urllib fallback	纯 HTML 解析回退	无需 Key

通过 web_search_enabled + web_search_api_key + web_search_engine 配置。

3.6 引用门禁 — agent.py

v0.8.0 新增：LLM 回答后校验引用编号。_second_pass() 中的引用校验逻辑：

- 系统提示强制要求每个具体事实/数字后面标注来源段落编号 [n]
- LLM 回答后提取所有 [n] 引用，检查编号是否在资料段落范围内
- 不存在的段落编号 → 告警追加到回答尾部
- 无引用 → 记录日志（不作为错误）
- 插件注入内容走**来源标记引用**：上下文含【联网搜索】等插件标记时，追加引用规则——引用插件信息标注 [插件名称]，引用知识库信息继续用 [n] 段落编号

3.7 KB 暂停写入

v0.8.0 新增：配置页自动分类规则表格每行增加暂停/恢复按钮。

场景	行为
自动路由入库	<code>auto_classify()</code> 从 rules 中过滤掉 <code>kb_paused</code> 列表中的 KB，文件自动路由到次高分的非暂停 KB
用户指定入库	<code>add_documents_to_kb()</code> 拒绝写入，提示"已暂停，请恢复或选其他 KB"
查询/检索	完全不受影响
恢复暂停	再次点击按钮，KB 恢复为可写入

3.8 重排序 — `engine/reranker.py`

三模式重排序 + FallbackRouter (v0.7.0 从 router.py 迁入)：

组件	职责
<code>ModelReranker</code>	transformer cross-encoder 打分排序
<code>RuleReranker</code>	规则引擎 (score_weight / recency / source_weight / boost_keywords)
<code>HybridReranker</code>	模型打分 + 规则微调
<code>FallbackRouter</code>	语义回退路由 (cross-encoder 对 query × KB 签名打分) + KB 签名生成阶段的文档片段打分

重排序不再参与路由决策 (v0.5.0 解耦)。路由改用嵌入模型余弦相似度。

3.9 NLI 三向分类器 — `engine/nli_classifier.py`

v0.9.0 新增，cross-encoder 3-class 模型 (contradiction / neutral / entailment)。

模型	语言
cross-encoder/nli-deberta-v3-base	英文 (SOTA)
MoritzLaurer/mDeBERTa-v3-base-xnli	多语言 XNLI
MoritzLaurer/mDeBERTa-v3-base-mnli-xnli	多语言 (双源训练)
BAAI/bge-reranker-v2-minicpm-layerwise	多任务
cross-encoder/nli-MiniLM2-L6-H768	英文 (轻量)
cross-encoder/nli-roberta-base	英文

标签输出格式: [NLI: entailment, 92%] 在 reranker 之后 (reranker 开时) 或向量召回之后 (reranker 关时) 运行。

除管道内三向分类外, NLI 分类器还通过 `verify(key, value)` 方法承担 **evidence 语义二次判断** (agent 动作校验共享 `get_nli_classifier()` 单例, 见 § 3.1): 硬编码原文校验不通过时判断 key 与 value 的语义一致性, 降低错误拒绝率。

3.10 Ranker 后插件点位 — before_response 旁路扩展 (已实现)

定位: `input_return` 类型插件的执行时机, 位于**完整检索管道 (含 reranker) 之后、LLM 综合回答之前**。相对 ranker 而言, 这是一个"检索已定、精排已定"之后的旁路注入点——锦上添花, 不影响主体功能完整性。

管道位置 (agent.py `chat()`, 实测代码锚点 L320-344):

用户查询 → [组合展开器] 多切片

→ 每片独立: route → retrieve → rerank → NLI → build_context (主体引擎管道)

→ [SM3 去重合并] 多片结果按内容哈希去重

→ [插件点位 · input_return] ← Ranker 后点位 (run_before_response)

插件读取 question / rag_context / thinking 等裁剪输入

产出补充内容 (如联网搜索结果)

→ [合并] 注入内容以分隔线 + "## 插件补充信息" 附加到 context (不改写主体管道产出)

→ [LLM 综合回答] _second_pass (3 插槽 prompt, 含插件来源引用规则)

→ [插件点位 · input_output] 回答后副作用 (run_after_response)

与 ranker 的相对位置：主体引擎管道内部 (route → retrieve → rerank → NLI → build_context) **没有任何插件点位，也不应该有**——管道内每一步都是主体功能，扩展点开在管道内部等于允许外部代码介入主体执行路径。插件点位开在整条管道**完成后**：插件只能消费管道的最终产出 (rag_context)，不能触达路由/检索/精排的任何中间环节。

稳定性契约（代码事实）：

契约	代码证据
失败透传	run_before_response 整体 try/except，异常仅 warning 日志，主流程照走 (agent.py L325-336)；单插件失败由 PluginManager._safe_execute 隔离，不影响其他插件
只消费不改源	插件输入按 input_fields 从 6 字段池裁剪；注入内容 附加 到 context 尾部 (existing + "\n\n---\n\n## 插件补充信息\n\n" + injected)，不修改、不覆盖主体管道产出 (agent.py L338-344)
五道防线	信息隔离 / 文件沙箱 / 超时熔断 (per-plugin timeout) / 输出校验 / SM3 签名校验 (_verify_signature)，与 §6.6 同一套
type 硬约束	插件 type 枚举硬校验，仅允许 input_return / input_output (manager.py L152-153)，非注册点位的执行路径不存在
主体变更 = 架构级	插件点位只增不改；主体（路由/检索/精排）要改必须是架构级变更，不允许补丁式侵入

设计理据 — 为什么点位在 ranker 之后：

- 检索与精排完成后，任何增强（外部检索补召回、行情查询、结果补充）都是锦上添花：不影响前面任何环节的正确性，也不因自身失败拖垮主体
- 在一个运行系统中，部分需求扩张导致的例如召回率等参数不理想时，正确动作是**旁路补足**（web_search 联网搜索插件就是这一原则的活实例：知识库覆盖不到的实时内容，由插件在管道完成后注入），而不是为指标回头改主体——哪怕保持低召回的稳定性，也不能引发系统崩坏级的不稳定
- 系统越大越难发现"蚁穴"和"蝴蝶翅膀"在哪里——这不是简单的试错问题，不是积累经验的问题，这是人类和 AI 共同存在的**注意力机制**的局限。用点位隔离代替试错，是稳定运行的重要前提

四、记忆系统 — memory.py

四层记忆 + 用户画像系统，统一管理短期对话、压缩摘要、知识缺口和用户习惯性格画像。

4.1 短期记忆

方法	功能	存储
<code>append_short_term(session_id, role, content)</code>	追加一条对话记录（超 2000 字符截断）	<code>data/sessions/{session_id}.txt</code>
<code>get_short_term(session_id)</code>	读取完整对话历史	返回 <code>str</code>
<code>clear_short_term(session_id)</code>	清空对话历史	删除文件
<code>pop_oldest_lines(session_id, n)</code>	弹出最旧的 N 行（默认 40）	返回被移除的文本
<code>short_term_line_count(session_id)</code>	当前行数	返回 <code>int</code>
<code>needs_compression(session_id)</code>	行数 > 100 触发压缩开关	返回 <code>bool</code>

4.2 长时记忆（压缩摘要）

方法	功能	存储
<code>store_compressed(session_id, summary)</code>	追加一条压缩摘要	<code>data/memory/compressed_{session_id}.txt</code>
<code>get_compressed(session_id, limit=3)</code>	返回最近 N 条摘要	返回 <code>str</code> （多摘要拼接）

压缩触发（`_compress_if_needed()`）：当 `short_term_line_count() > 100`（约 50 轮对话）时触发：

- `pop_oldest_lines()` 取出最旧 40 行对话
- 调 LLM 压缩为摘要（结构化指令要求保留核心需求、已得结论、追问方向、最近 3 条原文，200 字

以内)

3. `store_compressed()` 存入长时记忆

4.3 知识缺口记录

记录检索不到答案的查询，分析知识库覆盖盲区。保留最近 200 条，相同 query 自动累加计数。

```
{
  "query": "三个代表与老子无为而治的相同点",
  "kb": "政经文哲",
  "count": 3,
  "first_seen": "2026-07-08T04:20:13",
  "last_seen": "2026-07-08T04:25:27"
}
```

4.4 用户习惯与性格画像 (v0.6.0)

三层分析体系：

层级	方法	输出
规则级 语言分 析	<code>_classify_sentence(msg)</code>	句式 (statement/question/imperative/rhetorical) + 语气 (neutral/critical/curious/sarcastic/terse/enthusiastic) + 深度 (shallow/medium/deep)
OCEAN 五维人 格	<code>_ocean_delta()</code> → 衰减更新 (<code>PERSONALITY_DECAY=0.98</code>)	openness / conscientiousness / extraversion / agreeableness / neuroticism (0-1, 默认 0.5)
合成画 像	<code>get_persona()</code> → <code>build_persona_context()</code>	语言风格统计占比 + 人格标签 + 行为偏好文本

人格更新机制：

```
# 衰减 + 新样本加权
new_val = old * decay + delta * (1 - decay) # decay=0.98
personality[dim] = clamp(new_val, 0.0, 1.0)
```

人格标签映射 (`get_persona()` 中的 `_dim_label`):

维度	< 0.55	≥ 0.55
openness	守成型	探索型
conscientiousness	随性型	严谨型
extraversion	内敛型	外放型
agreeableness	对抗型	亲和型
neuroticism	稳定型 (< 0.45)	敏感型 (≥ 0.45)

4.5 与 Agent 的集成

```
chat(message)
  ↓
append_short_term(session_id, "user", message) # 写入用户输入
  ↓
_get_previous_turns() # 取上一轮问题/回答 (evidence 三源引用)
  ↓
_decide_with_retry(message, max_retries=5) # 两阶段: 模式判定 → 动作校验循环
  |—— 第一轮消息: system prompt (动作格式 + 查询类型参考场景)
  |   + 上一轮问答原文 + 压缩摘要 (概念禁止用作 entities/attrs) + 画像
  |—— LLM 决策 (query/search/import/chat)
  ↓
执行动作 → _second_pass(message, context, action) # 第二轮: 带上下文 + 历史 (跳过 reasoning)
  ↓
append_short_term(session_id, "assistant"/"reasoning", ...)
                                     # 写入助手回复/推理 (自动剥离
<<ACTION>> 标签)
record_habit(message, is_rag, ..., kb) # 记录习惯 + 语言分析 + OCEAN 更新
```

↓ 如果检索结果为空

```
record_gap(query, kb)
```

```
# 记录知识缺口
```

历史隔离 (v0.8.0)：第一轮决策不传完整历史对话，仅传压缩摘要作为 system context。第二轮 `_second_pass()` 仍携带带 [历史对话] 前缀的历史消息，保证跨轮追问的上下文连贯性。

ACTION 剥离 (v0.8.0)：写入记忆时自动使用 `re.sub(r'<{1,2}\s*ACTION\s+.*?>{1,2}', '', content, flags=re.DOTALL|re.IGNORECASE)` 剥离内部指令标签。

记忆角色：user / assistant / reasoning 三种角色写入短期记忆；reasoning 单独记录，第二轮历史消息构建时跳过（防止连续 role 破坏消息序列）。

4.6 会话管理

方法	功能
<code>new_session()</code>	新建会话；活跃会话数超 <code>memory.max_sessions</code> （默认 20）自动归档最旧的非活跃会话
<code>list_sessions()</code>	列出活跃 + 已归档会话（含首条用户消息预览 60 字符），按创建时间倒序
<code>archive_session()</code>	会话移入 <code>archives/sessions/</code> ，压缩摘要同步移入 <code>archives/memory/</code>
<code>delete_session()</code>	永久删除会话文件（含归档目录与压缩记忆）

对应 Web 端点：`/api/session/new|list|switch|archive|restore|delete`。

五、外部接口

5.1 Python 编程接口（API）

类	入口方法	返回格式
<code>Agent</code>	<code>.chat(message) → dict</code>	<code>{"text", "success", "reasoning", "kb", ...}</code>

类	入口方法	返回格式
	<code>.reset_session()</code>	无返回值
Memory	<code>.get_short_term(id) → str</code>	对话历史文本
	<code>.append_short_term(id, role, content)</code>	无返回值
	<code>.get_gaps(min_count) → list[dict]</code>	<code>[{"query", "kb", "count", ...}]</code>
	<code>.get_habits() → dict</code>	<code>{"total_queries", "chat_ratio", "personality", ...}</code>
	<code>.get_persona() → dict</code>	<code>{"linguistic_summary", "personality", "behavior"}</code>
LLMClient	<code>.chat(messages) → dict</code>	<code>{"text", "reasoning", "raw"}</code>
	<code>.list_models() → list[str]</code>	模型名列表
	<code>.check_health() → bool</code>	连接状态
RAGWrapper	<code>.query(question, kb_name) → dict</code>	<code>{"context", "docs", "kb", "has_context"}</code>
	<code>.import_file(path, kb_name) → dict</code>	<code>{"success", "doc_count", "kb"}</code>
	<code>.import_text(text, kb_name, title) → dict</code>	<code>{"success", "doc_count", "kb"}</code>
	<code>.list_kbs() → dict</code>	知识库字典
WebSearch	<code>.search(query, max_results) → dict</code>	<code>{"results": [{"title", "url", "snippet"}], "success"}</code>

5.2 技能依赖接口

技能模块	导入函数	用途
rag_core	retrieve_context	检索主入口（路由→检索→reranker→NLI→build）
rag_core	get_embeddings	嵌入模型管理（单例缓存）
rag_core	build_context	上下文构建（含 NLI 标签渲染）
knowledge_base _manager	list_knowledge_bases	知识库枚举
knowledge_base _manager	_load_rules / auto_classify	入库路由
knowledge_base _manager	sm3	SM3 国密哈希去重
config	load_config / save_config	配置持久化 + 模型路径自动修正
prompt_manage r	build_second_pass_prompt	3 插槽 prompt 构建
prompt_manage r	build_persona_prompt	用户画像注入
nli_classifie r	NLIClassifier.classify / NLIClassifier.verify	NLI 三向分类 / evidence 语义二次判断（§ 3.1）

5.3 Agent 动作协议（LLM ↔ Agent 通信）

LLM 在回复中嵌入 <> 标记控制 Agent 行为：

```
<<ACTION type="query" entities="实体1,实体2" attrs="属性A,属性B" rel="关系词"
      evidence='{ "词": "原文出处" }' kb="知识库名（可选，必须用户原话提及）">>
<<ACTION type="search" query="搜索词">>
<<ACTION type="import" content="入库的完整文本内容">
<<ACTION type="import" path="MANIFEST">           # 批量导入所有待入库文件
<<ACTION type="chat">>                             # 闲聊/直接回答也必须显式声明
```

- 不使用 **question** 参数（不会生效），成分一律标注进 `entities/attrs`
- 所有回复都必须显式声明动作类型（包括纯聊天），系统通过标记决定处理方式

LLM 分词语义规则（三槽位，v0.9.0 重写）：

- `entities`：取主体/名词。问题中涉及的核心事物、人物、概念，每个 `entity` 必须是单个概念（禁拼接分隔符）
- `attrs`：取目的/限定域。用户想查询的目标/用途/对象，可以是复合短语；排除比较意图词（异同、区别、对比等，归 `rel`）与疑问词（为什么/怎么/如何，非搜索维度）
- `rel`：取行为。实体间的动作/关系，填一个最贴切的词（如"对比"）
- `evidence`：**每个 `entity` 和 `attr` 都必须提供原文出处**。key 与 `entities/attrs` 写法精确一致，value 是原文单个连续子串，可来自当前消息/上一轮问题/上一轮回答三源；`entity/attr` 可对原文凝缩提炼，但提炼词须在原文有对应短语

场景层（可配置的前置分类）：槽位怎么填由查询类型场景约束——内置 4 类（`fact/compare/opposition/analysis`）+ 用户自定义（同 key 覆盖），LLM 不声明类型，参照最匹配场景的规则填槽。详见 § 1.4。

5.4 Prompt 3 插槽架构 + 自定义预设 — `prompt_manager.py`

系统提示词框架锁定不可改，暴露 3 个插槽由用户配置：

插槽	默认值	作用
<code>cite_form</code> <code>at</code>	"每个结论后面用 [n] 标注来源的段落编号"	控制引用标注格式
<code>output_st</code> <code>yle</code>	"用 Markdown 格式输出"	控制输出风格 / 格式
<code>fallback</code>	"如果资料中没有明确结论，可以结合资料进行分析推理，但不能编造不存在的内容"	控制无资料时的处理策略

预设管理：4 个内置预设（标准模式 / 深度分析 / 对比分析 / 友好对话）+ 用户自定义预设 CRUD。RAG 配置页 UI 中以下拉

分区显示，内置预设不可删除。

六、流程详解

6.1 完整请求生命周期

用户发送消息



web_ui.py → POST /api/chat



agent.chat(message)



记忆写入 → `_get_previous_turns()` (上一轮问答, 供 evidence 三源引用) → `_compress_if_needed()`



`_decide_with_retry` (两阶段: 模式判定单次调用 → 动作校验循环 ≤5 次, 禁止逃逸)

↓ 第一轮消息: system prompt (动作格式 + 查询类型参考场景规则)

+ 上一轮问答原文 + 压缩摘要 (概念禁止用作 entities/attrs) + 画像 + 当前消息



`_parse_action(reply)`

└─ chat (显式 type="chat" 或无标记) → 直接聊天回复 ✓

└─ 解析成功/错误 → 阶段 2 动作校验循环 → `_validate_action()`



↓ 通过



→ type == "query"

→ `_exec_query(entities, attrs, rel, kb)`

→ 三层切片展开 (entity 单独 / entity×attr / rel 语义两两配对)

→ for each slice:

rag.query(slice, kb_name, include_header=True)

→ retrieve_context(slice, ...)

→ route_query → retrieve_documents → reranker
→ NLI → build_context

→ SM3 去重合并 + 源文档头部块回填

→ [插件·Ranker 后点位] pm.run_before_response() —
input_return 插件注入上下文 (见 §3.10)

→ return {context, docs, kb}

→ _second_pass(message, context, action)

→ build_second_pass_prompt(context, question, kb) # 3
插槽 (含插件来源引用规则)

→ LLM 基于上下文生成回答

→ [插件] pm.run_after_response() — input_output 插件副作用

→ 引用门禁校验

→ 记忆写入 (user/assistant/reasoning) → record_habit()
→ record_gap()

→ type == "search"

→ search.search(query)

→ _second_pass(...)

→ type == "import"

→ path == "MANIFEST" → 批量导入

→ path == 具体路径 → import_file()

→ 含 content → import_text()

↓

回答返回前端 (含插件注入内容)

6.2 导入生命周期 — 完整管道

用户上传文件 (Web UI drag & drop)



POST /api/agent/upload-files → 保存到 data/imports/ + 写入 import_manifest.json



用户说"入库" → LLM 输出 <>



_exec_import() → 读取 manifest → 逐个文件:

1. _do_import(path, kb)

2. 入库路由 (auto_classify 决定目标 KB)

3. RAGWrapper.import_file() → import_documents_to_kb()

- 文档加载 (PDF: pypdfium2 提取 + 三层 OCR 判断; 其他: TextLoader)
- 三层切分流水线 (守卫栈 → 主策略 → 后处理)
- ChromaDB 写入 (SM3 去重 + SQLite+HNSW 写入前备份 + 写入失败自动回滚)
- KB 签名自动更新 (BCE 语义质心 → jieba → 停用词过滤 → BCE 排序 → top-12)



manifest try/finally 清空 + 临时文件清理



返回各 KB 分布明细

6.3 SM3 去重策略

```
import hashlib
seen = set()
for doc in all_docs:
    content = doc.page_content
    h = hashlib.new('sm3', content.encode("utf-8")).hexdigest()
    if h not in seen:
```

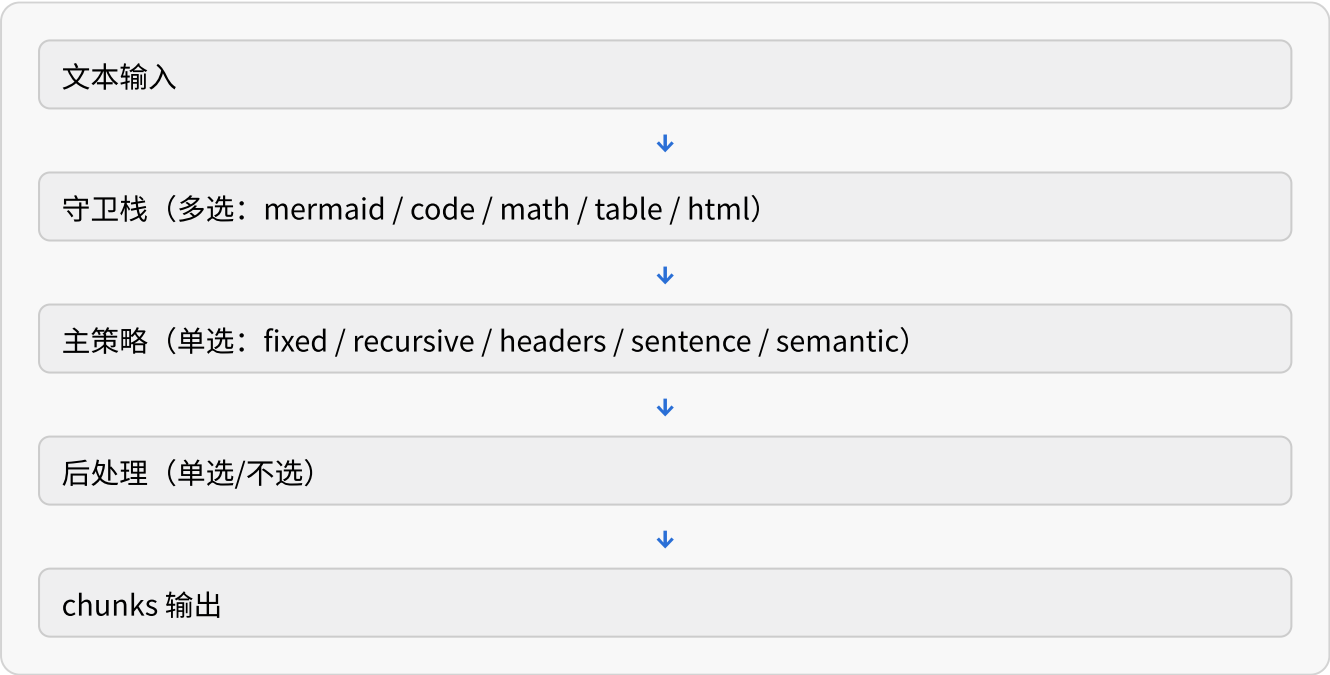
```
seen.add(h)
unique_docs.append(doc)
```

6.4 ChromaDB 容灾 (v0.7.0+)

场景	行为
写入前	<code>_backup_kb()</code> 备份 chroma.sqlite3 + HNSW 索引
写入失败	<code>_restore_kb()</code> 自动回滚到备份
查询 HNSW 损坏	<code>_try_repair_kb()</code> 自动清理损坏索引并重建
配置路径无效	<code>load_config()</code> 自动指向第一个已下载的同类型模型

6.5 文本切分架构 — `engine/text_splitter.py`

插件注册架构：



通过 `register_strategy()` / `register_guard()` 扩展自定义策略。

6.6 智能体插件系统 — `plugins/` (v2.1.0+)

针对应答链路的可扩展旁路增强，不侵入核心决策循环。

插件在智能体生命周期中的位置：

用户提问 → LLM 决策 → RAG 检索管道 (route → retrieve → rerank → NLI → build_context, 无插件点位)

→ [input_return 插件注入·Ranker 后点位] → LLM 生成回答 → [input_output 插件副作用] → 返回

↑ 检索完成后、回答前注入 ↑ 回答后执行副作用

两种插件类型（已实现）：

类型	时机	用途	示例
input_return	检索管道（含 rerank）完成后、回答生成前	结果注入 LLM 上下文	联网搜索补充（web_search）、网络 API 大模型（web_llm）
input_output	回答生成后	副作用（不注入上下文）	日志记录、结果缓存

6 字段池（智能体裁剪传递）：question / answer_draft / thinking / rag_context / session_id / plugin_dir

安全层级（5 道防线）：

防线	机制
信息隔离	只传插件声明的 input_fields，其余字段不可见
文件沙箱	运行时数据仅限 data/plugins//
超时熔断	每个插件独立 timeout，连续 3 次失败自动禁用
输出校验	返回 type 非 markdown/json/csv/plain_text 则丢弃
SM3 签名	可选国密哈希校验插件代码完整性

标准化接口：

```
class PluginBase(abc.ABC):
    async def execute(self, inputs: dict) -> dict:
        # 返回:
        {"type": "markdown|json|csv|plain_text", "content": "...", "priority": 0}
```

目录结构：

```
rag_assistant/plugins/          ← 插件框架代码
├── base.py                     ← PluginBase 基类
├── manager.py                  ← 发现/注册/生命周期/熔断
├── builtin/                    ← 内置插件（随系统发布）
│   ├── web_search/            ← 联网搜索
│   └── web_llm/               ← 远程大模型调用

data/plugins/                   ← 运行时数据目录
├── <builtin_plugin>/config.json ← 内置插件运行时配置
├── <user_plugin>/             ← 用户安装插件（代码 + 数据）
│   ├── plugin.json
│   ├── plugin_xxx.py
│   └── config.json
```

AI 插件生成器（Web UI 插件 Tab）：

插件 Tab 采用左右分栏布局——左侧 LLM 对话面板生成插件，右侧插件管理面板（启用/禁用/配置）。生成流程为二阶段 + 7 步校验：

用户描述需求 → 阶段1：LLM 评估可行性 → 确认 → 阶段2：LLM 生成代码

→ ① plugin.json 合法性 → ② Python 语法（ast.parse）

→ ③ AST 结构检查（防 PluginBase 重定义）

→ ④ 目录规划（user → data/plugins/） → ⑤ 原子写入
（tempfile+rename）

→ ⑥ SM3 签名 → ⑦ discover_and_register 刷新注册

配置项（llm_config.json）：

插件生成调用与主智能体共享 LLM 配置（model / max_tokens / timeout），仅 temperature=0.3 固定用于代码生成确定性。

七、部署与启动流程

7.1 启动模式

模式	命令	说明
Web 界面	<code>python main.py</code>	默认 port 8765，自动分配 RAG 配置子进程端口
CLI 对话	<code>python main.py --no-web</code>	stdin 交互，支持 <code>/reset</code> 指令
批量处理	<code>python main.py --batch --input q.json --output r.json</code>	结构化输入输出
管道 JSONL	<code>cat queries.jsonl python main.py - -jsonl</code>	逐行处理
数据迁移	<code>python main.py migrate</code>	从 local-rag-builder 技能迁移知识库/模型

7.2 Windows 一键启动 — `setup.bat`

`setup.bat`

↓

1. 检测 Python 3.9+（缺失则自动下载安装 Python 3.11）

2. `pip install -r requirements.txt`（首次装依赖）

3. 通过 `server.pid` 杀掉旧进程

4. 启动服务器（`chcp 65001` 修复中文乱码）

5. 轮询端口等待就绪（自适应等待，非硬编码秒数）

6. 自动打开浏览器 `http://localhost:8765`

7.3 PyPI 发布

- 蓝图文件： `blueprint_rag.json` 定义发布时包含/排除的文件

- 版本号: `rag_assistant/__init__.py` 唯一源
- GitHub Actions: `permissions attestations: write` + `skip-existing: true`

7.4 CLI 参数

参数	类型	默认	说明
<code>--port</code>	int	8765	Web 端口
<code>--host</code>	string	0.0.0.0	监听地址
<code>--data-dir</code>	string	<code>./data</code>	数据目录
<code>--config</code>	string	—	覆盖配置文件
<code>--no-web</code>	flag	false	CLI 对话模式
<code>--batch</code>	flag	false	批量处理模式
<code>--input</code>	string	—	JSON 输入文件
<code>--output</code>	string	—	JSON 输出文件
<code>--jsonl</code>	flag	false	管道 JSONL 模式
<code>--pidfile</code>	string	—	PID 文件路径
<code>migrate</code>	subcommand	—	数据迁移

7.5 退出码

退出码	含义
0	全部成功
1	部分失败
2	严重错误

八、依赖与存储

8.1 Python 包依赖

```
langchain>=0.1                # LangChain 框架
langchain-community>=0.3      # 社区加载器 (TextLoader 等; PDF 已直用
pypdfium2)
langchain-huggingface>=0.1    # HuggingFace 嵌入
langchain-chroma>=0.1        # ChromaDB 向量存储
langchain-text-splitters>=0.3 # 文本切分
chromadb>=0.5                 # 向量数据库
sentence-transformers>=3.0    # 句子嵌入
huggingface-hub>=0.20        # HF 模型下载
modelscope>=1.15             # ModelScope 模型下载 (国内源)
openai>=1.0                   # OpenAI 兼容 API (LM Studio)
easyocr>=1.7                  # OCR (扫描版 PDF)
requests>=2.28                # HTTP 客户端
duckduckgo_search>=4.0       # 联网搜索 (可选)
jieba>=0.42                   # 中文分词 (KB 签名关键词提取, v0.5.x 新增)
numpy>=1.24                   # 向量余弦相似度计算
```

8.2 外部服务

服务	用途	备注
LM Studio	LLM 推理 (localhost:1234)	OpenAI 兼容 API
Ollama	LLM 推理 (localhost:11434)	/api/chat 接口
ModelScope	模型下载源 (国内首选)	外部 API
HuggingFace Mirror	模型下载源 (国内备用)	外部 API
HuggingFace Official	模型下载源 (国际备用)	外部 API
HuggingFace Direct	模型下载源 (最后兜底)	外部 API
DuckDuckGo	联网搜索	免费 API, 无需 Key

服务	用途	备注
Tavily	联网搜索（备选）	需配置 API Key

8.3 存储依赖

存储	路径	说明
Chroma DB	data/kb/{name}/	向量知识库（每库一个 SQLite + HNSW 索引）
JSON 文件	data/config/rag_config.json	引擎配置（含 llm 子字典、prompt_slots、kb_paused）
JSON 文件	data/kb/kb_index.json	知识库索引
JSON 文件	data/kb/kb_signatures.json	KB 签名关键词
JSON 文件	data/kb/auto_classify_rules.json	自动分类规则
JSON 文件	data/models/model_index.json	模型索引（含 type 字段：embedding/reranker/nli）
JSON 文件	data/memory/kb_gaps.json	知识缺口（最多 200 条）
JSON 文件	data/memory/user_habits.json	用户习惯 + OCEAN 人格画像
JSON 文件	data/prompts/custom_presets.json	自定义 prompt 预设
TXT 文件	data/sessions/{id}.txt	短期对话
TXT 文件	data/memory/compressed_{id}.txt	LLM 压缩摘要

8.4 配置机制

- **配置文件：** `data/config/rag_config.json`
- **默认配置：** 在 `engine/config.py` 的 `DEFAULT_CONFIG` 中硬编码
- **配置加载顺序：**
 1. `DEFAULT_CONFIG` 默认值
 2. `rag_config.json` 实际值（合并到默认上）
 3. 旧版 LLM key 自动迁移到 `llm` 子字典
 4. 模型路径自动修正（失效路径 → 第一个已下载的同类型模型）
- **极客模式**（v0.8.3）：8 区块分区编辑（Prompt / 嵌入模型&检索 / 重排序 / 切片 / 路由层 / 知识库 / LLM / 其他）

九、安全与隐私

- **无外部调用：** 所有 LLM 请求发向本地 LM Studio / Ollama，不上传数据
- **本地知识库：** ChromaDB 向量库存储在本地 `data/kb/`，不离开用户机器
- **联网搜索可选：** 默认关闭，需用户手动启用
- **模型本地加载：** 所有模型（嵌入/路由/reranker/NLI）通过本地磁盘加载，
`local_files_only=True`
- **自包含 vendor：** `vendor/` 嵌入 bs4 / pypdfium2 / markdownify 等第三方库，零外部 pip 安装也可运行

十、版本演进要点

版本	新增/变更要点
文档同步 2026-09-04	对照代码 v2.4.1 全文复核：决策循环两阶段重写（模式判定 + 动作校验循环 max_retries=5 + 禁止逃逸）、evidence 原文锚定校验（三源 + NLI verify）、查询类型场景层（§ 1.4，v1.0.1 引入 analysis、后续修复长期存在）、Ranker 后插件点位解析（§ 3.10，before_response 点位实测锚定）、会话管理补录（§ 4.6）、Web 端点表补全
v2.1.0 b2	AI 插件生成器（二阶段 LLM + 7 阶段校验管道）；web_llm 多 profile 配置系统； setup.bat HNSW 修复
v2.1.0 b1	智能体插件系统（PluginBase + PluginManager + 5 道防线）；内置联网搜索插件；插件 Web UI 管理面板

版本	新增/变更要点
v2.0.0 b1	1.x → 2.x HNSW 索引引擎更换；Chroma 适配器重构；setup.bat 全量重建提示
v1.8.0	外部 API 端口独立（8767）；引擎独立化（engine/ 副本自包含）
v1.7.0	PROTOCOL 协议升级；KB 签名多向量路由
v2.4.1	版本号重发（2.4.0 内容）
v2.4.0	PDF 引擎迁移 pypdf → pypdfium2；乱码三层检测（二进制类型/信号2词间距/信号4常用字覆盖）；vendor 替换
v2.3.0	top-N 多 KB 路由（router.top_n/classify_threshold）；外部 API top_n 参数；死代码清理（FallbackRouter）
v1.5.0 b1	Web 配置页面内嵌（iframe 模式）；双端口架构
v1.3.0- beta	双面板 Web UI（配置 + 对话）
v1.2.0	组合检索（LLM 分词 + entities × attrs 穷举展开）
v1.1.0	四层记忆系统（短时/压缩/习惯/缺口）
v1.0.0	从 local-rag-builder 仓库外项目独立为正式版
v0.9.5	README 架构图补 NLI；NLI 模型探测遍历所有源修复
v0.9.0	NLI 三向分类器；网络探测并行化；Config 自动修正模型路径；组合查询两两配对 + 中文逗号
v0.8.0	KB 暂停写入；历史对话隔离；引用校验；OCR 触发条件修复
v0.7.0	KB 签名新流程（BCE→jieba→停用词→BCE排序）；精排/路由解耦；ChromaDB 容灾
v0.6.0	用户习惯画像系统（OCEAN + 语言分析）；Prompt 自定义预设
v0.5.0	出库路由彻底弃用 reranker，改用嵌入模型余弦相似度；KB 签名反哺
v0.4.0	入库路由独立（kb.auto_classify）；多知识库主开关（kb.enabled）
v0.3.0	ChromaDB 崩溃修复；状态机解析器重写；MANIFEST 批量导入

版本	新增/变更要点
v0.2.0	PROTOCOL.md; llms.txt; --batch / --jsonl 模式
v0.1.0	从 local-rag-builder v1.5.0 抽取为独立智能体

05 | Structured Writer 架构文档

一、系统概览

Structured Writer 是**双线架构**的结构化写作智能体：

通用写作线（v1.x 起）：模板驱动 + 子结构逐段生成 + 两级 RAG 增强：

用户选择/创建模板

→ [模板系统] 模板定义 meta+content+style+logic 四元结构

→ [用户填写] meta 字段（标题/作者/单位等），content 字段自动参与大纲

→ [大纲规划器] LLM 根据模板 content 生成结构化 JSON 大纲

节 1: 引言 → [子结构A，子结构B，子结构C]

.....

→ [交互式大纲] 用户可调整勾选/排序/字数/重点/RAG/辅助知识/局部重规划

→ [串行写作器] 逐子结构执行：两级 RAG → 引用注入 → 前文注入 → LLM 写作 → 续写 → 引用后处理

→ [合并输出] 全文章节拼接为 .md

小说模式线（v2.0.0b0 起，独立一条线）：

场景配置（人物/时代/地点/冲突）

→ 章数组（短3-6/中8-10/长11-15）+ 因果链验证

→ 逐章循环：

章内子结构规划（S01-S05，用户确认门控）

→ 逐段写作（上下文注入：角色表/人格/实体关系/时间线/情绪/上章轨迹）

→ 章检 (4维 8B + 格式 + 逻辑 + 推理 R1)

→ HARD → 修复弹窗 (T0 自动修 / T1 重构 / 跳过=通过)

→ 全书所有章节 done → 全文三检 (忠实度 / 承诺 / 收束) → 三检修复弹窗

1.1 核心设计原则

原则	说明
模板 > 空白	文章结构由模板定义 (meta+content)，LLM 和用户都在模板确定的边界内操作
子结构 > 整段	节拆分为 2-4 个子结构逐段写作，每段独立调 LLM，避免长上下文注意力衰减
材料分级 > 一股脑	节级 RAG 提供背景上下文，子结构级 RAG 提供针对性素材，文档元数据全局共享
引用后处理 > LLM 格式	LLM 写自然语言标记 (引用自{文件名})，确定性代码替换为编号 [N] 并生成参考文献
续写 > 截断	token 耗尽时不丢弃已写内容，自动续写最多 5 轮
可配置 > 黑盒	模板所有字段可编辑，大纲所有字段均可交互调整
容错 > 崩溃	RAG 超时不塞 prompt，空内容跳过，写作者异常降级为错误提示
状态持久 > 内存	进度写入 state_manager，断线重连可恢复
文件真相源 > session 状态	章级/段级续写跳过判定以磁盘文件为准，防 session 与磁盘分叉重复重写
跳过 = 通过	修复弹窗：勾选=重构，不勾选/全部跳过=立即标记通过 (不再重检)
全文三检只检全文	全文三检仅在全书所有章节 done 后触发一次 ("if 规划 else 全文三检")
章内检测不产章级问题	章内推理审核 sub 必填、非法/缺段直接丢弃；章级/跨章判定归全文完结层 (finalize_novel) 统一做，两级互不污染

原则	说明
结构级校验 > 语义猜测	代码只做枚举/结构校验（_classify_items）；引文回验：detail 的「」引文必须逐字存在于对应段正文，否则视为伪造审核丢弃——语义对错归 prompt（级别定义+正反例），代码绝不猜语义

二、后端架构（v3.1.0b1 定稿：LM Studio 统一管理）

历史决策：v3.1.0b1 起 llama.cpp 直挂后端整体废弃——llama-cpp-python 0.3.34 旧内核无新 MoE 优化，35B 写作仅 8 t/s vs LM Studio 20+ t/s。llm_client.py 删除全部 llama.cpp 分支，保留纯 HTTP 客户端。

角色	模型	后端	生命周期
写作/规划（35B）	qwen/qwen3.6-35b-a3b	LM Studio（lms load → GPU → HTTP localhost:1234）	任务内复用，任务结束自动卸载
判定 4维（8B）	qwen3-8b	LM Studio（统一管理勾选） / transformers 3B（不勾）	章检测完即卸（lms unload）
判定 R1（7B）	deepseek-r1-distill-qwen-7b	同上	同上
实体/行为/时间线提取（3B）	Qwen2.5-3B	永远 transformers（CPU）	常驻复用
通用线写作	用户配置（LM Studio/Ollama）	HTTP（1234 / 11434）	会话内

- **统一管理勾选**：planner/writer 后端都是 LM Studio 时可勾选 → 判定 8B/7B 走 LM Studio GPU；不勾 → transformers 3B/1.5B
- **ollama 场景**：写作后端为 Ollama 时统一管理禁用（判定模型仍是 LM Studio/transformers）
- **LM Studio 生命周期**：novel/lmstudio_probe.py 封装 lms.exe（load/unload/ps/import/server start）；判定模型用 make_lms_handle（带 _lms_model_key 标记）→ release 识别 lms 句柄 → lms unload
- **模型管理**：8B/7B GGUF 位于 LM Studio 模型库 ~/.lmstudio/models/（本地移入或自动下载 + lms import）；judge_model_keys 从 lms ls 解析（支持裸名 key 与 user/repo 形态）
- **窗口**：判定模型固定 16384（R1 思考链+JSON ≈13K）；8B 判定带 /no_think 关思考（LM

三、三层架构

层	组件	职责
表现层	web_ui.py (port 8770) / 配置 Tab / 对话 Tab	Web 界面、LLM 配置面板、模板编辑器、大纲交互、小说质检配置、修复面板、进度展示
业务层	planner.py / writer.py / rag_client.py / llm_client.py / state_manager.py / citation_validator.py + novel/* 子包	通用线（大纲规划、逐段写作、RAG、LLM、状态、引用）+ 小说线（规划/写作/章检/全文三检/修复）
基础设施	config_manager.py / main.py / data/ / setup.bat	配置读写、服务入口、数据持久化、一键启动

3.1 完整文件结构

structured-writer/

├─ main.py

├─ setup.bat

├─ requirements.txt

├─ CHANGELOG.md

├─ SCHEMA.md

├─ README.md

├─ LICENSE

├─ NOTICE

├─ pyproject.toml

├─ scripts/check_version.py

├─ docs/

├─ blueprint.json

├─ config.json

★ 入口 (HTTP 服务器 + 对外写作 API 8777)

Windows 一键启动 (任意 Python 版本, 自动装 transformers)

依赖清单 (transformers/torch, 无 llama-cpp-python)

版本更新日志 (Keep a Changelog 格式, 版本号唯一源见 __init__)

方案文档

项目说明

Apache 2.0

Apache-2.0 归属声明 (v3.1.9)

PyPI 打包配置 (dynamic version 动态读 __init__)

版本一致性门禁 (__init__/CHANGELOG/pyproject/README 头部四点校验)

设计文档 (repair_engine_design.md 等)

PyPI 发布蓝图

默认配置 (不含模板)

├─ structured_writer/

| └─ __init__.py

| └─ web_ui.py

| └─ planner.py

| └─ writer.py

| └─ rag_client.py

| └─ llm_client.py

| └─ state_manager.py

| └─ citation_validator.py

| └─ config_manager.py

| └─ external_api.py

★ 智能体核心包 (PyPI 包名 structured-writer-ldxs)

版本号唯一源

HTTP 服务器 + 前端 (GET/POST 路由表分发; 通知统一 toast, 无原生弹框)

通用线大纲规划器 (LLM 生成 JSON 大纲)

通用线串行写作器 (两级 RAG + 续写 + 引用后处理)

RAG 客户端 (调 rag-assistant :8767)

LLM 统一客户端 (纯 HTTP: LM Studio / Ollama)

会话状态管理 + 修复提示 (_repair_hints + repair_pending)

引用扫描+报告 (LLM 判定已删——引用编号由 all_rag_headers 前置规范确定性完成)

配置读写 + 模板分离存储 + 旧格式迁移

对外写作 API (/api/write, 8777 独立端口)

		aux_parser.py	# 辅助解析
		md2tex.py	# md → tex + xelatex 编译
		novel/	# ★ 小说模式子包
			novel_bridge.py # 场景配置 → 章数组 → 因果链 → outline → 项目初始化 → plan-chapter
			novel_writer.py # 小说写作引擎 (逐章 plan-chapter + 写作 + 章检门控 + 全文三检触发)
			novel_workflow_engine.py # 章检/全文三检编排 (子进程 finalize-chapter/finalize-novel)
			novel_repair_engine.py # 修复引擎 (T0 自动修/T1 重构、轮次、三检当场重检)
			novel_4dim_check.py # 章检 4 维判定 (时间/情绪/话题/角色, 8B 或 3B)
			novel_reasoning_check.py # 推理审核 R1 (5 维; 级别定义+正反例+detail 三要素+引文回验)
			novel_fidelity.py # 大纲忠实度检查 (全文三检)
			novel_pledge_check.py # 全文承诺检查 (全文三检)
			novel_logic_check.py # 逻辑检查 (4维 回退链)
			novel_entity_extractor.py # 实体提取 (3B + 正则兜底)
			novel_behavior_extractor.py # 行为提取 (3B + 正则兜底)
			novel_timeline*.py # 时间线提取 (3B + 正则兜底, day 累计解析)
			novel_state_manager.py # novel_state.json 读写
			novel_atomic_writer.py # 原子写入 + 未行标记
			novel_character_registry.py # 角色注册表 (name+aliases, 占位符拦截)
			novel_continuity.py # 规则连通性检查 (4维 回退)
			novel_style_check.py # 风格检查
			model_backend.py # 判定模型后端路由 (LM Studio / transformers) + _model_profile
			lmstudio_probe.py # LM Studio 环境探查 + lms 生命周期封装
			model_env_check.py # 环境探测 (transformers/torch 缺失自动安装)
		plugins/	# 数据源插件系统
			base.py / manager.py
			builtin/db_source/ # 数据库数据源插件 (SQLite/MySQL/PG/CSV)
		data/	# 运行时数据 (不出库)
			config.json # 运行时配置
			sessions/{id}.json # 对话状态
			archives/sessions/ # 归档会话
			outputs/{name}_{ts}/ # 生成结果 (md + 图片集)
			templates/user_templates.json # 用户自定义模板 (内置模板在代码中)
			examples/examples.json # 快速范例
			novel/projects/{id}/ # 小说项目 (data/novel_state.json + chapters/<章>/*.txt)
			models/ # transformers 模型 (Qwen2.5-3B / R1-1.5B 等)

四、模板系统

4.1 模板结构

每个模板由四个部分组成（小说模板额外带 `novel` 标记）：

```
{
  "meta": [...],           // 元数据区 — 短标识信息（标题/作者/单位/文号等）
  "content": [...],        // 内容树区 — 文章正文结构
  "style": "...",          // 风格提示词 — 控制文风语气
  "logic": "...",          // 逻辑提示词 — 控制 LLM 写作认知流程顺序
}
```

```
// 小说模板: "novel": {"mode": true, "kind": "chapters"} 等
}
```

meta 字段：每条含 `name/show_label/desc/source`。source 分三类：

source	含义
user	用户必须填写（如作者、单位、文号）
auto	用户可选填，留空由 LLM 生成（如标题）
llm	由 LLM 生成（如关键词），推荐放 content

content 字段：每条含 `name/show_label/type/kind/desc`（通用线）+ `citation_check/citation_format`（引用节）。

type	含义
leaf	单段内容，不拆子结构（摘要、关键词、参考文献）
section	需要拆 2-4 个子结构（引言、方法、结果、结论）

kind（小说模板特有）	含义
setting	设定节点（世界观设定/人物表），不输出正文，存状态
chapters	正文（多章锚点，由 AI 自由展开 L01-L15）

logical_order：控制 LLM 写作的认知流程顺序，而非文章最终顺序（0=先写 / 1=其次 / 2=最后写）。

4.2 内置模板 vs 用户模板

- 内置 9 套（代码常量，只读）：日常写作 / 学术论文 / 正式公文 / 新闻报道 / 技术报告 / 通用公文 / 论文综述 / 自定义 / **小说**
- 用户模板存 `data/templates/user_templates.json`，可创建/编辑/删除（内置只读）
- 另存为副本继承 `novel.mode/kind`；小说模板「题材」「篇幅」字段锁定（UI 禁用 + 保存校验双保险）

4.3 引用校验

content 字段支持 `citation_check` 和 `citation_format` 配置（`[x]=1`，格式：`=` 前为行内标记模板，后为参考文献列表前缀）。字段名含"参考文献"或"引用"时 `_normalize_template()` 自动设 `citation_check=true`。

4.4 模板生成（对话生成）

`POST /api/gen-template` → LLM 生成模板 → 多级 JSON 容错解析（最多重试 3 次）→ `_normalize_template()` 校验补默认值 → 存用户模板。

五、组件详解

5.1 通用线大纲规划器 — `planner.py`

`plan_outline()` 调用 LLM 根据模板的 meta+content 生成结构化 JSON 大纲。
`parse_outline()` 4 种解析策略（直接 `json.loads()` / `` ` json 提取` / `` ` 提取` / 首 `{` 截取），失败追加纠正指令重试（最多 3 次）。子结构规范化：leaf 不补、section 自动补默认子结构、补全 id/summary/word_count/_checked/_logical_order。

5.2 通用线串行写作器 — `writer.py`

`generate_article()` 逐节逐子结构串行调用 LLM：

- **两级 RAG**：节级查询（标题+节标题+summary）→ 【背景资料】；子结构级查询 → 【针对性资料】；`all_rag_headers`（文档元数据）全局共享
- **引用后处理**：正则扫描 引用自{文件名} → 去重编号 → 替换 `[N]` → 构建参考文献节
- **续写机制**：`finish_reason=="length"` 时追加"请继续写"重试（最多 5 次）；空内容放弃
- **事实自检**：内嵌标记法 【事实待核查】（零额外 LLM 调用），文末汇总「建议人工复审」清单
- **错误容错**：LLM 异常写错误提示、空内容跳过、RAG 超时不塞 prompt

5.3 小说线配置统一源 — `novel/nover_config.py` (v3.1.7)

小说线散落在各文件的常量统一收拢为单一配置源：`LENGTH_TARGETS` / `LENGTH_LABELS` / `LENGTH_CHAPTERS`（long 统一 11-20 章，去掉 `if i>15: break` 硬截断）/ `KEY_UPSCALE=1.5`（C4 通用线关键词放大）/ `JUDGE_TEMPERATURE` / `JUDGE_MAX_TOKENS`（C8 判定）/ `REPAIR_WORD_TOLERANCE` / `DEFAULT_NOVEL_STYLE`。涉及 `planner.py` / `writer.py` /

novel_workflow_engine.py / novel_state_manager.py / novel_context_loader.py / novel_writer.py / novel_bridge.py 等 10+ 文件统一 import。v3.1.8 通用线子结构字数同样统一：SUB_WORDS_MIN=200 / SUB_WORDS_DEFAULT=500 / SUB_WORDS_MAX=800 (planner.py 顶部常量，13 处硬编码替换)。

5.4 小说线入口 — novel_bridge.py

generate_scene_config(topic, template) # 场景配置 (人物含 MBTI+荣格原型/时代/地点/冲突/文风)

generate_chapters(setting, topic, length) # 章数组 (短3-6/中8-10/长11-15, 概述≥12字符+因果动词)

verify_causality(chapters) # 因果链验证 (不过 → 反馈重生成, 最多 2 次)

init_novel_project(project_id, ...) # 项目初始化 data/novel/projects/{id}/

plan_chapter_subs(state_path, chapter) # 章内子结构规划 (S01-S05, tone/emotions/writing_prompt≥50 硬校验, 末章 is_ending)

plan_novel_outline(...) # 主入口: 场景配置 → 章数组 → 因果链 → outline + 项目初始化

5.4 小说线写作引擎 — novel_writer.py

generate_novel_article() 逐章循环 (每章规划→确认→写作→章检):

- **续写恢复 (文件为真相源)**: 章子结构文件齐全 → 无论 session 状态跳过 + 同步 done; 文件不齐按状态降级
- **章级门控**: 仅 pending 章规划子结构; planning 等待用户确认 (_wait_confirm 轮询)
- **上下文注入**: 角色表/人格/实体关系网/时间线/情绪基调/上章行为轨迹/写作命题框 (三层分区: 目的★★★/背景★★/参考★)
- **写入**: 进程内直接落盘 (_write_sub_inline, 原子写 + 别名拦截 + 实体提取 + 字数三档校验), 失败停止整章
- **章检门控**: finalize_novel_chapter (子进程) → 有 HARD/FAIL → 修复轮次循环 (repair_rounds 默认 3) → 通过标章 done
- **全文三检守卫**: _all_done = all(section.status == 'done') —— 全书所有章 done 才执行 finalize_novel_full ("if 规划 else 全文三检")

5.5 章检与全文三检 — `novel_workflow_engine.py`

章检 (`finalize-chapter`, 每章):

检 查	内 容	模 型
4维	时间衔接/情绪匹配/话题过渡/角色承接（共享上下文 + 判定提示词分隔，叙事目的哲学）	8B (/no_think) 或 3B
格 式	末行标记/禁用模式/文件数	规则 (毫秒级)
逻 辑	角色消失/时间回退/概述偏离/实体关系断裂	3B (4 维 回退 链)
推 理 R1	因果合理性/人物行为一致性/情绪弧自然度/对话匹配度/论证可靠性（5 维）；判定 级别定义+  /  正反例+一致性铁律（v3.1.11）；detail 三要素=「」引用+对照 +方向 30-120 字（v3.1.12）；引文回验+段定位校验——引文不在正文即伪造审核 代码层丢弃（v3.1.13）；sub 必填，章级问题不落章内（v3.1.11）	7B

全文三检 (`finalize-novel`, 全书 done 后):

检查	内容	模型
fidelity 大纲忠实度	L1 词面全量筛 → 覆盖率<0.6 可疑段 3B 复核	3B
pledge 全文承诺	3B 按正文提取 flag → writer 推理判已兑现/未兑现/悬停	3B + writer
ending 结尾收束	末章末段 → 3B 判封闭/开放/悬停三型	3B

问题写入 `novel_state._full_repair` → `novel_writer` 同步进
`session._repair_hints[chapter].full_items`（与章检 issues 同层统一判定）。

5.6 修复引擎 — `novel_repair_engine.py`

```
run(state_path, chapter_dir, chapter, issues, mode, checked_subs,
repair_types) :
```

- **T0 自动修**（纯格式：末行编号/禁用模式）：代码直改正文，无弹窗
- **T1 重构**（内容问题）：勾选的子结构 → 写作模型重构（整段契约：保留首行标题/末行编号/别名/字数 ±15%/只输出正文）→ 校验 → 原子写 + 备份
- **R1 引导式局部改写**： `_build_guided_prompt`（完整正文 + R1 detail 问题描述，writer 自行定位局部改写）→ `_validate_guided`（三行保留 + 输出与原文有差异）
- **三检类型化重构**：fidelity/pledge/ending 变体指令 + `source_text` 定位；pledge 减法 = 移除悬置承诺+平滑衔接
- **当场重检**：勾选修复 = 重构成功后当场重跑对应检查（章检六项 / fidelity / pledge / ending），通过才移除修复项
- **模式**：手动（无上限，刷新面板） / 自动（`repair_rounds` 上限）；跳过=通过（`_repair_result.skipped` → 不再重检不重置）
- **共享实例**： `_create_repair_client` 复用共享 35B（规划/写作/修复同一模型只加载一次）；`_release_repair_client` 去 close（防僵尸实例）

5.7 判定模型后端 — `model_backend.py`

- `judge_backend(cfg)`：统一管理勾选 → lmstudio；否则 transformers
- `judge_model_keys`：lms ls 解析 8B/7B key（裸名 key 与 user/repo 形态）
- `lms_generate`：HTTP 生成（Qwen 系 chatml）；`make_lms_handle`：可调用句柄（带 `_lms_model_key` 标记）
- `ensure_judge_models`：缺失检测（库无 8B/7B → 明确提示下载 + 回退 transformers 3B）
- `release`：识别 lms 句柄 → lms unload（测完即卸）；transformers 句柄 → del + gc
- `_model_profile(model_cfg)`：按后端取配置槽（profiles 分槽，web_ui/repair 共用，旧格式兼容）

5.8 LLM 客户端 — `llm_client.py`

纯 HTTP 客户端（LM Studio / Ollama 双后端）：

后端	协议	接口	默认端口
LM Studio	HTTP (OpenAI 兼容)	<code>/v1/chat/completions</code>	1234
Ollama	HTTP (Ollama API)	<code>/api/chat</code>	11434

- `chat_detailed()` 返回 `{content, finish_reason}` (续写检测); `chat()` 返回纯文本
- `list_models()` 30 秒 TTL 缓存 (key=backend|base_url——LM Studio API 实测 2 秒, 缓存后秒回)
- `max_tokens/temperature` 从 config 读取存储为实例属性

5.9 状态管理器 — `state_manager.py`

管理会话全生命周期:

`load/get_state/set_phase/update_section/get_progress/set_status_text/fingerprint_check` + 修复提示:

- `save_repair_hint(chapter, result)`: 覆盖时保留已有 `_repaired` (防跳过被章检覆盖又弹)
- `get_progress()` 的 `repair_pending`: 章检 issues HARD/FAIL 或三检 `full_items` 非空且未 `_repaired` → 覆盖式最近章

5.10 配置管理器 — `config_manager.py`

- `config.json` 原子写入 (tmp → replace), 深层合并默认值
- `planner/writer` 模型配置: **profiles 分槽结构** `{backend, profiles: {lmstudio, ollama}}` (v3.1.0b1 移除 llama.cpp profile, v3.0.0b37 引入分槽, 前端切后端自动恢复对应槽)
- 内置模板 `DEFAULT_TEMPLATES` + 用户模板 `data/templates/user_templates.json`
- 旧格式迁移 `_migrate_old_templates()`

六、外部接口

6.1 HTTP 端点 (Web UI, 8770)

通用线:

端点	方法	说明
<code>/</code> 或 <code>/index.html</code>	GET	主页面 (配置 Tab + 对话 Tab)
<code>/api/config</code>	GET/P OST	读取/保存配置 (含模型 profiles 分槽)

端点	方法	说明
/api/llm/models	GET	扫描模型列表（30s 缓存）
/api/llm/test	POST	测试 LLM 连接
/api/llm/window	GET	窗口信息提示（只提示不强制）
/api/plan	POST	生成/重新规划大纲
/api/generate	POST	启动生成任务
/api/progress	GET	获取生成进度
/api/result	GET	获取最终结果
/api/stop	POST	停止生成
/api/sessions	GET	会话列表
/api/session/new load archive restore delete	-	会话管理
/api/chat	POST	对话消息处理（含写作意图检测）
/api/gen-template	POST	LLM 生成模板
/api/rag/status start stop	-	RAG 状态/冷启动/停止
/api/batch_auto / /api/batch_progress	-	批量自动撰写
/api/outputs / read / delete	-	已完成文章管理
/api/outputs/merge	POST	章级 md 拼合为整本（v3.1.14 修复：自 v3.1.3 起误注册 GET 表致拼合 404，从未成功）
/api/examples	-	快速范例

**小说线 (/api/novel/*): **

端点	方法	说明
/api/novel/status	GET	小说质检状态（判定后端/模型就绪/配置）
/api/novel/checks	POST	保存质检配置（章检/全文三检开关/统一管理/修复轮次）
/api/novel/install	POST	安装缺失模型（8B/7B 下载到 LM Studio 模型库 + import）
/api/novel/confirm	POST	章内子结构确认（勾选/字数覆盖/重点）
/api/novel/replan_sub	POST	子结构级重规划
/api/novel/replan_statuses	GET	重规划 in-flight 状态
/api/novel/repair/preview	GET	修复面板分级清单
/api/novel/repair/apply	POST	开始修复（checked_subs + repair_types）
/api/novel/repair/rollback	POST	回滚正文+state
/api/novel/repair/skip	POST	全部跳过（= 标记通过，不再弹）
/api/novel/repair/status	GET	修复进度轮询（带 session_id 会话隔离）

对外写作 API (8777 独立端口): POST /api/write（同步写作：prompt/template/images/rag/format）、GET /api/health、GET /api/capabilities。

6.2 RAG 外部依赖

依赖	端口	说明
rag-assistant 外部 API	8767	知识库查询 (/api/kb/query)，v2.2.10+ 提供文档元数据

依赖	端口	说明
rag-assistant 冷启动	18765	子进程 Web UI 端口（冷启动用）

6.3 配置结构（config.json）

```
{
  "planner_model": {"backend": "lmstudio", "profiles": {"lmstudio": {...},
"ollama": {...}}},
  "writer_model": {"backend": "lmstudio", "profiles": {"lmstudio": {...},
"ollama": {...}}},
  "selected_template": "通用公文",
  "rag_path": "",
  "context_review_length": 8000,
  "fact_check_enabled": false,
  "max_sessions": 20,
  "novel_checks": {"chapter": true, "format": true, "reason": true,
    "full_fidelity": true, "full_pledge": true,
"full_ending": true,
    "auto_repair": false, "repair_rounds": 3,
    "unified_management": true}
}
```

- 模型配置为 **profiles 分槽**：切后端（lmstudio/ollama）自动恢复对应落盘配置（地址/模型/参数互不覆盖）
- 小说质检配置存 `novel_checks`（章内 4 维/格式/推理 R1 + 全文三检三开关 + 自动修复/轮次 + 统一管理）

七、UI 布局

对话界面为三栏布局（会话管理 / 对话交互 / 已完成文章）。配置 Tab 含：模型配置（后端下拉 LM Studio/Ollama + 地址 + 模型列表）、模板管理、RAG 配置、写作参数、**小说质检区**（章内检测/全文检测分组 + 点位标注 + 统一管理勾选 + 检测模型按钮）。

通知与确认统一走自绘组件（v3.1.14）：错误/警告/成功用右上角 toast（`showToast`：error 红 6s / warn 橙 3.5s / info 绿 3.5s，点击即消），删除走 inline 二次确认——全站无浏览器原生 alert/confirm/prompt。

7.1 小说确认面板

章内子结构规划完成后弹出：勾选（取消=跳过）、字数覆盖、重点标记。确认后进入写作。

7.2 修复面板（章检 HARD / 三检问题）

— L02 《信号与躯壳》全文质检需处理 —

HARD ☒ SOFT ☒ （级别过滤，未勾选级别子结构隐藏）

☒ [4维] S02 话题过渡无叙事目的

☒ [推理] S02 对话匹配度（用词超出角色）

☐ [R1] S04 情绪弧转变突兀

勾选 = 用写作模型重构修复；不勾选 = 立即标记通过

[全部跳过] [开始修复]

- T0 自动修不弹窗；T1/三检进弹窗；跳过=通过（不再弹）
- 手动模式无上限（修复→自动重检→刷新面板）；自动模式有轮次上限

八、写作管线

8.1 通用线完整请求生命周期

用户点击"开始生成"



POST /api/generate → _handle_generate()

- 加载会话状态 → 应用勾选过滤 → 应用排序
- 探测 RAG 8767 → 创建 RAGClient（如果在线）
- 从模板构建 citation_config → 创建 LLMClient（配置读取）
- 启动后台线程 _run_generation()



generate_article(outline, rag_options, llm_client, state_mgr, rag_client, template, citation_config)

- 逐节（按 logical_order 排序写作，按 content[] 顺序输出）

```

|   |— 节级 RAG (all_rag_headers 累积) → 写入 ## 节标题
|   |— 逐子结构：子结构 RAG → 构建 prompt → LLM → 续写 → 事实标记 → 写入 ### 子标题
|   |— 引用后处理 (citation_check) → 事实自检汇总 → 引用验证
|   |— 写入 .md → data/outputs/{timestamp}_{title}/
|   |— 更新 phase="done"

```

8.2 小说线完整请求生命周期

选择「小说」模板 → 输入题材/篇幅 → 生成



plan_novel_outline(): 场景配置 → 章数组 → 因果链验证 → 项目初始化



generate_novel_article() 逐章循环:

```

|— 章内子结构规划 (plan_chapter_subs) → 确认面板门控 (_wait_confirm)
|— 逐段写作 (三层上下文注入 + 原子写 + 实体/行为/时间线提取)
|— 章检 (finalize_novel_chapter 子进程) → HARD 拦截 → 修复轮次循环
|   |— 修复引擎 run(): T0 自动修 / T1 重构 / R1 引导式 / 三检类型化 → 当场重检
|— 通过 → 标章 done → 章级 md 落盘

```



全书所有章节 done → 全文三检 (finalize_novel_full) → 三检修复项同步 session.full_items



三检修复弹窗 (勾选修复当场重检 / 全部跳过) → 处理完放行 → 整本拼合 (手动)

九、版本演进要点

版本	新增/变更要点
v3.1.1 4	web_ui: 拼合路由 GET→POST 归位 (自 3.1.3 起拼合从未成功过); 12 处原生 alert() 全部 toast 化, confirm/prompt 零残留
v3.1.1 3	推理审核引文回验 (「」引文逐字回验正文, 伪造审核丢弃) + 段定位校验; prompt 术语统一 (正文唯一称谓) + 评估对象逐维写明; check_version 四点校验 (README 头部版本戳: 有则必须等于 __version__, 中英文标签泛化)
v3.1.1 2	推理审核 detail 三要素 (「」引用+对照+方向, 30-120 字); 正例去泛泛句——修复引擎拿到可定位问题, 斩断"盲重写→再审→再报"死循环
v3.1.1 1	判定型 prompt 范式统一 (级别定义+  /  正反例+一致性铁律, 8 处); 代码删语义关键词猜测只留结构校验 (_classify_items); 章内/章级检测分离 (sub 必填, 章级归 finalize_novel)
v3.1.9	NOTICE (Apache-2.0 归属声明)
v3.1.0 b5	8B/7B 全链路实机验证 (7B 推理审核 / 8B /no_think 关思考); 纯验证无代码改动
v3.1.0 b4	lms ls 裸名 key 过滤修复; 8B/7B 移入 LM Studio 模型库 (自动识别无需 import)
v3.1.0 b3	判定窗口 UI 移除 (固定 16384); ollama 场景禁用统一管理
v3.1.0 b2	setup.bat 去 py 3.11 优先 (任意 Python 版本)
v3.1.0 b1	llama.cpp 直挂后端整体废弃 → LM Studio 统一管理 (写作/规划 35B + 判定 8B/7B); 新增 lmstudio_probe.py; llm_client 删全部 llama.cpp 分支; config 移除 llama.cpp profile
v3.0.0 b33- 41	llama.cpp 推理参数优化 (后被 b1 废弃); 模型配置 profiles 分槽; 模型恢复修复 (_modelValues 内存值); 模型列表缓存

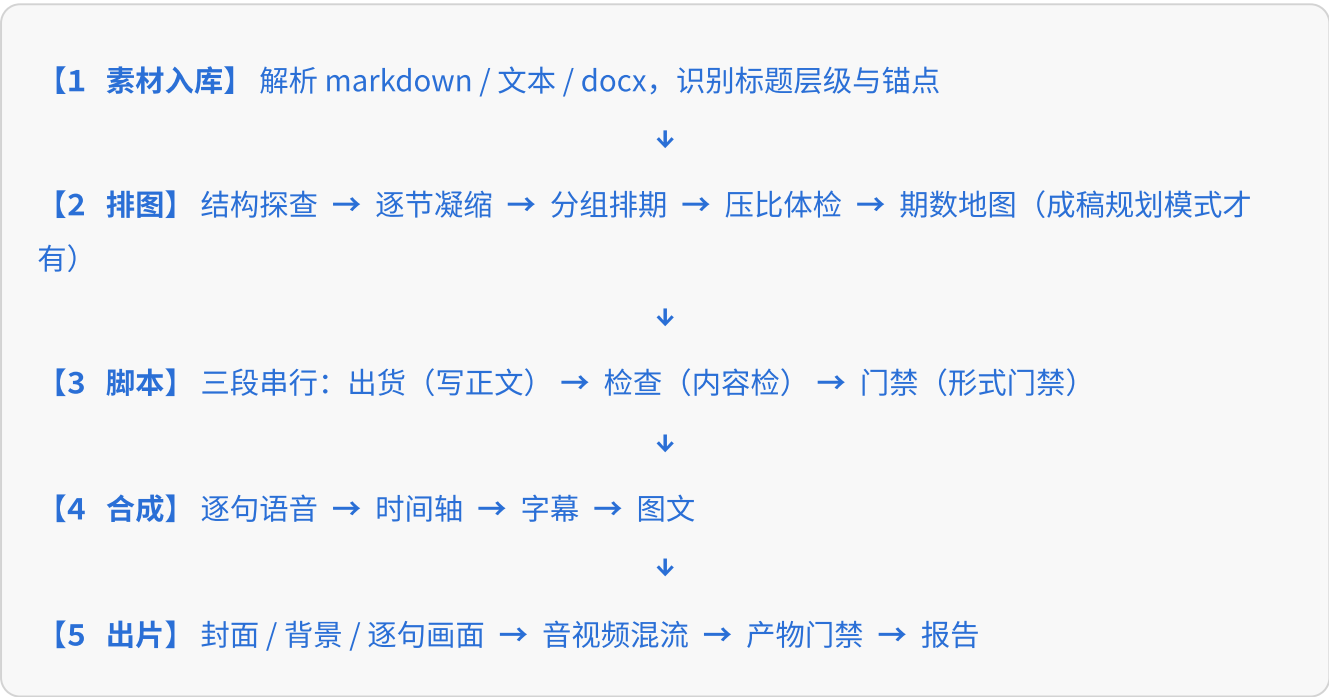
版本	新增/变更要点
v3.0.0 b29- 32	跳过=通过；全文三检触发守卫（if 规划 else 全文三检）
v3.0.0 b20- 28	35B 不常驻（weakref 共享）；修复引擎共享/独立/防重入/会话隔离/静默失败修复/超窗降级
v3.0.0 b17- 19	llama.cpp 实例进程级共享；判定模型弱引用（测完即卸）
v3.0.0 b15- 16	修复面板 HARD/SOFT 级别过滤
v3.0.0 b10- 14	章检 prompt 哲学（内容一致→叙事目的）；KV 量化 flash_attn；窗口动态；n_gpu_layers VMM 修复
v3.0.0 b0	门禁体系废除；模型架构 3→2（去 bge）；全文三检全 LLM 化；三检修复弹窗复用 + 当场重检
v2.4.0 b0	重规划全链路 UI；三层分区上下文；三提取器统一（实体/行为/时间线）
v2.3.x	续写文件真相源；实体清洗死循环；章检 HARD 拦截（ok→issues 判定）
v2.0.0 b0	小说模式引入 （P1-P4：模板+路由 / novel 子包 / 模型层 / 检查体系）
v1.9.0 b0	插件系统（数据源插件）；大表蓝皮书取数
v1.7.0	对外写作 API（8777）；md2tex/pdf；快速范例；两级局部重规划；_tpl_key 血缘
v1.2.0	辅助资料（图片/文字/表格特化管线）；输出目录化
v1.1.0 b9	模板存储分离；引用校验字段；IMRaD 内置模板；三栏布局

版本	新增/变更要点
v1.0.2 8	事实自检内嵌标记法；temperature 可配置；RAG 停止；批量自动撰写；会话归档
v0.2.5 b3	PyPI 发布准备：app/ → structured_writer/
v0.1.0	项目骨架、LLM 客户端、会话管理、大纲规划器、串行写作器

06 | podcast-maker 架构与规范体系文档

一、系统概览

Podcast Maker 是**管线型智能体**：输入是素材，输出是成片。中间五步，每一步都有独立的产物落盘、独立的判据、独立的失败方式：



它不是"一次调用出一个成品"的写法，这是全文的总纲。生成一段文本、写一期脚本、出一期片，每一步都在环里：判一次 → 指出问题落在哪一句 → 定点改那几句 → 再判。**环是它的基本执行单位，串行是环与环之间的连接方式**——这条主线在第四章逐层展开。

1.1 核心设计原则

原则	说明
判与写分离	写的人不管合不合规，判的人不重写稿子；对一份可用稿，只有"判"与"定点修"两种动作
环有界	每个环都有明确的轮次上限，全部来自配置或模块常量；没有"改到好为止"的无限循环

原则	说明
每轮落盘	一轮定稿就写一次盘——生成一轮十几分钟，等全程跑完才第一次写盘，中途卡住就什么都没有
判断权在人	轮次用尽仍不过 → 带着问题交人，不空转、不硬猜、不假装通过
前置规范 > 后置验证	能写进 JSON Schema 的约束不靠提示词劝；枚举里去掉就是当场填不出来
单一出口	会变的设定各有唯一来源函数，提示词、门禁、界面同源，不存在"两份表"
单位同源	一切字数账走同一把尺（有效字：汉字 1、标点 0.5、西文词 1.5）；不同源的数不许相减
确定性特权	排版、合成、粘合、字幕换行这些可枚举的事，交给代码，不给模型
测量与验收同尺	判定与修复共用同一把尺（音色体检的点名与重出候选验收走同一组判据）；测量的量（基频、谱心、时长）只住在唯一的换算入口里

1.2 三种项目模式

模式在立项时定死，之后不可改——它决定的不是"偏好"，而是**这条路成立的前提存不存在**：

模式	料从哪来	排不排图	走哪条写作路	前提
成稿规划 (mapped)	已有整部作品	排	分段（逐期配额、防重复、段级核账）	有地图与逐节凝缩，才摊得出配额、才谈得上"这段该讲多少"
逐期即兴 (episodic)	每期临时给	不排	整篇	当场给料、出完即止，没有凝缩可分
单集 (single)	当场给	不排	整篇	恒一期，出完即完结

三条路的岔口只有一个：**有没有"逐节凝缩"这个中间层**。有，配额与查重才有依据；没有，整篇一次写完就是这条路该有的样子。这不是三套实现——脚本阶段之后（检查段、门禁段、定点修补、粘合）三条路汇进同一套判据，一处不多、一处不少。

成稿规划模式还有一道退路：本期凝缩读不出来（项目还没排图、版本对不上）就**退回整篇**照常出稿。理由是一句工程话：**不让人卡在"一步跑不动"上**。

二、三层架构

2.1 三层

层	组件	职责
表现层	<code>web_ui.py</code> (HTTP 服务 + 四个页面: 项目 / 脚本 / 合成 / 配置)	立项建树、勾期跑批、脚本生成与判稿、合成出片、配置编辑、任务进度轮询
业务层	<code>pipeline.py</code> (单期管线编排)、 <code>script_engine.py</code> (脚本: 生成 + 检查 + 门禁 + 修补)、 <code>planner.py</code> (排图)、 <code>probe.py</code> (结构探查与凝缩)、 <code>ingest.py</code> (素材解析)、 <code>source_store.py</code> (两级取料)	五步管线各自的判据与动作
基础设施	<code>config_manager.py</code> (配置唯一定义处)、 <code>llm_client.py</code> (后端兼容)、 <code>project_store.py</code> (项目登记)、 <code>layout.py</code> (目录布局唯一定义处)、 <code>duration_model.py</code> (时长模型)、 <code>bins.py</code> (ffmpeg/ffprobe 定位唯一定义处)、 <code>assets_factory.py</code> / <code>audio_engine.py</code> / <code>tts_engine.py</code> / <code>subtitle_engine.py</code> / <code>video_engine.py</code> / <code>aigc_label.py</code>	配置、通信、落盘、时长、画面与声音

三处"唯一定义处"值得点名: `layout.py` 定义项目内的全部路径, `config_manager.py` 定义全部配置点位 (含类型、分组、界面形态、默认值), `duration_model.py` 定义字数与时长之间唯一的换算系数。任何一处要拼路径、读配置或做换算, 都从这三个地方取——各写一份, 就是同一件事有两个真值来源。

2.2 完整文件结构

podcast-maker/	
├─ main.py	# 入口 (Web 服务 / --check / --continue / --voices / --fonts)
├─ config.json	# 用户配置 (平铺点号键, 如 "script.gate_rounds")
├─ setup.bat	# Windows 一键启动 (装依赖 + 起服务)
├─ requirements.txt	# 轻依赖 (第三方只有两个包)
├─ CHANGELOG.md / PROTOCOL.md / README.md / ARCHITECTURE.md	
├─ podcast_maker/	# ★ 智能体核心包

		__init__.py	# 导出 + __version__ 唯一源
		web_ui.py	# HTTP 服务、四个页面、任务轮询
		pipeline.py	# 任务队列 + 单期管线编排 (脚本 → 合成 → 出片 → 报告)
		script_engine.py	# ★ 脚本引擎: 生成、检查、门禁、定点修补
		planner.py	# 排图: 分组排期 + 压比体检重排
		probe.py	# 结构探查: 分层、锚点、逐节凝缩
		ingest.py	# 素材解析 (纯解析, 不调模型)
		source_store.py	# 成稿入库 + 两级取料 (整期素材 / 逐节原文)
		paradigms.py	# 范式卡、风格卡、对话形式卡 (数据层, 不是代码分支)
		layout.py	# 项目目录布局的唯一定义处
		config_manager.py	# 配置点位的唯一定义处
		llm_client.py	# 多后端 LLM 客户端 (仅标准库 urllib)
		duration_model.py	# 时长模型 (字数 ↔ 时长 ↔ 语速 的唯一换算入口)
		bins.py	# ffmpeg / ffprobe 的唯一定位、探测与一键安装 (PATH → 项目 bin/ → None)
		project_store.py	# 项目登记表
		tts_engine.py	# 语音合成: 音色注册表 + 逐句合成 + 时长回填 + 试听
		audio_engine.py	# 编码参数校验 + 拼接 + 降噪 + 响度归一 + 背景音乐
		subtitle_engine.py	# 字幕: 断行 (唯一入口) + 四份同源产物 (SRT / LRC / 整秒 TXT / 洁版 TXT) + 断词率统计
		video_engine.py	# 画面: 背景 + 动画 + 说话人指示 + 字幕烧录 → 横竖屏 mp4
		assets_factory.py	# 资源工厂: 背景图、封面三尺寸、BGM —— 全部代码生成
		aigc_label.py	# AIGC 合规标识的唯一入口 (元数据隐式标识 + 产物打标)
		resources/	# 内置字体、BGM 等随包资源
		tts_service/	# 本地语音服务 (Qwen3-TTS, 独立环境, 按需安装)
		tools/	# 端到端脚本、界面冒烟、音量校准
		tests/	# 单元测试
		projects/	# 项目树 (见 2.3)

2.3 项目目录（产物布局）

项目目录不是"一期的产物堆在一个文件夹里", 而是**每类产物自己的子目录**, 全部由 `layout.py` 定义:

```
projects/<项目名>/
├── 素材/           # 原始稿按项目入库落盘, 供排图与逐期取用
├── 地图/           # 期数地图 (map.json) 与逻辑段划分
├── 脚本/           # 各期脚本 json (含逐期对话形式留档)
├── 音色/           # 音色自举的参考音频与画像
├── 封面/           # 封面三尺寸
├── 背景/           # 逐句画面用的背景
├── 音视频/        # 成片 mp4 (横屏 / 竖屏)
├── 字幕/           # SRT / LRC / 整秒 TXT / 洁版 TXT (四份同源)
├── 图文/           # 图文稿
├── 报告/           # 合成阶段的产物门禁报告
└── 过程/           # 中间产物: 脚本门禁报告、锁文件、中间稿
```

"这个项目的全部材料"就是一整个文件夹，拿走它等于拿走全部。项目登记表 `_projects.json` 留在输出目录根上。

三、模块划分

二十一个模块，按"谁在管线里干什么"分四组。**分组的边界不是文件大小，是"改动会牵动谁"**——同一组的模块要么共用一个真值源，要么共用一把尺。

3.1 模块总表

模块	组	一句话职责
<code>web_ui.py</code>	管线编排	HTTP 服务 + 单页界面（项目 / 脚本 / 合成 / 配置四页）
<code>pipeline.py</code>	管线编排	单期管线编排：素材 → 脚本 → 声音 → 字幕 → 画面 → 产物 + 校验报告
<code>project_store.py</code>	管线编排	项目登记：一档节目一个项目，项目下按期推进
<code>layout.py</code>	管线编排	项目目录布局的 唯一定义处
<code>probe.py</code>	内容生产	结构探查：把素材扫成结构化 IR（形式证据 / 语义判定 / 单元凝缩三层）
<code>planner.py</code>	内容生产	期数地图：把成稿排成播出计划，以及往已有计划里插新料
<code>paradigms.py</code>	内容生产	编排范式：素材类型卡 / 风格卡 / 对话形式卡（ 数据层，不是代码分支 ）
<code>script_engine.py</code>	内容生产	脚本引擎：前置模板 + 填空 + 门禁 + 检查 + 回灌重试 + 锁文件
<code>ingest.py</code>	内容生产	素材摄入：粘贴 / 上传 / 书稿锚点切章（纯解析，不调模型）
<code>source_store.py</code>	内容生产	素材库：原稿按项目入库落盘，两级取料

模块	组	一句话职责
<code>tts_engine.py</code>	媒体加工	语音合成：音色注册表 + 逐句合成 + 实测时长回填 + 音色体检（漂移判定与种子偏移定点重出） + 试听
<code>audio_engine.py</code>	媒体加工	音频：编码参数校验 + 拼接 + 降噪 + 响度归一 + 背景音乐
<code>subtitle_engine.py</code>	媒体加工	字幕：断行的 唯一入口 + 四份同源产物（SRT / LRC / 整秒 TXT / 洁版 TXT） + 断词率统计
<code>video_engine.py</code>	媒体加工	画面：背景 + 动画 + 说话人指示 + 字幕烧录 → 横竖屏 mp4
<code>assets_factory.py</code>	媒体加工	资源工厂：背景图、封面三尺寸、BGM—— 全部由代码生成
<code>aigc_label.py</code>	媒体加工	AIGC 合规标识的 唯一入口 （元数据隐式标识 + 产物打标）
<code>config_manager.py</code>	基础设施	配置点位的 唯一定义处 （四张封闭表）
<code>llm_client.py</code>	基础设施	多后端 LLM 客户端（仅 Python 标准库），fail-closed
<code>duration_model.py</code>	基础设施	时长模型：字数 ↔ 时长 ↔ 语速的 唯一换算入口
<code>bins.py</code>	基础设施	ffmpeg / ffprobe 的 唯一定位处 ：定位 / 探测 / 一键安装
<code>__init__.py</code>	基础设施	导出公共符号 + <code>__version__</code> 唯一源

三处“唯一定义处”是这张表的重点：`layout.py` 管路径、`config_manager.py` 管配置与词表、`duration_model.py` 管换算——任何一处要拼路径、读配置或做换算，都从这三个地方取。两处各写一份，就是同一件事有两个真值来源；而这类错误最难查，因为两边看起来都对。

3.2 管线编排组：管“跑哪一步、跑到哪”

这一组不产出内容，只决定什么时候调谁、失败了怎么办、东西落在哪。

web_ui.py (HTTP 服务 + 单页界面) — 界面是深色自包含单页：无外部 CDN、无 emoji、CSS / JS 内联、仅系统字体。所有控件的值域、默认值、枚举档位由后端下发 (统一推动点位)，前端不另持一份档位表。四个页面用 hash 路由；脚本页与合成页切进去默认不重拉数据 (0 请求)，只有项目页与配置页会重拉；全 UI 无轮询，唯一在转的是任务进度。

pipeline.py (单期管线) — 每个阶段都有门禁，门禁不过即写锁文件硬中断 (fail-closed)，**不产出次品**。产物校验覆盖音画时长差、实测码率与采样率、断词率、字体、水印。锁文件带续跑语义：修完问题用 `--continue` 指定树根与期号，从阻断点接着跑；已存在的产物默认跳过。

project_store.py (项目登记) — 一档节目一个项目，项目下按期推进。**哪怕只出一集也立项**——它同样有节目名、有封面、有产物，同样要有个地方记住这些。从前单集走的是“不建项目”的第二条路，产物直接落到输出目录根上，与别的项目混在一起，那条路已删掉。

layout.py (目录布局) — 立项即建目录，此后这个项目的**全部材料**都在它自己名下。项目内十一类产物各归其位 (见 2.3)，`_projects.json` 登记表留在输出目录根上。**这个模块是路径的唯一真值源**，别的模块要拼路径就到这里取。

3.3 内容生产组：管“讲什么、怎么切、怎么写”

这一组是全部语义判断的所在，也是“确定性特权”划得最清的地方——**能枚举的交代码，判不了的才交模型**。

probe.py (结构探查) — 排地图失败的头号原因不是模型不行，是**没人告诉它这批素材该怎么组织**。所以先探结构，再谈排期。分三层：**形式证据** (纯代码，层级归一、编号族归纳、杂项识别、切料可切性——同一份素材跑两次结果必须一样)、**语义判定** (一次分类调用，只回三个字段：这是什么类型的素材、哪一层是切分单位——是分类任务不是生成任务)、**单元凝缩** (每个单元一次调用，把一节压成它的逻辑骨架：说清了什么、反对什么、有哪几条主线以及跟着每条主线的细节区分)。凝缩结果落盘后，**重排一次图不会把这 N 次调用再烧一遍**。

planner.py (期数地图) — 分工是一句话：**代码管三件确定的事，模型只做语义**。分量 (一期装多少原文、产能预检、压比体检) 是算术——交给模型，同一份素材两次能排出 60 期和 5 期，两个都“合乎提示词”；期号与落点是抄录——抄错一条这一期就取不到料；只有**分组、标题、主旨、要点**是语义判断，交模型。目标期数来自项目字段 `planned_episodes` (留空=按素材体量定，填数=按数排；排图落库后由地图回填，排出几期即几期)，素材撑不起目标期数时在凝缩前中止并回报最多可排期数，超出单次上限 (`script.map_max_episodes`) 按上限排并出警告。压比不达标就整体重排 (有限次)，重排复用已有凝缩 (不重烧 N 次调用)，输入带上“哪几期不达标、为什么不达标”。

paradigms.py (编排范式) — 一排地图的通病是提示词里没有方法论，只有一句“按素材自身的推进顺序排期”——那句话对什么素材都成立，也就等于什么都没说。真正决定一张地图好坏的是几件事 (什么算一个完整的表达单元、哪些天然可以合、什么值得单开一期、期与期之间靠什么承接)，而**这几件事**

在方法论专著、小说故事集、采访稿里根本不是同一套。所以它们做成卡：加一种素材类型是加一张卡，不是加一个 `if`。风格卡与对话形式卡同住这里，三处唯一出口（范式解析、形式解析、连句上限）也从这里发。

`script_engine.py`（脚本引擎）— 包里最大的一个模块，也是全项目判据最密的地方，第五章整章讲它。这里只说它的分工口径：**模型只填两个空——台词文本、情绪标签（从受控词表里选）**，其余全部由代码完成（结构、编号、字数统计、时长估算、格式归一、验收）。它还带一条数字纪律：**提示词里的数字是“生成引导量”（模型可操作：字数、句数），门禁表里的数字是“验收判据”（代码执行，模型无权判断自己是否达标）**——时长、偏差、断词率这些模型无法自测的量，一律不出现在提示词里。

`ingest.py`（素材摄入）— 支持 `md / txt / docx`，不支持 PDF。理由一句话：**PDF 是出版格式而不是记录格式**——它的章节结构是排版时投影出来的，要拿回来只能反推，反推就是猜；正文顺序同样是投影出来的。这个模块**纯解析、不调模型**——解析靠的是格式本身的确定性，交给模型等于把确定性换成概率。

`source_store.py`（素材库）— 上传的内容若只活在界面内存里，刷新即失，项目也就无从记住“这一期讲的是哪一段”。所以素材要落盘：`素材/`下一份索引加每份素材一个文件。它对外提供**两级取料**：整期素材（给首写轮）与逐节原文（给按节取料的那几处）。

3.4 媒体加工组：管“稿子怎么变成声音、字幕、画面”

这一组不对内容做任何判断，只负责把已定稿的稿子**确定性地**变成产物。四条共同的纪律：**产物门禁不过就 fail-closed、时长一律实测（不许下游自己估）、参数一律先校验再交给外部程序（不许静默回退）、能算出来的就用代码算（不调模型）**。

`tts_engine.py`（语音合成）— 音色注册表 + 逐句合成 + 实测时长回填 + 音色体检

（`timbre_repair`，唯一入口）+ 试听。逐句而不是整篇，因为合成回来的是实测时长，逐句才有时间轴可言。两条引擎适配（在线朗读引擎与本地语音服务），变速统一保持音高（原做法重采样等于变速且变调，两条路径语义不等价）。还有一个资源处置值得记：**合成前默认先把常驻的语言模型请下显存**——本地语音上显卡要占约 5GB，而驻留的语言模型动辄十几 GB，两者同时在场必然抢显存；而语音合成本来就用不到语言模型。

音色体检是 v1.1.0 起逐步长成的另一条“判与修分离”的环。种子由内容确定性派生（`seed = sha256(音色指纹 + 文本)`）保证可复现，代价是同输入必出同波形——**漂移句重跑不变，必须有人判定、有路修复**：

- **判定（模型之外，纯测量）**：整期拼接之前逐句测六维——基频中位数、谱心、响度、存在感频带（2.4-4.8kHz 占比，必须在降采样前量）、<300Hz 能量占比、基音抖动；基线按角色建立（均值与标准差分角色，不混算），z 归一后走**三判据合验**：四维加权总分、暗淡子分（基频下偏 × 谱心变暗 × 抖动的单边组合）、身份下限（基频深落叠加低频饱满 = 音色身份漂移族），任一命中即点

名。权重与阈值全部来自人耳定标样本（15 个耳标样本 15/15 分离），钉在

`tests/test_timbre_guard.py` 回归。

- **修复（种子偏移，不重摇韵律）**：点名句按 `seed+1 → +2 → +3` 固定偏移序列重出，温度不动（重出的韵律与邻句同分布）；全败改复用同角色最近一条未点名句子的种子（只换抽样点）；仍不过保留历史最优并标记人工审，**不静默出货**。修复后重测时长并回填，字幕与视频时间轴使用修复后波形。候选验收与检测共用同一组判据——重出候选必须三判据全部通过才转正。
- **合成段收口为纯函数全量重建**：删除“音频数 == 句数即整体复用”的分支——改稿不增删行时该判据会静默复用旧音频，成片念的还是旧稿。合成只认当前脚本，`reuse` 旗子仅保留脚本与背景图的续跑语义；体检挂点唯一化到 `synthesize()` 尾部。
- **服务进程挂 Windows Job 对象**（`JOB_OBJECT_LIMIT_KILL_ON_JOB_CLOSE`）：主程序无论怎么死，内核自动收走整棵服务进程树——本地语音服务带着几个 G 显存权重，不能变孤儿。

audio_engine.py（音频）— 编码参数的物理约束校验是本模块的唯一入口。有一个具体事故做依据：原配置 192 kbps 配 22050 Hz，被编码器静默钳制到 160 kbps——本模块把这种组合拦在启动之前，**报错而不改数**。其余职责是拼接、降噪、响度归一、背景音乐（含人声闪避）。

subtitle_engine.py（字幕）— 断行的唯一入口，四级算法：**token 保护**（英文词、数字、连字符词不被切开）→ **标点优先**（优先在句读处断开）→ **禁则校验**（中文避头尾：禁则字不作行首、不作行末）→ **二分回退**（无合法断点时取最近的合法位置，实在无解才硬切并记录）。原项目逐字符硬切，实测 393 处折行里有 42 处断在词中间。断词率必须为 0，否则产物门禁判 FAIL。版式定为**两档**（v0.42.0 从三档而来：单行档改为框内横滚、双行档删除），两档共用同一个框、同一套 Style、同一套字宽口径；框的几何（`frame_of`）是界面实时预览与画面烧录共同的出处。

一期落盘**四份字幕**，全部从同一份 `script / timings` 同一次生成，**不从产物反解**：**SRT**（画面与通用播放器）、**LRC**（音频平台的字幕位只走歌词渲染，没有 SRT 解析通道——把合规的 SRT 交上去播放页就是没有字幕，所以另落一份它认得下的：一行一条、只写起始时间、时间标签两位百分秒，59.999 会进位成 01:00.00）、**整秒 TXT**（`[mm:ss]` 带方括号，给解析器）、**洁版 TXT**（`mm:ss` 摘掉方括号直接接正文，给人的眼睛与外部工具）。方括号是给解析器的定界符，给读它的人就是噪声——摘掉后时间戳仍是定宽一列。说话人名判据各份同源（`speaker_name_shown`），不另立开关。

video_engine.py（画面）— 背景 + 动画 + 说话人指示 + 字幕烧录 → 横屏 / 竖屏 mp4。硬约束是**时长对齐**：视频流时长显式等于音频时长，**不用猜**——原做法用无限静图源配“取最短”参数，音频结束后又排空了 76 帧。

assets_factory.py（资源工厂）— 背景图、封面三尺寸、背景音乐，**全部由代码生成，不调用模型**。档位一律取自有限枚举表。字体先探测、再使用；探测不到即报错，**不静默回退**到渲染不出中文的默认字体。

aigc_label.py（AIGC 合规标识）— **唯一入口**，依据《人工智能生成合成内容标识办法》与配套强制性国家标准，做两件事：元数据隐式标识 + 产物打标。音频在片头最前加一句语音声明，**字幕时间轴随**

声明时长整体后移，不然声画从第一句就错位。

3.5 基础设施组：管"真值住在哪"

bins.py（外部二进制定位）— ffmpeg/ffprobe 的**唯一定位处**：`locate()` 按 ① **PATH** → ② **项目 bin/** → ③ **None** 返回绝对路径，各模块整个传给 subprocess ⇒ 不依赖 PATH 解析 ⇒ 一键安装落进 bin/ 后当场可用；`state()` 与 `locate()` 共用同一顺序 ⇒ **探测报的 = 实际跑的**；`install()` 下载 → sha256 校验 → 解包落 bin/，校验不符一律抛错丢弃临时件——**绝不把来路不明的二进制放进 bin/**（fail-closed）。它是 v0.47.0 的根因修复：此前定位散在四个模块各写一遍、且 `shutil.which` 只搜 PATH，安装等于没装。

config_manager.py（配置总表）— 全项目唯一的推动点位，四张封闭表：**参数表**（全部可配置点位：类型 / 值域 / 默认 / 归属组）、**模式表**（枚举型点位的档位定义）、**风格表**（风格倾向的三维取值）、**门禁表**（判据 / 阈值 / 级别 / 阶段）。界面控件的取值范围与默认值、后端校验规则、流水线执行参数全部由这四张表派生——**禁止在别的模块另写阈值或值域**，分散点位的枚举成本是乘积级的。语篇词表也住在这里，它同时是 JSON Schema 的枚举、门禁的判据、界面下拉的选项，三处必须一字不差。

llm_client.py（LLM 客户端）— 只用 Python 标准库实现，不依赖任何厂商 SDK，兼容多家后端与任意 OpenAI 兼容端点。它要接住的是**后端行为差异**而不是某一家：思考段与答案混在一段时要剥成对的思考段（**没闭合的不动**——分不清哪段是思考，硬剥只会砍掉答案）；从回复里取 JSON **不用"第一个花括号到最后一个花括号"**（思考里出现过花括号就会截出半截思考加答案），改为取**最后一个解析成功的完整对象**；输入额度由使用者定，程序不探后端窗口多大。**fail-closed**：任何网络或协议异常都抛出错误，绝不返回一份空稿让人以为写成功了。

duration_model.py（时长模型）— 字数 ↔ 时长 ↔ 语速的**唯一换算入口**，任何模块都不得自行写换算系数。原项目把"每秒 4 个汉字"写进提示词，实测系统性偏高 18–27%——**系数写进提示词就等于把它交给了幻觉**，所以它只能住在一个地方。

__init__.py — 导出公共符号与版本号唯一源。版本号同时被断言检查，改了这里不改落地处会当场报错。

四、循环体系：大环套小环

这是本篇的主轴。这套系统的执行结构，本质上是一个五层的环套环——但要先把一句话钉死，否则后面全都会看错：

三段不是循环，是串行。出货 → 检查 → 门禁 是一条直线，走一次就往前走；循环在每一段内部。把三段说成"三个循环"是把串行误读成迭代，把"段内成环"误读成"段间轮转"——这两件事在代码里是两套不同的控制结构，混起来就再也说不清"第几轮"到底是谁的第几轮。

4.1 五层循环总览

【L0 批量循环】 for i, no in enumerate(eps, 1) ← 勾了几期跑几期（串行）

└─ 【L1 单期管线】_run_episode: 脚本 → 合成 → 出片 → 报告（一次直线）

└─ 【L2 脚本阶段】三段串行（非循环）

└─ 出货段 ── 环：整篇重试 / 分段两环

└─ 【L3 排图环】逻辑拆分环（首写+1 次） → 装箱 → 规划环（首写+2 次）

└─ 【L4 段队列环】while queue ── 逐写作段

└─ 【L5 段内轮次环】首写 1 次 + 修改 ≤5 轮

修改轮 = 补字 → 查重 → 替换 → 计算

└─ 【判修内环】补丁重发（同批补丁下一轮重发）

└─ 检查段 ── 环：判（模型）1 次 + 定点修 2 次

└─ 门禁段 ── 环：判（代码 9 项）1 次 + 定点修 3 次

└─ 【L2' 合成阶段】逐句三趟：语音 → 字幕 → 画面（每句一次）

层	环	转什么	轮次点位（配置值 → 实际次数）	谁判	出口
L0	批量	逐期	勾选的期数	人（失败跳过，同因连败到阈值停）	全部跑完
L1	—	单期五步（直线，非环）	—	—	出片或报错
L2	出货	写一份可用稿	segment_parse_rounds（2 → 含首写共 3 次）	不判	产物不可用就重发；用尽报错
L3	排图（分段路）	逻辑拆分 → 装箱 → 段主旨	LOGIC_SPLIT_RETRIES（1 → 2 次） / plan_rounds（2 → 3 次）	程序校验条数	拼出段清单

层	环	转什么	轮次点位（配置值 → 实际次数）	谁判	出口
L4	段队列	逐写作段	段数（不定，可回插）	—	队列空
L5	段内轮次	补字 / 查重 / 替换 / 计算	<code>segment_fix_rounds</code> （5 → 首写 1 次 + 修改 ≤5 轮）	程序核字数	进容差，或轮次用尽
—	判修内环	判 → 定点修	与所在段同轮（状态机，不是独立循环）	代码（形式） / 模型（内容）	过了，或修满
L2	检查	内容检 + 定点修	<code>check_rounds</code> （3 → 判 1 + 修 2）	模型	过了往下，修满也往下
L2	门禁	形式门禁 + 定点修	<code>gate_rounds</code> （3 → 判 1 + 修 3）	代码	过了定稿，修满也定稿
L2	逐句	语音 / 字幕 / 画面	句数	程序 + 产物门禁	三趟跑完

配置里一共只有五个轮次数（`gate_rounds` / `check_rounds` / `segment_fix_rounds` / `segment_parse_rounds` / `plan_rounds`），加三个模块内常量（逻辑拆分重试 1 次、排图整体重排 2 次、凝缩重试 2 次）。**没有一个环是"改到好为止"**——上限全部写死在配置或常量里，这一点 4.11 还会专门讲。

注意表里那个容易看错的地方：**配置值与实际次数不是一回事**。检查段的 `for attempt in range(check_rounds)`（3）与门禁段的 `for attempt in range(gate_rounds + 1)`（4）写法不同——两段配置值同是 3，实际迭代次数是 3 与 4，即**首判之外，检查段能修 2 次、门禁段能修 3 次**。不要从"配置都是 3"推出"两段轮次一样"。

4.2 L0：批量循环 — 逐期串行

最外层是 `api_batch` 起的一个后台任务：

```
for i, no in enumerate(eps, 1): ← 勾了几期跑几期
```

```
    if job.stop: 中止（记下"做完 N 期，剩 M 期没做"）
```

```
    这一期失败 → 记一笔、跳过、继续跑后面的
```

```
    同一原因连续失败到阈值 → 停下来交人
```

三条纪律写在这里，都不在代码之外：

- 1. **串行，不并行。** 语音合成与画面渲染吃满机器，两个一起跑只会互相拖慢，还可能爆显存。批量的价值在"不用守着"，不在"同时开工"。
- 2. **单期失败不拖垮整批**，但同一原因连败到阈值就停——继续跑下去只是把同一个错误重演 N 遍。
- 3. **中止是"做完手上的"**，不是"立刻断"：正在跑的那一次调用掐不断（本地模型一个请求就是一次不可分割的生成），下一期不开就是了。

批量之外，界面也可以单期触发——那时 L0 退化成一次调用。**这一层是"期"的节拍，不是生成逻辑的一部分。**

4.3 L1：单期管线 — 两个阶段，各管各的账

一期跑五步，但脚本与合成是彻底分开的两个阶段：

	脚本阶段	合成阶段
干什么	写稿、判稿、改稿	把稿子变成声音、字幕、画面
判谁	内容（模型判）+ 形式（代码判）	产物本身（音频有没有、画面全不全、时长对不对）
失败怎么办	在环里修；修不满带着问题落盘	fail-closed ：不过就写锁文件、不出片
边界	—	不写稿、不判稿

合成的第一件事是**取稿**，取不到直接报错，绝不顺手现写一份——"现场生成"等于把审过的稿换成模型新写的另一份，而合成端根本没有判它的资格。脚本阶段留下的未通过项在这里只提醒、不拦人：同一件事两个阶段都管，人连出片的机会都没有。

这条分工有一个具体的目的：**两个阶段各自的账各自结，报告不互相污染**。脚本阶段的结论落在 `过程/<期号>/gate.generate.json`，合成阶段的结论落在 `报告/<期号>.report.json`。两者之间只有一次提醒：点合成时读那份脚本结论，有未通过项就弹框逐条列出，人点「仍然合成」才继续，并留一个凭证。

4.4 L2：脚本阶段 — 三段串行，各成各的环

这是最容易看错的一层。 三段是**串行的**，每段内部才是**循环**：

出货段（写正文，只求一份可用稿；产物不可用在本段重试）

↓ 过了这一段，手上有一份"可用稿"

检查段（内容检：承诺链检必判、语义检按开关）

↓ 判 + 定点修，修到过或修满 check_rounds 都往下走（不早退）

门禁段（形式门禁：代码判九项）

↓ 判 + 定点修，修到过或修满 gate_rounds 都落盘

定稿（粘片头尾 → 落盘 → 返回）

三段各自成环、各用各的预算，互不侵占对方的轮次：

段	判的人	判什么	轮次点位	出口
出货段	—	什么都不判，只求一份可用稿	segment_pars e_rounds（含首写共3次）	产物不可用就重发；用尽报错停下
检查段	模型	承诺链检（ 必检 ）∪ 语义检（ 开关、默认关 ）	check_rounds （判1次 + 定点修2次）	过了转门禁；修满 也 转门禁
门禁段	代码	JSON 合法、字段完整、语篇词表、连说上限、句长、单句时长、措辞禁忌、可朗读、末尾标点（ 九项硬项 ）+ 总时长偏差（ 软项 ）	gate_rounds （判1次 + 定点修3次）	过了定稿落盘；修满落盘退出

为什么门禁排最后？ 不是排法偏好，是结论的时效问题：它每轮**重新判形式**，再把这之前内容检的结论并回同一份报告。反过来的排法（门禁在前）里，中间检查段的补丁会插入新句（加行，可能顶破连说上限），改完没人复核形式——报告上留着旧稿的结论，「报告绿、稿子坏」的静默放行就是这么来的。

为什么检查段修满也往下走？ 因为它排在门禁前面，形式还没判——内容没过不等于稿子不能用。**不早退**是这一段的设计意图：把"内容有疑虑但形式需要复核"和"稿子根本不能用"分成两件事。

三段内部零重写。 对一份可用稿，它们只有「判」与「定点修」两种动作。重写只属于出货段——那是"写"的事，不是"判"和"改"的事。这条纪律的现实理由：整篇重出会把没毛病的两百句重新摇一次骰

子，改好的地方又带进新毛病，轮次全耗在打地鼠上。

4.5 L3：出货段的两条路 — 整篇重试，或分段两环

出货段按路线分岔。路线由项目模式定死（见 1.2），不是开关：

整篇路（逐期即兴 / 单集，或成稿规划退回来）只有一个环：

```
for k in range(parse_rounds + 1): ← 最多 3 次
```

调模型整篇写 → 解析 JSON → 拆句

产物不可用（拆不开 / 一句可用内容都没有） → 重发

可用 → 出这一段

分段路（成稿规划）有两组环，先后串行：

【第一组：排图环】

逻辑拆分 `split_by_logic` ← 模型按内容把节分组成段，只回节序号

└─ 环: for attempt in range(`LOGIC_SPLIT_RETRIES` + 1) (2 次)

两次都答不出合法分组 → 退回"一节一段"

装箱 `pack_segments` ← 纯算术，按输入额度逐段过桶

规划轮 `_segments_plan` ← 模型给每段写一句段主旨 + 全篇起标题

└─ 环: for attempt in range(`plan_rounds` + 1) (3 次)

段数钉死 (`schema minItems == maxItems == 段数`)，反复不过就按各段首节的凝缩主旨代填

【第二组：段队列环】 见 4.6

排图环里三个环的判据各不一样，这是刻意的：

环	判什么	不合格怎么办
逻辑 拆分	只判 节序号 合不合法（越界、重复、漏节）	归一化：越界与重复丢掉、组内升序、 漏掉的节补成独立一段 ；两次不行退回一节一段
装箱	装不装得下（算术）	在该逻辑段 内部 按整节切开；单个节自己就超容 → 报错 ，绝不从节中间切
规划 轮	只判 条数 == 段数	代填；段边界与配额一个字不动

三道判据里没有一道判"分组好不好"——**分组只看语义，摆着字数会把模型往"按字数摊匀"上带**。递进去的【节清单】也只给「节号 + 标题 + 凝缩」，一个体量数字都不给。

装箱那一步有一条硬规矩要单独说：**切分单位只有整节凝缩**。逻辑段装不下就在它内部按整节切；单个节自己就超桶容量时报错（要求调输入额度倍率或回地图拆期），**绝不再从节中间切开**——半个节取不出原文（取料一次只认一个落点），也算不出账。

4.6 L4：段队列循环 — 最反直觉的一处

拼出段清单之后，真正的写作在段队列里跑：

queue = [[段号, 段对象, 该段覆盖的节号, 段配额], ...]

while queue: ← L4 段队列环

i, g, pieces, quota0 = queue.pop(0) ← 从头取一段

动态配额：前面各段的差额按比例摊进本段，总账始终咬住目标

段内轮次（4.7）

while True: 写 → 补 → 查 → 换 → 账

写完 → script 追加 → 进下一段

最反直觉的一点：这个队列会被"回插"。段内首写之前有一道检查——本段原文按 token 折算后超了本次调用的可用额度，就**按整节把这一段切成几批**，把这几批 `queue.insert(n, [i, g, one, q])` 插入队列**当前段的位置**，然后 `break` 出去。

段 3 的原文装不下 → 切成 3a / 3b 两批

↓ `queue.insert(0, [3a]) / queue.insert(1, [3b])` ← 插回队首

↓ `break` 出这一轮的段内循环

↓ `while queue` 再取一次，取到的就是 3a

于是同一段可以变成几段重新排队，段号还是同一个 `i`（日志里表现为"段 3：本段原文折合 N token 超可用 M，按整节分成 2 批分次喂完"）。这不是"循环里套了个循环"，是环自己把工作量拆开、重新入队——用一个队列把"递归"压成了平面循环，好处是中止、进度、落盘三件事都还在这一个环里管得住。

分批的配额不是平均分，是按有效字占比摊（ $\text{每批配额} = \text{本段配额} \times \text{本批料的字} \div \text{本段料的字}$ ），下界取配额地板——token 只用于判"要不要分批、按哪切"这一件事，不进配额。同一件事里两把尺混用，就是"这一节被派了它写不出的字数"的经典来源。

4.7 L5：段内轮次 — 写作一次，之后是"补字 → 查重 → 替换 → 计算"

段内那个 `while True` 是整个系统里最内、也最讲究的一个环。它的形状是（首写 1 次 + 修改最多 5 轮，共 ≤ 6 次迭代；上限由 `segment_fix_rounds` 管，计数只数修改轮）：

【第 1 轮】写作 + 计算

整段写（只此一次，之后不再重写整段）

算有效字 → 与配额比 → 记下最接近的那一版 → 判是否进容差

【第 2~6 轮】补字 → 查重 → 替换 → 计算

按上一轮末的差额决定：少了补字（`apply_insert`）/ 多了压字（`apply_trim`）

↓ 落地成功

查重 `find_repeats`（判重范围：整篇；动刀范围：本段）

↓ 有重复

替换 `apply_replace`（行数不变，同长度换内容）

↓

计算：算有效字、记 best、判容差 → 进容差就退出；轮次用尽也退出

三点设计意图，每一点都能追到一个具体的坏现象：

1. **"计算"放在轮末。** 它是这一轮的收口，差额供下一轮开头补字用。从前计算在补字之前，等于拿着上一轮的账去决定这一轮补多少，差了一拍。
2. **补字之后立刻查重、就地替换。** 补字是复读的源头——程序只能说"还差 800 字"，模型最省力的补法就是**换个说法把说过的话重说一遍**：不踩"不许编造"、字数还随便控。所以查重紧跟补字，发现重复**就地换成新内容而不是删**。
3. **替换"按句点名、按字数落地"。** 换掉的句子必须落在 [原句字数，单句上限] 之间。为什么不能换短？换短了 → 轮末核账又报缺口 → 下一轮又补 → 又可能复读，绕回老路。**"同长度换内容"是这条路的稳定性条件，不是审美。**

替换提示词必须看得到全篇已写脚本，这一条值得单独说。问题虽然发生在本段、原文素材也只给本段，但模型要判断"这句跟别处撞没撞"，就必须看得到所有已经写过的句子——「【本段原文】(素材)」与「【已写脚本】(全篇正文，逐句带编号)」分成两块摆，再给「【要换掉的那句】」。同理，**补字轮也带上全篇已写正文**：首写轮本来就拿着全篇已写正文，补字轮原先只有本段那三块，提示词却要它"到素材里挑正文还没讲到的点"——它看不见前几段讲了什么，必然撞上去。

段内轮次的落地校验有一个刻意的缺口：**两条路都不判句长**。句长是**全篇一把尺**的量，归门禁判、两个方向都有处方；写作阶段替它把关，代价是把「一条坏句」放大成「整段停摆」——一条 7 字句让 40+ 条新增整批作废，那一段的缺口一轮都没再试。

4.8 判修内环：判与修之间还夹着一个"补丁重发"状态机

检查段与门禁段的每一轮里，动作都是「判 → 定点修」。但"修"这一步失败之后**不重写、也不重摇**，而是**下一轮重发同一批补丁**：

第 N 轮：判 → 得到 42 个落点 → 一次性发一批补丁

└─ 落地成功 41 条、拒 1 条

拒收原因记下 → 下一轮把「上一轮为什么没落地」连同**同一批补丁**重发

└─ 全部落地 → 下一轮重新判（形式每轮全套重判）

这一段有三个必须讲清的规定动作：

- **补丁逐条落地，不是全有或全无。** 单条不合格（越界、没点名、空文本、并句校验不过、两条互相冲突）**只拒该条**，其余照落；被拒的原因随下一轮提示词递回去。「重发同一批」因此是有依据的重试，不是重摇骰子。从前一条坏就整份作废，实测一轮十几分钟白烧。**结构性错误**（条目不是对

象、句号不是整数)才整份退——那种输出从根上不能用。

- **能定点的判据是"问题带不带句号"**。形式门禁全都自带句号(它本来就是逐句判的);内容检要模型自己给。带句号的定点改;结构坏了只能整篇重出;**指不出句号的记一笔「未修好」,不动稿子、交人工**——让模型为一句它自己都说不清的问题重写两百句,只是重新摇一次骰子。
- **贵的东西按需跑,便宜的东西每轮全跑**。形式门禁走代码、毫秒级,每轮全套重跑(改一句话的字数会牵动单句时长与总时长,改一个用词可能撞上新的禁用词);内容检走模型、要读整篇,只重判上一轮没过或没判过的那几项,已通过的结论从上一轮带过来。

4.9 上游的两个环:凝缩与排图

脚本阶段之外,还有两个环在**项目级**跑(一次性,不是每期):

【凝缩环】 `probe.condense_units`: 逐单元(一次调用只看一节)

```
for n, (sid, s) in enumerate(units, 1): ← 单元数 = 节数
```

```
for _ in range(attempts): ← 内环:重试 2 次
```

```
llm.chat(... 凝缩这一节 ...)
```

已有凝缩结果直接跳过(重排一次图不该把 N 次调用再烧一遍)

【排图环】 `planner`: 分组排期 → 压比体检

```
while retry and retried < MAP_RETRIES: ← 最多 2 次整体重排
```

```
把不达标的期连同原因交回模型整体重排
```

两层环的判据一个是"能不能解析出结构"(凝缩),一个是"每期压比在不在区间内"(排图)。**凝缩那一步失败是致命的,不做降级**:凝缩结果正是排图分组的依据,拿不到它还要往下走,排出来的图看着完整、实际是按字数硬切出来的——那才是假功能。

4.10 L2': 合成阶段 — 第三类循环,逐句三趟

脚本是一条"环里改句子"的路;合成是一条**"逐句重复同一个动作"**的路。同一份稿子被逐句走过三趟:

第 1 趟 语音: for 每一句 → 合成音频 → 探测时长 → 拼时间轴

第 2 趟 字幕: for 每一句 → 按时间轴切行(确定性换行,最多 N 行)

同一份时间轴同时落 SRT/ASS 与 LRC(画面烧录用前者,平台歌词位用后者)

第 3 趟 画面：for 每一句 → 画说话方高亮 + 字幕 + 画面元素



混流 → 产物门禁（音频在不在、时长对不对、画面全不全） → 报告

这一类环和前面所有环都不同：它没有**"轮次"这个概念**——句数就是次数，不判、不修、不重试（失败即报错）。它之所以要单独列出来，是因为它让**"句"这个单位贯穿全系统三处**：判据按句指认（门禁报"第 N 句"）、生成按句落地（替换按句点名）、合成按句执行。三处口径一致，编号才不会漂。

另外，界面上还有一个客户端侧的环：**任务进度轮询**（1.2 秒一次，读 `/api/task/`）。它是整个系统里唯一**"一直在转"**的环，但也只在有任务时转；页面切到脚本页或合成页默认不重拉数据。

这个环有一个只有它才会踩的坑，值得单独记一笔：**"还在跑就歇一会儿再读"这一句必须真的等着**。看板的形状是"读一次任务台账 → 还在跑就歇会儿再读"，从前歇那一下是"起个定时器就走"——`await` 它的调用方当场往下执行，于是**按钮在任务刚起步的一刻就恢复了**，看着像已经收工；人再点一下，第二个任务就起来了。修法是把递归链串成一条真等到收工的线：跑着就摁不动，收工才恢复，中止按钮的显隐跟着走。判别口径也随之定下来：**看有没有调用方 `await` 它**——有，就必须串成一条 Promise；只有回调式的那个看板例外。

同一处还挂着一条属于批任务的账：**"进度只增不减"是"单期"的规矩**（它服务的是已跑完的格数，本就不该倒回），而批任务是**串行多期共用一个进度口**——第二期一开局就顶着上一期跑完时的高位，整期看着不动、反倒像卡死。所以每期开头把**本期**进度清零，界面按 $(\text{第几期}-1+\text{本期进度})/\text{总期数}$ 折算成整批进度：公式是界面从前就在用的那一个，缺的只是后端这一步。

4.11 循环的公共纪律

五层环之上还有六条纪律，一条都不许破：

纪律	具体规定	为什么
有界	每个环的轮次上限来自配置或模块常量，没有"改到好为止"	无限重试 = 无限烧钱 + 无限延迟，且模型不会因为多跑几轮就变得听话
每轮落盘	一轮定稿就往出片时读的位置写一份（落"当前最新那版"，与最终返回的一致）	生成一轮十几分钟，等全程跑完才第一次写盘，中途卡住／被中止就什么都没有
中止只在轮间看	放弃指令只在轮次之间读——正在跑的那一次调用掐不断，但下一轮不再开	掐不断不等于要装作没收到：一轮写完带着已有稿子收工，比让人等它把剩下的几轮全跑完再丢弃强

纪律	具体规定	为什么
零重写	对一份可用稿，只有"判"与"定点修"；重写只属于出货段	整篇重出会把改好的地方重新摇骰子
落点不许为 0	契约把句号收紧为「 ≥ 1 且不超总句数」；给不出就程序拿 <code>quote</code> 反查（去空白比对子串，不花模型调用），反查不到记「未修好」	填了假句号，改稿的人会去改一句没毛病的台词，真正的问题原封不动
进度分段折算	预算表五段首尾相接（准备 0~10%、写稿 10~72%、检查 72~86%、门禁 86~96%、收尾 96~99%）， 只增不减	不然进度条会倒着走；写稿段最耗时却一度没有刻度，实测轨迹只有四跳、全挤在收尾几十秒（v0.45.0 修复）

五、核心机制

前面讲的是"环怎么转"，这一章讲"环里判什么、凭什么判"。挑六个决定性的。

5.1 单一出口：六类设定的唯一来源

系统里六类会变的设定，每一类都只有一个出口函数，**提示词、门禁、界面全部从它取**：

设定	唯一出口	谁在用它
素材类型（范式卡）	<code>paradigms.resolve_paradigm</code>	提示词、写脚本、凝缩
风格倾向	<code>_style_block(preset)</code>	四条写作路
对话形式	<code>paradigms.resolve_form</code>	提示词、门禁、界面
连说上限	<code>paradigms.run_caps</code>	提示词、门禁 <code>ab_run_limit</code>
语篇词表	<code>config_manager.DISCOURSE_OR DER</code>	提示词、schema 枚举、门禁、界面下拉
角色与节奏	<code>_hosts_text</code> / <code>rhythm_text</code>	四条写作路

连说上限这一行有一个现成的例证（v0.38.0）：捧眼形式（`anchor`）的 A 连说上限由 1 放宽到 2，**改的只是卡上的一个数**——提示词四处（整篇【文体依据】、分段生成要求第 2 条、补字铁律、替换铁律）、形式门禁 `ab_run_limit` 的判据、界面下拉里挂的上限，**全部自动跟着变，没有第二处要改**。放宽的是

许可（门禁阈值），不是**形状**（常态节奏）：形状示范与节奏句一个字没动，示范里 A 仍是一句一插（ $1 \leq 2$ ，依然合规）。这件事的反面同样值得记：正因为上限只有一个出口，"改了这个数会影响谁"是可以一次说清的——**答不出这个问题的出口，就是没做到唯一**。

为什么这件事值得单列一节？ 因为"两份表"的代价不是维护麻烦，是**判据打架**：提示词按 A 表写、门禁按 B 表判，稿子就会陷在"改了还是不过"里，而人从日志上看不出是两把尺。语篇词表那一份尤其典型——它同时是 schema 的枚举（约束解码，模型填不出表外的词）、门禁的判据、界面下拉的选项，三处必须一字不差，所以它只能有一份。

对话形式还多一条纪律：**逐期留档**（脚本/`<期号>.form.json`）。判稿读本期自己的档，配置值只管"下一次生成用哪种"——否则改一次全局配置，历史各期的判据会跟着一起变。

5.2 字数核账：一把尺、一次摊派、一条地板

脚本的字数账是全系统最容易被做错的地方，因为"字数"这个词下面藏着三把不同的尺：

量	单位	归谁
原文有效字 (chars)	有效字（汉字 1 / 标点 0.5 / 西文词 1.5）	配额权重 ——这一节能写出多少稿
原始字符数	字符	装箱折 token ——这一节占多少输入
摘要自己的字数	字符	已无消费者

三把尺混用的后果是同一件事：**那一节被派了它写不出的字数，只能把自己刚写过的话换个说法再讲一遍**。所以配额只能按**有效字**摊（段配额 = 段内各节配额之和，段 token = 段内各节 token 之和，段原文 = 段内各节原文按序拼——全是加法，没有除法），并且全篇同源：成稿目标出自 `chars_for_target`（时长 × 速度 × 语速常数，常数的单位就是**有效字/秒**），地图压比也是拿期的有效字比成稿目标，三处同一把尺，摊出来的配额才和压比可比。

摊完之后是三道量：

段容差 = $\max(\text{配额} \times 15\%, \text{配额地板} \times 15\%)$

配额地板 = 软句数下限 × 每句最少字 ($15 \times 8 = 120$)

软句数 = $\text{配额} \div \text{期望句长}$

← 与总时长门禁同一把尺

地板只管一件事：提示词里那句"约 N 句"不许与配额矛盾——配额低于"N 句 × 每句最少字"时，提示词就一边要模型写 15 句、一边只给它不足 120 字的额度。它**不是**碎段合并那把尺（那一把乘**期望句长**，

判"这一段的料够不够单开一段", 是产能判据) ——两把尺差三倍, 值不同才对。

差额按比例滚入后面各段的配额, 误差逐段吸收; **各段容差之和恰是全篇容差** (地板派生量的性质决定的), 所以"每段各自合法 $\Rightarrow$ 全篇必在容差内"这句话是可证的, 不是感觉。

5.3 前置规范: 约束解码 + 落点不许 0 + quote 反查

能写进 schema 的约束, 不靠提示词劝。脚本生成直接给 JSON Schema: 语篇词表的档位落在 `enum` 上 (填不出表外的值)、承诺检的落点 `line` 落在 `minimum: 1 / maximum: 总句数` 上 (约束解码实测吃硬约束)、补丁的 `text` 落在 `minLength` 上。只加下限不加上限——实测后端对 `maxLength` 是硬截断, 一个残句没有下一道工序替它兜。

约束解码管得住"枚举", 管不住"模型指不出具体哪一句"。所以落点还有一条反查通道:

模型给出 {line, quote, problem}

- └ line 有效 (≥ 1 且 $\leq$ 总句数) $\rightarrow$ 就用它
- └ line 给不出/为 0 $\rightarrow$ 拿 quote 去稿子里反查 (去空白后比对子串, 不花模型调用)
- └ 查到 $\rightarrow$ 用查到的句号
- └ 查不到 $\rightarrow$ 记一笔「未修好」, 不早退、不重写、其余项照修

"模型没给结论"不等于"检查通过"。模型漏返回某一项时, 那一项记为「未判定」落进待复核——把"没检查"记成"检查通过", 报告上的绿灯比不检更坏。

5.4 并句可行域: 把"数学无解"的判据换成"一定满足"的区间

连说上限被顶破时 (同一个人连着说了太多句), 修法是把那几句**并成一句** (`absorb`, 并句是唯一的减行方式——补丁 schema 里没有 `speaker`, 换人不许)。这条路的判据曾经有个硬伤, 值得作为一条架构教训记下来:

旧判据: 并后字数 $\geq$ 原句合计 $\times 70\%$ 。提示词: 每句 8~40 字 (单句上限 40)。矛盾: 原句合计 > 57 字时, $0.7 \times \text{was} > 40$, 两条要求同时满足在数学上不可能。而且"70%"这条规矩一个字都没进提示词——模型不知道有这条判据, 只会按"合并=精简"写, 必然过短。日志里并句因此一次没成功过。

现在的判据是一个恒非空的可行域, 并且判据与处方调同一个函数 (`absorb_span`):

```
hi = 单句上限 (40)
lo = round(hi  $\times$  0.7) = 28
```

若 原句合计 < lo: lo, hi = round(合计 × 0.7), 合计
lo = max(lo, 单句下限)
hi = max(hi, lo)
判据: 并后字数 ∈ [lo, hi], 否则拒, 并把区间报出来

← 退成"相对原句"的区间
← 不低于单句下限
← 保证非空

两条性质是设计目标，不是巧合：**区间恒非空，上限必然 ≤ 单句上限**（并出来的句子不会再被句长门禁打回，否则就是"修了 A 撞破 B"的死循环）。处方里直接写给模型「每句 28~40 字」这样的**具体数字**——一个判据如果只有判的人知道，它就是一条只会拦人、不会教人的规矩。

这里还有一条容易混的阶段纪律：**并句只做合并，保真判据只有 70% 这一条**；并出来的句子偏长，由下一轮的句长门禁报出、走普通定点改压缩。两条完全不相干的事不许写进同一段提示词——一个提示词只管它自己那一步的事。

5.5 粘合与程序标签：确定性特权

片头、前期回顾、片尾这三段文字由**程序逐字拼**，不发给模型：

- **粘的位置是"整期定稿那一刻"，且在"全稿格式收束之后"**。顺序错了会出具体事故：粘完再跑一遍格式收束，语篇词表会把片头标签"洗"成中性档，成片里片头那行标签就不是"开场"了。
- **粘完只补时长**，不再跑任何判据；**不进轮、不进门禁、不核字数**——它们不是模型写的，拿模型的标准去判它们没有意义。
- **片头尾的标签不在这份语篇词表里**（它们是程序专有词），所以模板只从程序专有词里取词、不发给模型、不进界面下拉；只有脚本页"重判"那一次放行，生成侧一律不传。

同理，排版（流式布局，位置由上一个元素的实测包围盒推出）、字幕换行（确定性算法，最多 N 行）、画面元素（同一张表取件）都是**代码说了算**。这一类"可枚举的事"交给模型，是自找不稳定。

5.6 两级门禁

"门禁"在这个系统里指两个不同粒度的东西，别混：

	脚本门禁	产物门禁
时机	脚本生成阶段（写完就判）	合成阶段（产物出来才判）
判什么	九项形式项（硬）+ 总时长偏差（软）	音频在不在、时长对不对、画面全不全
不过怎么办	定点修 ，修到过或修满轮次；修满仍不过 → 落盘交人	fail-closed ：写锁文件、不出片

	脚本门禁	产物门禁
结论落点	过程/<期号>/gate.generate.json	报告/<期号>.report.json

脚本门禁排在**生成阶段**而不是产物之后，理由只有一条：**检出来还改得动**。等音频视频都渲染完再检，片子已经定死，检出来也没意义。

产物门禁是 fail-closed 的：不过就写锁文件、不产出次品；同时把被拒的脚本与逐项明细一并落盘，避免只留一句"门禁未通过"而无从排查。锁文件还带续跑语义：修完问题后用 `--continue` 指定树根与期号，从阻断点接着跑。

六、配置体系

6.1 配置文件

`config.json` 是**平铺点号键**的一层结构（键名形如 `script.gate_rounds`、`gate.max_chars`、`tts.voice_a`），不是嵌套段。点位覆盖脚本、门禁、语音（含音色体检四键）、音频、背景音乐、画面、输出等分组，具体数目以配置页与 `config_manager.py` 为准。

全项目只有一个配置注册表（`config_manager.py`）：每个点位在这里定义类型、默认值、分组、界面形态与取值范围。管线的 `cfg.get(...)`、界面的控件渲染、校验与警告，全部读同一份定义。

6.2 轮次与容差点位

点位	默认	管什么
<code>script.gate_rounds</code>	3	门禁段：判 1 次 + 定点修 3 次
<code>script.check_rounds</code>	3	检查段：判 1 次 + 定点修 2 次
<code>script.segment_fix_rounds</code>	5	段内修改轮上限（首写另计 1 次）
<code>script.segment_parse_rounds</code>	2	出货段产物不可用时的重发上限
<code>script.plan_rounds</code>	2	规划轮（段主旨 + 标题）重试上限
<code>script.map_max_episodes</code>	60	单次排图期数上限（超出按上限排并出警告）

点位	默认	管什么
<code>script.segment_tol_frac</code>	15	段容差比例（与总时长门禁同一把尺）
<code>gate.max_chars</code> / <code>gate.min_chars</code>	40 / 8	单句字数上下限；并句可行域由它推出
<code>gate.max_deviation_pct</code> / <code>min_deviation_seconds</code>	15 / 20	总时长偏差软门禁
<code>script.dialogue_form</code>	—	下一次生成用的对话形式（判稿读逐期留档）
<code>script.check_semantic</code>	关	语义检开关（承诺链检无开关，必检）

6.3 界面与配置同源

配置页不是手写的表单，而是**从配置注册表渲染出来的**：每个点位声明自己在哪个功能区、用哪种控件形态（滑杆 / 开关 / 下拉 / 输入框 / 路径），界面按功能区渲染，行内控件按"滑杆 → 开关 → 下拉 → 输入框"的固定桶序排列，一面一格、一类控件一行。

界面冒烟测试对账的就是这条同源关系：**配置页控件数 == 配置点位总数、每个控件都在功能区登记表里（无漏登记、无幽灵登记）、拖动滑块后服务端配置真的变了（回读确认）**。单元测试跑在 Python 里看不见界面，而最贵的一类界面缺陷——控件找不到自己、下拉只剩英文裸值——只在真浏览器里才现形，所以这一层必须用真浏览器跑。

七、外部接口

7.1 LLM 后端

`llm_client.py` 只用标准库 `urllib` 实现，不依赖任何厂商 SDK，兼容 LM Studio / Ollama / 自定义 OpenAI 兼容端点。它要接住的是**后端行为差异**，而不是某一家：

- **思考段与答案混在一段**：剥掉成对的思考段（`reasoning_content` 单独字段的也认），**没闭合的不动**——分不清哪段是思考，硬剥只会砍掉答案。
- **JSON 提取不用"第一个 { 到最后一个 }"**：思考里出现过花括号就会截出半截思考加答案；改为**取最后一个解析成功的完整对象**。

- **输入额度由使用者定：**输入额度 = 最大输出 × 倍率，程序不探后端窗口多大（懒加载的接口只会回"未加载"），只报换算结果，不对后端提要求。

7.2 CLI 接口

用途	命令
启动 Web 界面	<code>python main.py</code> （默认 8811， <code>--port</code> 指定， <code>--port auto</code> 自动挑空闲）
仅检测 LLM 后端	<code>python main.py --check</code>
从锁文件续跑	<code>python main.py --continue <树根> --episode 3</code>
列出可用音色 / 字体	<code>python main.py --voices</code> / <code>python main.py --fonts</code>
指定后端	<code>python main.py --backend <名> --base-url <地址> --model <模型></code>
单元测试	<code>python -m unittest discover -s tests -t .</code>
界面冒烟（真浏览器）	<code>python tools/ui_smoke.py --url http://127.0.0.1: --shot-dir <目录></code>
端到端（走 HTTP）	<code>python tools/e2e_http.py</code>

7.3 Web 界面

单页界面，四个页面，`hash` 是唯一路由（可直接把 `#config` 发给人）：

页面	干什么
项目	立项建树、项目卡片、素材入库、排图（排图依据弹窗含素材类型 / 重点方向 / 计划期数 三态：留空按素材定、填数按数排）、勾期跑批、设为当前项目
脚本	生成脚本、判稿（带本期期号）、逐句查看与手改、门禁表、重判
合成	选期合成、出片、试听、查看报告与产物

页面	干什么
配置	全量配置（按功能区渲染）、路径类点位填本机绝对路径、试听

页面切换有一个刻意的设计：**脚本页 / 合成页切进去默认不重拉数据**（0 请求），只有项目页 / 配置页会重拉；全 UI 无轮询，唯一在转的是任务进度（1.2 秒一次）。刷新时保留当前勾选——勾一次丢一次是不可接受的。

任务看板（批任务与单期脚本各一块）的按钮状态**跟着任务是否真收工走**：跑着时开工按钮摁不动、中止按钮露着，收工才恢复；批任务另按（第几期-1+本期进度）/总期数 把本期进度折算成整批进度（两处的机制见 4.10 末）。

7.4 HTTP 端点

服务端 `http.server` 实现，端点按业务域分组：

域	端点
配置	<code>/api/config</code> 、 <code>/api/pickfile</code> （路径类点位开对话框选目录）
素材	<code>/api/ingest</code> 、 <code>/api/sources</code> 、 <code>/api/source</code>
脚本	<code>/api/script/generate</code> 、 <code>/api/scripts</code> 、 <code>/api/script</code> 、 <code>/api/script/save</code> 、 <code>/api/script/issues</code> 、 <code>/api/script/gate</code>
合成	<code>/api/render</code> 、 <code>/api/batch</code> 、 <code>/api/job/stop</code> 、 <code>/api/estimate</code>
项目	<code>/api/projects</code> 、 <code>/api/project</code> 、 <code>/api/plan</code> 、 <code>/api/continue</code> 、 <code>/api/report/</code> 、 <code>/api/assets/regenerate</code>
语音	<code>/api/voices</code> 、 <code>/api/voice/profile</code> 、 <code>/api/voice/build</code> 、 <code>/api/tts/preview</code> 、 <code>/api/tts/state</code> 、 <code>/api/tts/setup</code>
后端	<code>/api/backend</code> 、 <code>/api/backends</code> 、 <code>/api/models</code> 、 <code>/api/backend/test</code>
资源与任务	<code>/api/engines</code> 、 <code>/api/fonts</code> 、 <code>/api/fontfile</code> 、 <code>/api/bgm/</code> 、 <code>/api/file/</code> 、 <code>/api/task/</code> 、 <code>/api/jobs</code>
运行环境	<code>/api/deps/state</code> 、 <code>/api/deps/install</code> （ffmpeg/ffprobe 探测与一键安装，走 <code>/api/task/</code> 轮询）

7.5 对外产物

一期跑完，落盘的成品是：

产物	位置
脚本 json	脚本/<期号>.json （逐期对话形式留档：脚本/<期号>.form.json）
成片音视频	音视频/ （横屏 mp4 + 竖屏 mp4）
字幕	字幕/ （SRT / LRC / 整秒 TXT / 洁版 TXT，四份同源）
图文稿	图文/
封面	封面/ （三尺寸）
AIGC 合规标识	全部产物写入元数据隐式标识（七要素 JSON）；音频片头最前加一句语音声明，字幕时间轴随声明时长整体后移不错位
门禁报告	过程/<期号>/gate.generate.json （脚本）、报告/<期号>.report.json （合成）
音色体检报告	过程/<期>/timbre_repair.json （基线、逐句分数、重试记录、人工审名单）

八、依赖与存储

8.1 依赖

依赖	用途	说明
edge-tts	在线朗读引擎	客户端库 LGPL-3.0；输出受微软服务条款约束
Pillow	背景图 / 封面渲染	HPND
ffmpeg / ffprobe	音频与视频处理的全部工作	定位走 bins.py （PATH → 项目 bin/ ）；机器上没有时配置页「运行环境」卡可一键安装（国内镜像下载，sha256 校验）

依赖	用途	说明
本地语音 (Qwen3-TTS)	离线朗读	不在主依赖里 ：torch 等约 4.8 GB，单独放在 <code>tts_service/</code> 的独立环境，按需安装
Python	运行时	3.11 或更新

除以上两项第三方包，其余全部走 Python 标准库：HTTP 服务、LLM 客户端、JSON Schema 校验、字幕生成都是自己写的。

8.2 存储

落点	内容
<code>config.json</code>	用户配置（平铺点号键，唯一配置真值）
<code>projects/<项目>/</code>	项目树，11 类产物各归其位（见 2.3）
<code>_projects.json</code>	项目登记表（留在输出目录根上）
<code>过程/<期号>/</code>	脚本门禁报告、锁文件、中间稿
<code>报告/<期号>.report.json</code>	合成阶段的产物门禁结论

一个项目一个文件夹，拿走它等于拿走全部。

九、边界与不做什么

架构文档不列边界等于没写，因为"不做什么"和"做什么"一样是设计决定。

不做什么	为什么
合成端不写稿、不判稿	现场生成等于把审过的稿换成另一份；合成端没有判它的资格与判据
写作阶段不判句长	句长是全篇一把尺的量，归门禁判且两个方向都有处方；写作阶段替它把关会把"一条坏句"放大成"整段停摆"
不从节中间切料	半个节取不出原文、也算不出账

不做什么	为什么
不把"没检查"记成"通过"	模型漏返回的项记为「未判定」进待复核；报告上的假绿灯比不检更坏
不让模型改说话人	说话人不在补丁 schema 里——把 A 的问话翻给 B，就成了 B 自己问自己，那是拿格式代替语义
不用删的方式治重复	删了核账又报缺口 → 下一轮又补 → 又复读；所以重复走"就地换"。整期去重只作 最后兜底
不报无依据的推断	去重后不再推断"这说明素材撑不满目标时长"——实测那一期料是目标的 2.09 倍，把责任推给素材方向是错的。报告照实说：删了几句、压比多少、缺口多少字
不做无限重试 / 不空转	轮次用尽就交人；"再试一轮"不是可靠性，是延迟
不替使用者决定输入额度	额度由使用者按自己的后端定；程序只报换算结果，不对后端提要求

已知的开放项（如实列出）：

- 连说轮次里"主讲人连着说太少"（少于 3 句）目前**没有门禁判据**——实测中出现得不多，但一旦出现没有任何一道门管它，属新设计待定。
- 单元测试与界面冒烟保证了"控件真的接上了"；音色体检的判定与修复已在真实成片上验证（2c 期 253 句产线判定与人工听感完全一致；15 个耳标样本三判据 15/15 分离；2da 0054 修复前后复现探针数值）。四份字幕的产物链已由真音频段的写盘集成测试覆盖。仍缺的是**跨平台播放侧**的长期回归——LRC 在各音频平台的渲染行为由平台侧决定，只能以真实投稿持续观察。

十、版本演进要点

版本	新增/变更要点
v1.4.0	排图依据新增「计划期数」 ：三态输入（留空＝按素材体量定、填数＝按数排），产能预检兜底（素材撑不起在凝缩前中止并回报最多可排期数）；产能中止报错指明修改入口，排图报告弹窗加「去改排图依据」直达； 移除「加一期」 ——手工加的期没有素材落点，出片按落点取料取不到内容

版本	新增/变更要点
v1.3.0	音色体检扩为六维三判据合验 ：新增暗淡子分与身份下限，补上「基频下偏 × 低频饱满」的身份漂移盲区；修复保底由温度兜底改为复用最近干净句种子；服务进程挂 Windows Job 对象，主程序死亡内核收树还显存
v1.2.0	判定分四维（+存在感频带 2.4-4.8kHz，权重在修复前音频回放定标）；音频层日志句号改用文件名口径
v1.1.0	音色体检（timbre guard）落地 ：逐句三维判定（基频/谱心/响度，按角色建基线，人耳定标钉回归）+种子偏移定点重出（seed+1→+3，温度不动）；合成段收口为纯函数全量重建——删"音频数==句数即整体复用"分支（改稿不增删行会静默念旧稿）
v1.0.0	功能面冻结，转语义版本 ；CHANGELOG 体例收紧为现象/根因/修法/验证；README 补功能与用法；版本齐步测试（含架构册抬头）
v0.47.0	ffmpeg/ffprobe 定位收成 bins.py 唯一处 （PATH → 项目 bin/ → None），配置页「运行环境」卡一键安装（国内镜像，sha256 校验 fail-closed）——此前定位散在四模块且只搜 PATH，安装等于没装
v0.47.1	启动抬头印两遍修复（展示层）
v0.46.0	字幕第四份：洁版 TXT （整秒戳摘掉方括号）——方括号是给解析器的定界符，给人的眼睛是噪声；四份同源同次生成，不从产物反解
v0.45.0	脚本页进度条接上写稿段 ：预算表收成五段（准备/写稿/检查/门禁/收尾）——此前写稿段零刻度，实测轨迹只有四跳全挤在收尾
v0.44.0	一期三份字幕：SRT / LRC / 整秒 TXT 同源生成
v0.43.0	落盘保留汉字与西文之间的空格——提示词示范的字距不再落盘即被抹平

版本	新增/变更要点
v0.42.0	字幕版式三档并两档：单行改框内横滚、双行删除，两档共用同一个框；回顾整切三条
v0.41.0	歌词档改固定框+实时预览（框几何 <code>frame_of</code> 成为预览与烧录共同出处）
v0.40.0	字幕分三档，「歌词」档料多长排多少行
v0.39.0	字幕多落一份 LRC ：音频平台的字幕位只走歌词渲染，没有 SRT 解析通道；与 SRT 同源同一次生成（不从 SRT 反解），用原始整句不断行、只写起始时间、时间标签两位百分秒；说话人名判据与 ASS 同源（LRC 没有样式层可承载 A/B 双色）
v0.38.1	任务看板"等收工"的链串成一条真等待的线（从前按钮在任务刚起步的一刻就恢复，能起第二个任务）；批任务每期开头把 本期 进度清零——"只增不减"归单期所有
v0.38.0	捧眼形式（ <code>anchor</code> ）A 连说上限由 1 放宽到 2——只动卡上一个数，提示词四处 / 门禁 / 界面自动跟着变；放宽的是 许可 不是 形状
v0.37.0	段内轮次重排为「补字 → 查重 → 替换 → 计算」 ；重复由"事后删"改为"就地换"（行数不变、同长度换内容）；并句判据换成恒非空的可行域区间；补字轮与首写轮视野对齐（都带全篇已写正文）；整期去重降级为最后兜底
v0.36.0	三段串行定序：出货 → 检查 → 门禁 ；补丁从"全有或全无"改成逐条落地 + 拒收原因回灌；段级量配置化；末尾标点两步判
v0.35.0	门禁与内容检拆成两段各自成环 ；语义检降为开关（默认关）、承诺链检必检；两段内部零重写

版本	新增/变更要点
v0.34.6	粘合挪到格式收束之后：片头标签是"开场"、片尾是"收束"（顺序错的根因修复）
v0.34.4	每种对话形式带一段「形状示范」（三到七句实例，进整篇 / 分段 / 插入三处提示词）
v0.34.0	删掉后处理 <code>enforce_max_run</code> （机器改 speaker 是添乱）：连说超限改走"门禁点名 → 模型并句"
v0.33.0	对话形式逐期留档（脚本/ <code><期号>.form.json</code> ）——判稿读本期自己的档
v0.32.0	风格倾向甩掉"提问频率"与"情绪密度"两维；对话形式成为唯一的上限来源
v0.31.0	连句上限从素材类型卡搬进「对话形式」；卡只讲形态，不再管上限
v0.30.0	段主旨落盘（旁挂文件）、前期回顾开关、配置页控件形态改规矩
v0.28.2	插入轮上下文与写脚本同源 ：原文、段主旨、范式卡、标签列接通
v0.28.0	句尾标点进硬约束：提示词教"按语义选符号"，门禁管"有没有"

版本	新增/变更要点
v0.27.0	<code>emotion</code> 字段回归但换血：只装语篇标签（八词表定形）
v0.26.4	碎段合并：低于阈值的逻辑段并进配额较小的邻居
v0.26.0	脚本侧 <code>emotion</code> 字段整体删除：一句台词只剩 <code>speaker</code> 和 <code>text</code>
v0.25.0	配额权重的单位改正：原始字符数 → 原文有效字
v0.23.0	配额权重换成"凝缩对应的原文量" ——一段复读的根本原因是配额摊歪了，不在提示词
v0.22.0	段边界改成两步走（先按内容逻辑分组，再逐段过输入额度）；切分单位统一到整节凝缩
v0.21.0	粘合只挂「整期定稿」这一个点：片头尾不进轮，不参与生成、不过门禁、不核字数
v0.17.0	「后端窗口」这层账本撤掉：程序不再替后端记账，只做换算与切段
v0.16.0	三层拆开、各用各的尺：段划分改由程序装箱（纯算术），规划轮缩成"起标题 + 每段一句话题目"

版本	新增/变更要点
v0.14.0	段内补字从"把现有句子改长"改为" 新增句子插进去 "（原做法在缺口偏大时退化成整段重打）
v0.11.0	AIGC 合规标识整体落地 ：元数据隐式标识（七要素）+ 片头语音声明 + 字幕时间轴随之后移
v0.10.0	素材容量从写死的下限改成由（时长、档位）实时推导的公式
v0.8.0	两条写作路线不再由开关选，而由项目模式定死 （成稿规划走分段，逐期即兴走整篇）
v0.6.0	项目做成树形结构 ：立项即建目录，产物分阶段落盘；路径收口到一个模块
v0.4.0	内容检从"产物出来之后"挪进脚本生成的重试回路
v0.1.0	首个版本：从零构建完整管线（素材 → 脚本 → 声音 → 字幕 → 画面）

一句话概括这条线：**从"一次生成 + 事后清洗"一路走到"分段成环 + 段内成环 + 判修分离 + 零重写"**。每一版都不是加功能，而是把"判的人"和"改的人"分得更清，把"环"划得更小、更明确。

07 | Orchestrator 架构文档

一、系统概览

Orchestrator 是一个链驱动技能编排器，核心理念：**LLM 只做两头，中间是死的**。

用户编排 Pipeline (Pipeline Tab) → 选择链 + 任务描述 + 上传数据 (对话 Tab)

→ Round 1: 需求分析 (LLM + 16 工具, 按用户提示词前处理)

→ Round 2: (可选) skill-sub 优化 (衔接兼容性检查)

→ Round 3+: 死执行 (subprocess 跑 skill 脚本, LLM 不参与)

→ 最终输出整理 (LLM 按用户提示词整理交付)

Orchestrator 不是聊天工具。 没有选链就没有对话——普通对话 (ReAct) 已在 v2.8.0 彻底移除。

1.1 核心设计原则

原则	说明
链是主体	没有链就没有对话。Pipeline 编辑器是主界面，对话区是执行结果展示
LLM 只做两头	LLM 出现在需求分析 (前处理) 与最终输出整理， 不干预链执行
中间是死的	链执行 = subprocess 真跑 skill 脚本，确定性、可复现、可审计
skill 是原子池	每个技能可拆解为可执行原子 (CLI/脚本/子步骤)，编排 = 挑原子组链
编排器不替 skill 干活	skill 自带领域能力 (如 RAG 检索在 local-rag-builder 自己的 rag_core.py)，编排器只提供通用衔接工具
纯提示词 skill 不编排	无可执行脚本入口的技能直接过滤，不进入 Pipeline 列表
配置可调	超时/max_tokens 从 settings.json 读取，非硬编码

1.2 与 v2.0 的关键差异

维度	v2.0 (LLM 编输出)	v2.8 (真执行 + LLM 两头)
执行机制	每步 <code>llm.chat()</code> 编输出	subprocess 真跑 skill 脚本 (<code>chain_engine _run_script</code>)
LLM 角色	步骤执行器	前处理 + 输出整理 ，中间不碰
普通对话	无链时回退 ReAct	已彻底移除 ，无链直接拒绝
工具	7 个	16 个 (+文件操作 6 + 数据工具 3)
技能过滤	全部显示	可执行过滤 (有主脚本入口才显示，32 个)
loop	输出不回传 (假循环)	输出回传 (每轮结果喂下一轮)
输入通道	仅文本	对话框纯文本 + 上传通道 (<code>/api/upload</code>)
端口	8765/8766	8788 (避 RAG 8765-8767、写作 8770/8777)

二、双入口架构

Orchestrator/

|—— main.py

|—— orchestrator/

| |—— __init__.py

| |—— web_ui.py

| |—— agent_loop.py

词, 非 ReAct)

| |—— chain_engine.py

| |—— chain_model.py

| |—— skill_scanner.py

| |—— agent_config.py

| |—— llm_client.py

CLI 入口 (Web UI/批处理/管道)

Python 包

导出 + 版本唯一源 (__version__)

Web UI 服务器 + 前端内嵌 HTML

ORCHESTRATOR_SYSTEM_PROMPT (编排器本体提示

真执行引擎 (_run_script subprocess)

数据模型

SKILL.md 扫描器

配置管理

LLM 通信 (OpenAI 兼容, 多后端)

```

|   |—— direct_llm_client.py    # GGUF 直载 (CLI 用)
|   |—— model_manager.py        # 模型管理
|   |—— memory.py               # 记忆系统
|   |—— static/web_ui.js        # 前端 JS
|   |—— chains/                 # 已保存 Pipeline
|   |—— tools/                  # 16 个工具 (4 模块)
|—— data/config/settings.json   # 配置 (唯一生效文件)
|—— input/                      # 上传文件落盘目录
|—— archive/                    # 归档 (tkinter gui_agent 等死代码)
|—— PROTOCOL.md                 # 接口契约
|—— README.md                   # 使用手册
|—— setup.bat                   # 一键启动 (固定 8788)
|—— requirements.txt            # 依赖

```

2.1 Web UI 模式 (主)

```

python main.py --web           # 默认 8788
setup.bat                      # 一键启动 (安装依赖 + 启动 + 自动开浏览器)

```

三 Tab:

Tab	功能
对话	选链 + 任务描述 + 上传文件 → 需求分析 → (skill-sub) → 死执行 → 输出整理
配置	LLM 后端 / 提示词 (系统只读 + 用户提示词) / 技能路径
Pipeline	可视化 seq/par/loop 编排器 (只显示可执行技能)

2.2 批处理 / 管道模式

```

python main.py --batch input.json output.json    # JSON 批处理 (真执行)
cat queries.jsonl | python main.py --jsonl       # JSONL 管道 (真执行)

```

批处理/管道走 chain_engine 真执行 (subprocess 跑技能脚本), LLM 不可用时降级为步骤规划输出。

三、Web UI 架构

3.1 HTTP 服务器

基于 Python 标准库 `http.server.BaseHTTPRequestHandler`，零外部依赖。
`ThreadingTCPServer + daemon_threads`（单线程会阻塞全站，已修）。

路由	方法	功能
<code>/</code>	GET	Web UI 主页面（HTML 内联）
<code>/static/</code>	GET	静态资源（web_ui.js、marked.min.js 本地化）
<code>/api/chat</code>	GET/POST	链驱动执行（无 pipeline 拒绝）/ 配置重置
<code>/api/config</code>	GET/POST	配置读写（落盘 settings.json，联动 agent.llm）
<code>/api/skills</code>	GET	技能列表（只显示可执行技能）
<code>/api/llm/models</code>	GET	可用模型列表
<code>/api/llm/test</code>	POST	LLM 连通性测试
<code>/api/pipelines</code>	GET/POST	Pipeline 列表 / 保存
<code>/api/pipelines/run</code>	POST	执行 Pipeline（真执行）
<code>/api/pipelines/delete</code>	POST	删除 Pipeline
<code>/api/upload</code>	POST	文件上传（base64 → input/，路径穿越净化/50MB/重名序号）

3.2 对话四阶段（LLM 只做两头）

Round 1 — 需求分析（前处理）

LLM(用户任务 + user_prompt + 16 工具清单 + 附件路径)

→ 用工具处理数据 (db_query/read_table/image_info) , 产出最小中间输入



Round 2 — skill-sub 优化 (可选, 勾选开启)

衔接兼容性检查 → 黏连转换 → 里程碑标记



Round 3+ — 死执行

subprocess 跑 skill 脚本 (params → CLI 参数, 前步输出 → stdin/下一步输入)

LLM 不参与, 结果原样



Round 4 — 最终输出整理 (输出侧)

LLM 按 user_prompt 整理交付 (Markdown/表格/文件路径)

明确: 不保证完全按提示词——Pipeline 已产最终形态则如实呈现

3.3 用户提示词的语义 (v2.7.0 定稿)

用户提示词 = 你对"理解任务"和"呈现结果"的偏好, 不干预链的执行细节。

- **输入侧**: 注入需求分析 prompt (前处理偏好: 怎么理解任务、怎么处理数据)
- **输出侧**: 注入最终输出整理 prompt (格式偏好: Markdown/表格/简洁)
- **中间执行**: 不注入 (skill 脚本是死的, 提示词插进去也没用)

3.4 Pipeline 编辑器

三栏布局, 左侧技能列表只显示可执行技能 (有主脚本入口, 32 个):

操作	说明
双击左侧技能	添加到画布 (seq 模式)
点 + 并行组	创建 par 容器 (ThreadPoolExecutor 真并发)
点 + 循环组	创建 loop 容器 (for 真循环, 输出回传)
双击节点	编辑参数 (params → CLI 参数)

操作	说明
右键模式切换	seq/par/loop 切换
保存	模态框输入名称 → chains/{name}.json
运行	发送到后端真执行引擎

四、执行引擎（真执行）

4.1 链执行（_execute_tree）

核心执行函数，支持三种模式，**全部 subprocess 真执行**：

```
_execute_tree(nodes, output, depth, step_counter, prev_output):
    for node in nodes:
        if mode == "seq":
            result = _run_skill_node(name, params, prev_output) # subprocess
            prev_output = result # 传递下一步
        elif mode == "par":
            ThreadPoolExecutor: # 真并发
                for child in children: executor.submit(_run_skill_node, ...)
        elif mode == "loop":
            for t in range(times): # for 真循环，每轮结果回传下一轮
                last_result = _execute_tree(children, ..., prev_output=loop_input)
                loop_input = last_result # 输出回传 (v2.4.0 修复)
```

loop 语义澄清：loop 是"链上这一段子步骤按固定次数重复执行，每次把结果喂给下一次"——类似编程的 `for` 循环（次数固定、无决策），**不是** agent 式 `while` 自适应循环（模型决定何时停）。loop 与 par 正交可嵌套：loop 每轮内部可并行，轮间串行（依赖上一轮输出）。

4.2 单技能执行（_run_skill_node → chain_engine）

1. `_find_skill_dir(name)` # 找技能目录（/，随宿主平台安装位置而定）

2. `_get_skill_scripts(sdir)` # 收集 scripts/ 下可执行脚本

3. `_get_main_script(name, scripts)` # 入口匹配（下划线/连字符归一化 + `main/cli/run` 标准入口 + 排除纯配置脚本）

4. `_run_script(main, cli_args, ...)` # subprocess 真跑

- `node.params` → CLI 参数 (`command/args/--key value`)

- 前步输出 → `stdin` 传入

- 捕获 `stdout/stderr`, 超时返回 [超时]

入口匹配规则 (v2.5.0 定稿):

- 技能名下划线/连字符归一化匹配 (`workday-calendar` ↔ `workday_calendar.py`)
- 标准入口名: `main.py` / `run.py` / `index.py` / `cli.py`
- 排除纯配置脚本: `settings.py` / `config.py` / `__init__.py` / `_paths.py` 不算入口
- 其余任意 `.py/.sh/.bat` 都算可执行入口

当前可编排技能: 32 个 (66 个技能中过滤掉无入口的)。

4.3 skill-sub 优化

算法主导的链分析 (非 LLM 自由发挥):

1. 读 SKILL.md → 提取 `tags/triggers/description`

2. 算法规则比较步骤间输入输出兼容性

- 不兼容 → 插入转换步骤

- 兼容 → 直接传递

3. LLM 模糊回退 (仅当算法无法确定时)

4. 里程碑自动标记

五、数据模型

5.1 PipelineNode (chain_model.py)

```
@dataclass
class PipelineNode:
    id: str                # UUID hex[:8]
    skill_name: str        # 技能 slug
    display_name: str      # 显示名称
    mode: str              # seq | par | loop
    children: list         # 子节点
    loop_times: int        # 循环次数
    input_text: str        # 用户输入
    params: dict           # 技能参数 (→ CLI 参数)
    extra: dict            # skill-sub 优化数据 (黏连点/里程碑)
```

5.2 保存格式 (chains/{name}.json)

```
{
  "name": "my-pipeline",
  "nodes": [...],          // 扁平化节点
  "tree": [...]            // 完整树结构 (含 params/extra)
}
```

六、LLM 客户端

6.1 架构

基于纯 `urllib` 的 OpenAI 兼容客户端，支持多后端：

后端	地址
LM Studio	<code>http://localhost:1234/v1</code>
Ollama	<code>http://localhost:11434/v1</code>

后端	地址
OpenAI 兼容	自定义

6.2 关键配置（data/config/settings.json，唯一生效）

```
llm.backend:      lmstudio
llm.base_url:     http://localhost:1234/v1
llm.model_name:   qwen/qwen3.5-35b-a3b    # 配置持久化，启动不再被硬编码覆盖（v2.5.0 修复）
llm.timeout:      1800 秒（链步执行超时）
llm.max_tokens:   40960（每步最大输出）
```

七、工具系统（16 个）

分工原则：skill 自带领域能力（RAG 检索、颜色计算、工作日历都在 skill 自己的 scripts/ 里），编排器只提供**通用衔接工具**。

模块	工具	用途
file_tool.py	read_file / write_file / list_directory	基础文件读写
file_ops_tool.py	copy_file / move_file / delete_file / append_file / make_dir / find_files	链中间产物流转 （v2.3.0 新增，安全校验：空路径/系统目录/根路径拦截）
web_tool.py	web_fetch / web_search / python_execute	网络 + 代码执行
data_tool.py	db_query / read_table / image_info	前处理防 token 爆炸 （v2.7.0 新增：SQLite 查询写拦截/csv/xlsx 摘要/图片元数据）
skill_loader.py	load_skill	加载技能定义

前处理阶段：LLM 通过 `_tools_summary` 注入 16 工具清单，先调工具取必要信息（不整读大文件/数据库）。

八、配置系统

8.1 配置文件（data/config/settings.json）

```
{
  "llm": { "backend": "lmstudio", "base_url": "http://localhost:1234/v1",
           "model_name": "qwen/qwen3.5-35b-a3b", "timeout": 1800,
  "max_tokens": 40960 },
  "agent": { "max_steps": 20, "verbose": true },
  "search": { "backend": "duckduckgo", "presets": [] },
  "prompt": { "user": "" },
  "skills": { "dirs": [] }
}
```

配置持久化链路：界面保存 → 落盘 settings.json → `_recreate_llm` 联动 agent.llm → 重启后 make_llm 优先用配置值（不硬编码覆盖）。

8.2 提示词

提示词	状态	说明
系统提示词	只读	<code>ORCHESTRATOR_SYSTEM_PROMPT</code> （编排器本体：LLM 只做前处理+输出整理，非聊天机器人）
用户提示词	可编辑	仅作用于前处理与输出格式，不干预 Pipeline 执行

九、CLI 接口

```
python main.py --web                # Web UI（主模式，默认 8788）
python main.py --batch in.json out.json # 批处理（真执行）
python main.py --jsonl              # 管道模式（真执行）
python main.py --port 8788          # 指定端口
```

十、演变路线

阶段	状态	内容
v1.0-v1.2	完成	ReAct 初始架构 → Skill Pipeline 基础编排
v2.0	完成	链驱动架构重塑：Web UI / 真并行循环 / skill-sub / PROTOCOL
v2.1	完成	致命缺陷修复：双响应崩溃 / 假执行 / 配置加载 / CDN / 单线程
v2.2	完成	端口 8788 / setup.bat 重写 / 归档 tkinter
v2.3	完成	链执行真执行化 （subprocess）+ 6 文件工具 + 编排过滤
v2.4	完成	loop 真循环（输出回传）+ 技能入口过滤
v2.5	完成	配置持久化 / 会话历史 / 系统提示词真实化 / 技能 9→32
v2.6	完成	文件上传通道 （对话框纯文本 + 数据走上传）
v2.7	完成	LLM 只做两头 + 3 数据工具（防 token 爆炸）
v2.8	完成	彻底移除普通对话 ，编排器定位纯化 + 文档补齐
v2.9+	规划	跨链状态共享、分支条件、循环变量

十一、目录结构（agent/Orchestrator/）

```
agent/Orchestrator/  
├── main.py                # CLI 入口  
├── orchestrator/          # Python 包  
│   ├── __init__.py        # 导出 + __version__ 唯一源  
│   ├── agent_loop.py      # ORCHESTRATOR_SYSTEM_PROMPT（编排器本体提示词）  
│   ├── agent_config.py    # 配置管理  
│   ├── chain_engine.py    # 真执行引擎（subprocess）  
│   ├── chain_model.py     # 数据模型  
│   ├── memory.py          # 记忆系统  
│   ├── llm_client.py      # LLM 通信  
│   ├── direct_llm_client.py # GGUF 直载（CLI）  
│   └── model_manager.py    # 模型管理
```

```
|   |—— skill_scanner.py    # 技能扫描
|   |—— web_ui.py          # Web UI 服务器
|   |—— static/web_ui.js   # 前端 JS
|   |—— chains/            # 已保存 Pipeline
|   |—— tools/             # 16 工具 (file/file_ops/web/data/skill_loader)
|—— data/config/settings.json # 配置
|—— input/                 # 上传文件
|—— archive/              # 归档死代码
|—— PROTOCOL.md           # 接口文档
|—— README.md             # 使用手册
|—— setup.bat             # 一键启动
|—— requirements.txt       # 依赖
```

08 | silprespec-orchestrator 架构文档

一、系统概览

silprespec-orchestrator 是前置规范编排器，核心理念：LLM 只填空不决策，前置规范 > 后置验证，PY 确定性组合，槽位是减法不是加法。

用户输入 + 工具集

→ [Orchestrator 主控]

—— 输入分类（穷举 6 类别，LLM 填空分类 → PY 查表映射）

—— 选编排模式（serial / parallel, PY 确定性）

—— 分解子任务（LLM 填空，每子任务指定一个工具）

—— 生成进度地图（ProgressMap，贯穿全局）

→ [逐子任务执行]

for each subtask:

Mapper（选 14 种组合之一 + 设参 + output_limit）

→ LLM 填空选组合编号，PY 查表验证

Composer（PY 确定性组合，调 exec_recipe）

→ 生成原子（LLM 填空） → 后处理原子（PY） → 校验原子（PY） → 观测原子（PY）

→ 不通过则 retry loop（最多 max_retry 次）

Executor（LLM 填空生成工具输入 → HTTP 调智能体 API）

Adapter（步骤间适配：能直通则直传，不能则 loop 回 Mapper 选适配组合）

→ 汇总输出

编排器不替智能体干活。 智能体自带领域能力（RAG 检索在 rag-assistant、写作在 structured-writer），编排器只做头部规划：选前置规范 → 填空 → 交付给智能体走内部流程。

1.1 核心设计原则

原则	说明
LLM 只填空不决策	LLM 出现在选组合（填编号）、生成内容（填槽位/填文本）， 不决定流程走向 。流程由 PY 确定性查表驱动
前置规范 > 后置验证	先约束再生成（14 种组合在生成前就钉死格式/值域/槽位），不是生成后检查
PY 确定性组合	每个组合骨架是 生成→后处理→校验→观测，PY 部分确定性查表，只有生成那步是概率的
槽位是减法不是加法	槽位定义"只准填这些"，多余 key 是编造，不是"多给了信息"
保留聪明剥夺自由度	LLM 负责填空（聪明），PY 负责约束（剥夺自由度）。两者正交不冲突
穷举 > 猜测	14 种组合在当前原子库下穷举完，不存在第 15 种。不靠 LLM 猜该用什么规范
进度地图贯穿全局	每步 LLM 都看到完整用户初始输入 + 输入分类 + 全局进度，不盲目执行
递归自举	编排器自身的前置规范也来自 14 种组合（Adapter 适配时 loop 回 Mapper 选组合）
配置推动	CLI > config.json > DEFAULT_CONFIG，max_tokens / timeout 从配置读取

1.2 与 Orchestrator 的关键差异

维度	Orchestrator（链驱动）	silprespec-orchestrator（前置规范编排）
核心定位	技能链执行（subprocess 跑脚本）	前置规范选择+填空（LLM 填空 → 调智能体 API）
LLM 角色	前处理 + 输出整理（两头）	选组合 + 填空生成（贯穿每步，但只填空不决策）
中间执行	subprocess 真跑 skill 脚本	PY 确定性组合（exec_recipe）+ HTTP 调智能体

维度	Orchestrator（链驱动）	silprespec-orchestrator（前置规范编排）
规范来源	用户编排 Pipeline	14 种穷举组合自动选择
loop 语义	for 真循环（次数固定）	retry loop（校验不通过则重试）+ Adapter loop（适配回 Mapper）
端口	8788	8789

二、目录结构

silprespec-orchestrator/

```

├─ main.py                # CLI 入口 (Web UI / 单次查询 / 检测)
├─ setup.bat              # Windows 一键启动 (端口 8789)
├─ requirements.txt       # 依赖清单 (零第三方依赖, 纯标准库)
├─ config.json            # 配置 (LLM + orchestrator + combos)
├─ CHANGELOG.md           # 版本更新日志
├─ PROTOCOL.md            # 接口契约
├─ llms.txt               # AI 可读项目自描述
└─ README.md              # 项目说明

```

```

├─ silprespec_orchestrator/  # ★ 智能体核心包
|   └─ __init__.py          # 导出 + __version__ 唯一源 (0.1.0)
|   └─ orchestrator.py      # 编排器主控 (分类 → 选模式 → 分解 → 执行 → 汇总)
|   └─ progress_map.py      # 进度地图 + 输入分类 + 编排模式选择
|   └─ mapper.py            # 选组合 + 设参 (LLM 填空选, PY 查表验证)
|   └─ composer.py          # PY 确定性组合器 (调 exec_recipe)
|   └─ executor.py          # LLM 填空 + HTTP 调智能体 API
|   └─ adapter.py           # 步骤间适配 (直通 or loop 回 Mapper)
|   └─ combo_registry.py    # 14 种穷举组合声明
|   └─ tool_registry.py     # ToolSpec/FieldSpec/ExampleSpec + 三智能体注册
|   └─ atoms.py             # 原子库 (10 原子 + Recipe + exec_recipe 通用执行器)
|   └─ pipeline_model.py    # 数据模型 (WayConfig/WayResult/5 种方式)
|   └─ llm_client.py        # LLM 统一客户端 (LM Studio/Ollama/自定义)
|   └─ web_ui.py            # Web UI 服务器 (http.server)
|   └─ static/              # 前端静态资源
|       └─ index.html        # 四 Tab 页面 (编排/组合/工具/配置)
|       └─ style.css         # 样式 (深色 Tab + #667eea 主色)
|       └─ web_ui.js         # 前端逻辑 (LLM 配置 + 工具详情 + 组合展示)

```

三、核心编排架构

3.1 主控流程 (orchestrator.py)

```
Orchestrator.run(user_input, tool_names):
```

① 分类 (穷举 6 类别)

```
classify_input(user_input, chat)
```

→ LLM 填空分类 (只输出类别 id) → PY 查表映射

→ (extract / generate / analyze / verify / transform / orchestrate)

② 选工具

_select_tools(tool_names) → 从 TOOL_REGISTRY 取 ToolSpec 列表

③ 分解子任务 (LLM 填空)

```
_decompose(user_input, category, tools, chat)
```

→ LLM 根据工具清单把任务拆为子任务数组

→ 每子任务: {"name": "步骤名", "desc": "描述", "tool": "工具名"}

→ 失败回退为单任务

④ 选编排模式 (PY 确定性)

```
select_orchestration_mode(category, num_steps)
```

→ num_steps ≤ 1 → serial

→ extract/transform/verify → parallel

→ generate/analyze → serial

→ 默认 → serial

⑤ 生成进度地图 (ProgressMap)

记录 user_input + category + mode + 每步状态

贯穿全局, 每步 LLM 都能看到

⑥ 执行

serial → `_run_serial` (串行, 前步输出经 Adapter 传下一步)

parallel → `_run_parallel` (ThreadPoolExecutor 并行)

⑦ 汇总输出

`_summarize` → 编排结果 + 每步状态 + 组合标签 + 工具标签

3.2 串行执行与步骤间适配 (`_run_serial`)

```
_run_serial(subtasks, pm):
```

```
    prev_output = {}
```

```
    for i, st in enumerate(subtasks):
```

```
        step = pm.steps[i]
```

```
        tool = get_tool(st["tool"])
```

① 步骤间适配 (Adapter)

```
        if tool and prev_output:
```

```
            adapted = adapter.adapt(prev_output, tool, pm)
```

→ can_accept? 上一步输出的 key $\supseteq$ 下一步工具的必填字段

→ 能直通 → 直接传递 (prev_output)

→ 不能直通 → loop 回 Mapper 选适配组合 → Composer 执行适配

```
        else:
```

```
            step.input_data = {"query": st["desc"]}
```

② 选组合 + 设参 (Mapper)

```
        combo, config = mapper.map(tool, st["desc"], pm)
```

→ LLM 填空选组合编号 (1-14) → PY 查表验证

→ 设参数 + output_limit (从 combo.output_limit 和配置合并)

③ PY 确定性组合 (Composer)

```
comp_result = composer.compose(combo.id, st["desc"], config)
```

```
→ exec_recipe(way_id, wc, user_input, chat)
```

```
→ 生成原子 → 后处理原子 → 校验原子 → 观测原子
```

```
→ 不通过则 retry loop (最多 max_retry 次)
```

④ 调智能体 (Executor)

```
exec_input = comp_result.get("filled", step.input_data)
```

```
exec_result = executor.execute(tool_name, exec_input, pm)
```

```
→ LLM 填空生成工具输入 JSON → HTTP POST 调智能体 API
```

⑤ 记录状态

```
success → pm.mark_done(step_id, exec_result)
```

```
failure → pm.mark_error(step_id, error)
```

```
prev_output = exec_result # 传下一步
```

3.3 并行执行 (_run_parallel)

```
_run_parallel(subtasks, pm):
```

```
ThreadPoolExecutor(max_workers=min(4, len(subtasks))):
```

```
每子任务独立 _run_one:
```

```
Mapper → Composer → Executor (无 Adapter, 并行步无依赖)
```

```
as_completed 收集结果 → pm.mark_done / mark_error
```

并行步之间无前步输出传递 (无 Adapter)，各步独立执行后汇总。

四、三层 loop 架构

4.1 retry loop — exec_recipe 内部 (atoms.py:576)

```
exec_recipe(way_id, wc, user_input, chat):  
    for attempt in range(max_retry + 1): # ← retry loop  
        ctx = AtomCtx(user_input, cfg, chat, attempt)  
        GENERATORS[recipe.generate](ctx) # LLM 填空生成  
        for pp in recipe.postprocess:  
            POSTPROCESSORS[pp](ctx) # PY 后处理  
        VALIDATORS[recipe.validate](ctx) # PY 校验  
        if ctx.valid or not recipe.retry:  
            wr.filled = _filled_for(...) # 通过 → 取结果  
            break  
        else:  
            wr.exhausted = True # 耗尽 → 取最后一次  
        for ob in recipe.observe:  
            OBSERVERS[ob](ctx, wr, attempts) # PY 观测
```

retry 语义：校验不通过时重试 LLM 生成（最多 max_retry 次），不是无限循环。每次重试 LLM 看到相同的 prompt，靠概率性生成尝试不同输出。recipe.retry=False 的组合（如 deterministic_pin）不重试——代码钉死后一次定论。

4.2 Adapter loop — 步骤间适配 (adapter.py:20)

```
Adapter.adapt(prev_output, next_tool, pm):  
    if next_tool.can_accept(prev_output.keys()):  
        return prev_output # ← 直通（贯穿直通点）  
    else:
```

```
# ← loop 回 Mapper 选适配组合
combo, config = mapper.map(next_tool, adapt_subtask, pm)
result = composer.compose(combo.id, prev_text, config)
return adapted_input
```

Adapter loop 语义：上一步输出不能直通下一步时，loop 回 Mapper 选一个适配组合，用 Composer 把上一步输出转换为下一步工具所需的输入格式。这是"递归自举"——编排器自身的前置规范也来自 14 种组合。

4.3 串行 loop — 子任务遍历 (orchestrator.py:111)

```
_run_serial(subtasks, pm):
    for i, st in enumerate(subtasks):          # ← 串行 loop
        adapted = adapter.adapt(prev_output, tool, pm)
        combo, config = mapper.map(tool, st["desc"], pm)
        comp_result = composer.compose(combo.id, st["desc"], config)
        exec_result = executor.execute(tool_name, exec_input, pm)
        prev_output = exec_result              # 输出回传下一步
```

串行 loop 语义：子任务依次执行，每步输出经 Adapter 适配后传下一步。类似编程的 `for` 循环（次数固定=子任务数），轮间串行（依赖上一轮输出）。

五、前置规范 — 14 种穷举组合

5.1 组合分类

分类	编号	名称	PY 范式	场景
基础方式	1	pure_guide	LLM生成→PY校验(约束)	开放生成、续写、摘要
	2	diverge_correct	LLM生成→PY后处理(正则)→PY校验(纠偏目标)	创意生成、文案、扩写
	3	deterministic_pin	LLM生成→PY后处理(钉死)→PY校验(封死目标)	格式固定、编号重排

分类	编号	名称	PY 范式	场景
	4	detect_report	LLM生成→PY后处理(检出)→PY校验 (有检出即成功)	数值核查、事实核查
值域限定	5	enum_select	LLM生成(选词)→PY校验(集合内)	分类、标注、情绪判断
	6	condense_enum	LLM生成(凝练)→PY后处理(过滤)→PY校验(无编造)	标签凝练、主题提取
	7	slot_extract	LLM生成(填槽)→PY后处理(解析)→PY校验(无多余key)	信息提取、结构化
	8	required_min	LLM生成(填槽)→PY后处理(解析)→PY校验(必填齐全)	表单填写、必填校验
复合后处理	9	diverge_detect	LLM生成→PY后处理(纠偏+检出)→PY校验(有检出)	创意+核查
	10	diverge_condense	LLM生成→PY后处理(纠偏+过滤)→PY校验(无编造)	创意+标签
	11	detect_condense	LLM生成→PY后处理(检出+过滤)→PY校验(无编造)	核查+标签
精确校验生成	12	range_bound_gen	LLM生成(填槽)→PY后处理(解析)→PY校验(区间内)	数值校验、范围检查
	13	exact_match_gen	LLM生成(填槽)→PY后处理(解析)→PY校验(精确相等)	精确提取、固定值
	14	enum_filter_fabricate	LLM生成(选词)→PY后处理(过滤)→PY校验(无编造)	分类+防编造

穷举完备性：当前原子库（3 生成 + 4 后处理 + 10 校验 + 6 观测）下穷举完，不存在第 15 种。每个组合 = Recipe（PY 确定性查表）+ output_limit + 场景标签。

5.2 Recipe 结构

```
@dataclass
class Recipe:
    generate: str = "text"           # 生成原子: text / select / slot
    generate_arg: str = ""           # 生成参数: extra_check / required_min
    postprocess: list = []           # 后处理原子链: [deterministic,
enum_filter, ...]
    validate: str = "none"           # 校验原子: in_set / no_extra /
required_full / ...
    retry: bool = True               # 是否重试
    observe: list = []               # 观测原子: [hit, fabricated, extra_keys,
...]
```

执行顺序：生成 → 后处理（链式）→ 校验 → 观测。PY 部分确定性查表，只有生成那步是 LLM 概率填空。

5.3 前置规范 vs 后置验证

维度	前置规范（本系统）	后置验证（传统）
约束时机	生成前钉死格式/值域/槽位	生成后检查再修
LLM 自由度	填空（槽位已定义）	自由生成后校验
失败处理	retry loop（重新填空）	后处理修复
确定性	PY 后处理 + 校验	LLM 自修复

六、原子库（atoms.py）

复用自 silprespec-emulator，为编排器提供原子化执行能力。

6.1 五类原子

类别	原子	数量	说明
生成	text / select / slot	3	LLM 填空生成（自由文本 / 候选词选择 / 槽位填充）
后处理	deterministic / enum_filter / detect_report / json_parse	4	PY 确定性后处理（正则替换 / 枚举过滤 / 检出上报 / JSON 解析）
校验	in_set / no_extra / required_full / in_range / eq_exact / guide / diverge / deterministic / detect_report / none	10	PY 确定性校验
控制流	retry	1	校验不通过则重试
观测	hit / fabricated / extra_keys / left_empty / flagged / changed	6	PY 确定性观测指标

6.2 通用执行器（exec_recipe）

```
exec_recipe(way_id, wc, user_input, chat):
    recipe = recipe_for(way_id) 或 wc.recipe (custom)
    for attempt in range(max_retry + 1):
        ctx = AtomCtx(...)
        GENERATORS[recipe.generate](ctx)          # 生成
        for pp in recipe.postprocess:
            POSTPROCESSORS[pp](ctx)                # 后处理链
        VALIDATORS[recipe.validate](ctx)          # 校验
        if ctx.valid or not recipe.retry: break
    for ob in recipe.observe:
        OBSERVERS[ob](ctx, wr, attempts)          # 观测
    return WayResult(filled, success, metrics)
```

七、工具接口规范 (tool_registry.py)

7.1 三层泛化结构

ToolSpec (工具契约)

```
└─ input_fields[] → FieldSpec (每个输入字段)
    └─ name / type / required / default / description / example / options
└─ output_fields[] → FieldSpec (每个输出字段)
└─ examples[] → ExampleSpec (引导示例)
    └─ title / input / output / explanation
└─ internal_prespec[] → 内部前置规范链 (如 路由 → 检索 → 重排序 → NLI → 生成)
└─ capabilities[] → 能力标签
└─ limitations[] → 局限标签
```

7.2 已注册三智能体

工具	端口	接口	内部前置规范
rag-assistant	8767	/api/kb/query	路由 → 向量检索 → 重排序 → NLI验证 → 提示词模板
structured-writer	8770	/api/write	模板选择 → 大纲规划 → 逐段生成 → 引用后处理 → RAG素材检索
silprespec-emulator	8789	/api/emulate	原子库(10原子) → Recipe配方 → 14种穷举组合 → 验证指标

7.3 直通判定 (can_accept)

```
def can_accept(self, available_keys: list) -> bool:
    required = [f.name for f in self.input_fields if f.required]
    return all(r in available_keys for r in required)
```

上一步输出的 key 包含下一步工具的所有必填字段 → 直通；否则 → Adapter loop 回 Mapper 选适配组合。

八、组件详解

8.1 Mapper — 选组合 + 设参 (mapper.py)

```
Mapper.map(tool, subtask, progress_map):
```

① 选组合 (LLM 填空 + PY 查表验证)

```
prompt = "为子任务选最合适的前置规范组合（只输出编号 1-14）"
```

```
out = llm.chat(prompt, max_tokens=10, temperature=0.1)
```

→ 提取数字 → get_combo(id) → PY 查表验证存在

→ 失败回退为 combo[0] (pure_guide)

② 设参 + output_limit

```
config = default_config(combo.way_id)
```

合并 combo.output_limit 和配置的 output_limit_cfg

→ max_length → 写入 output_constraints

→ max_fields → 截断 slots

LLM 只填空选编号，不决定流程。PY 查表验证编号合法，失败回退为最安全的纯引导。

8.2 Composer — PY 确定性组合 (composer.py)

```
Composer.compose(combo_id, user_input, config):  
    combo = get_combo(combo_id)          # PY 查表  
    wc = WayConfig(way=combo.way_id, config=config)  
    result = exec_recipe(combo.way_id, wc, user_input, chat)  
    return result.to_dict()
```

零 LLM 决策。Composer 只做 PY 查表 + 调 exec_recipe。LLM 填空发生在 exec_recipe 内部的生成原子。

8.3 Executor — LLM 填空 + 调智能体 (executor.py)

```
Executor.execute(tool_name, input_data, progress_map):
```

① 构造提示词

```
prompt = 进度地图 + 目标工具 + 工具说明 + 输入要求 + 输出格式 + 当前输入数据
```

```
→ "请生成调用该工具所需的输入参数 (JSON) "
```

② LLM 填空生成工具输入

```
llm_out = llm.chat(prompt, max_tokens=800, temperature=0.3)
```

```
tool_input = parse_json(llm_out) # 提取 JSON
```

③ HTTP 调智能体 API

```
POST tool.url + tool.endpoint → 返回工具输出
```

8.4 Adapter — 步骤间适配 (adapter.py)

```
Adapter.adapt(prev_output, next_tool, progress_map):  
    if next_tool.can_accept(prev_output.keys()):  
        return prev_output                # ← 贯穿直通  
    else:  
        # ← loop 回 Mapper  
        adapt_subtask = "把上一步输出适配为 {tool} 所需的输入格式"  
        combo, config = mapper.map(next_tool, adapt_subtask, pm)  
        result = composer.compose(combo.id, prev_text, config)  
        return adapted_input
```

贯穿直通点：上一步输出的 key $\supseteq$ 下一步工具必填字段时，直接传递，零 LLM 调用。否则 loop 回 Mapper 选适配组合，用 Composer 执行格式转换。

九、进度地图 (progress_map.py)

9.1 输入分类 (穷举 6 类别)

类别	说明
extract	信息提取 (实体/关系/属性)
generate	内容生成 (文案/文章/摘要)
analyze	分析推理 (比较/评估)
verify	数据核查 (数值/事实/合规)
transform	格式转换 (重构/翻译)
orchestrate	多步编排 (复杂任务)

LLM 填空分类 (只输出类别 id) → PY 查表映射。失败回退为 orchestrate。

9.2 编排模式 (穷举 3 模式)

模式	触发条件	说明
serial	num_steps ≤ 1 / generate / analyze / 默认	串行，前步输出传下一步
parallel	extract / transform / verify	并行，ThreadPoolExecutor
loop	(预留)	循环，每轮结果回传

9.3 ProgressMap

```
@dataclass
class ProgressMap:
    user_input: str          # 完整用户初始输入 (贯穿全局)
    category: str            # 输入分类
    orchestration_mode: str  # 编排模式
    steps: list[StepStatus]  # 每步状态
    completed: list          # 已完成步骤
```

每步 LLM 都通过 `progress_map.summary()` 看到全局进度，不盲目执行。

十、Web UI 架构

10.1 HTTP 服务器

基于 Python 标准库 `http.server`，零外部依赖。端口 8789。

路由	方法	功能
<code>/</code>	GET	主页面 (<code>static/index.html</code>)
<code>/static/*</code>	GET	静态资源 (CSS/JS)
<code>/api/combos</code>	GET	列出 14 种组合
<code>/api/tools</code>	GET	列出已注册工具 (含完整 ToolSpec)
<code>/api/config</code>	GET	获取配置
<code>/api/llm/models</code>	GET	列出 LLM 可用模型
<code>/api/llm/test</code>	GET	测试 LLM 连接
<code>/api/config</code>	POST	保存配置到文件 + 更新内存
<code>/api/run</code>	POST	执行编排

10.2 四 Tab 布局

Tab	功能
编排	任务输入 + 工具勾选 + 执行编排 + 结果展示
组合	14 种穷举组合表格 (编号/名称/描述/方式/场景/输出限制) + 点击查看 Recipe
工具	三智能体完整接口契约 (输入字段/输出字段/引导示例/能力/局限/内部前置规范)
配置	LLM 后端选择 / 模型下拉 / 测试连接 / 超时+MaxTokens / 编排参数 / 保存配置

10.3 样式统一

深色 Tab 栏 #2d2d44 + 主色 #667eea，与 Orchestrator/rag-assistant/structured-writer 统一。

十一、LLM 客户端 (llm_client.py)

基于纯 urllib 的 OpenAI 兼容客户端，支持多后端：

后端	地址	模型发现
LM Studio	http://localhost:1234/v1	/v1/models
Ollama	http://localhost:11434	/api/tags
自定义	任意 OpenAI 兼容	/v1/models

统一接口： chat / chat_detailed / test_connection / list_models （30 秒缓存）。

十二、配置系统

12.1 配置文件 (config.json)

```
{
  "llm": {
    "backend": "lm-studio",
    "base_url": "http://localhost:1234",
    "model": "qwen/qwen3.6-35b-a3b",
    "api_key": "not-needed",
    "timeout": 180,
    "max_tokens": 4096,
    "temperature": 0.7
  },
  "orchestrator": {
    "max_steps": 20,
    "max_retry": 3,
    "verbose": true,
    "output_limit": {
```

```

    "soft_guide_max_length": 500,
    "diverge_correct_max_length": 800,
    "slot_extract_max_fields": 5
  }
},
"combos": {
  "default_output_limit": { "max_length": 500, "max_fields": 5 }
}
}

```

12.2 配置优先级

CLI 参数 > config.json > DEFAULT_CONFIG (main.py 常量)

12.3 配置持久化链路

界面保存 → POST /api/config → 合并到 server.config → 落盘 config.json

→ 下次启动 load_config 读取 → make_llm 从配置构造 LLMClient

工具配置由 tool_registry.py 的 _init_default_tools() 硬编码注册（含完整 FieldSpec/ExampleSpec），不从 config.json 读取。

十三、CLI 接口

python main.py --web	# Web UI (主模式, 默认 8789)
python main.py --query "分析这份报告"	# 单次任务执行
python main.py --check	# 检测 LLM 连接
python main.py --list-combos	# 列出 14 种穷举组合
python main.py --list-tools	# 列出已注册工具
python main.py --backend ollama	# 切换 Ollama 后端
python main.py --port 8789	# 指定端口

十四、数据流总览

用户输入 "分析茅台酒工艺并写报告"

```
└─ classify_input → ("analyze", "分析推理")
└─ _decompose → [
```

```
{"name": "查询工艺", "desc": "茅台酒制作工艺", "tool": "rag-assistant"},
```

```
{"name": "写报告", "desc": "基于工艺写分析报告", "tool": "structured-writer"}]
```

```
└─ select_orchestration_mode → "serial"
└─ ProgressMap(user_input, "analyze", "serial", 2 steps)
```

```
└─ Step 0: 查询工艺
  └─ Adapter: 无前步 → input_data = {"query": "茅台酒制作工艺"}
  └─ Mapper: LLM 选组合[1] pure_guide → config = {guide_prompt, output_constraints}
  └─ Composer: exec_recipe("pure_guide", ...) → LLM 填空生成 → PY 校验
  └─ Executor: LLM 填空生成工具输入 → POST rag-assistant:8767/api/kb/query
  └─ prev_output = {answer, docs, summary, sources}
```

```
└─ Step 1: 写报告
  └─ Adapter: can_accept? {answer, docs, ...} ⊇ {topic, template}?
```

→ 不能直通 → loop 回 Mapper 选适配组合

→ Composer 把 {answer, docs} 适配为 {topic, template, material}

```
└─ Mapper: LLM 选组合[1] pure_guide → config
└─ Composer: exec_recipe → LLM 填空生成
└─ Executor: LLM 填空生成工具输入 → POST structured-writer:8770/api/write
└─ prev_output = {article, outline, references}
```

└─ _summarize → 编排结果 + 每步状态 + 组合标签 + 工具标签

09 | 技能编排器到 agent 编排器——编排对象迁移与“圈”的 z 轴

一、两篇并列文档之间，空着一条变化线

架构 06 与架构 07 是两篇并列的系统文档：各自的系统概览、机制拆解、边界声明、运行速查齐备，互不引用对方的章节——只在各自的第一节放了一张"与上一代的关键差异"对比表。导读为它们补了代际定位：第 I 代 orchestrator (v2.8.1)、第 II 代 silprespec-orchestrator (v0.1.0)，并各自标注实验性。

但代际定位不是演进论述。两篇并列文档回答的是"各自是什么"，导读回答的是"各是什么代"——**没有人回答"为什么会有第二代、这一步改了什么、什么没有变"**。这条线恰好是理解两位编排器最有价值的部分：

- Orchestrator 的圈：**纯粹人工编排**——技能链由人写死，LLM 只做两头，中间确定性执行；
- silprespec 的圈：**表象上 LLM 自主编排**——它已经足够 ReAct 的样子：每步 LLM 在场、有循环、会调用工具、有进度记录；但它绝不是 ReAct——仍是有限枚举，看似决策让 LLM 做，实则决策依旧在人这里。同一个圈，在接近人的 z 轴上整圈上移了一个坐标——圈还是那个圈，只是位置不同。

本篇文章的命题就是这句话的展开：**先看清迁移了什么，再拆穿表象逼近 ReAct 的错觉，最后把 z 轴画出来——圈的位置变了，圈的性质没变。**

二、迁移了什么：对象、通道与 LLM 的在场位置

两代编排器的差异可以收敛到一张迁移表。表中每一项都不是孤立的技术选型，而是**编排对象换了之后被迫跟着换的东西**。

迁移维度	Orchestrator（第 I 代，v2.8.1）	silprespec（第 II 代，v0.1.0）	变化性质
编排对象	技能：SKILL.md 声明 + scripts 脚本的扩展单元	智能体：独立 HTTP 服务（自带 Web UI 与外部 API）	对象的物理形态换了

迁移维度	Orchestrator (第 I 代, v2.8.1)	silprespec (第 II 代, v0.1.0)	变化性质
调用通道	subprocess 确定性执行技能脚本	HTTP POST 调智能体 API	通道从进程内到网络
LLM 在场位置	只做两头 (前处理 + 输出整理), 中间全程不在场	每步在场 (读任务、选组合、填工具输入、判进度), 但只填空	从"两端胶水"到"全程旁站"
循环形态	for 真循环 (次数固定, 无嵌套)	三层嵌套有界循环 (retry → adapter → serial)	从平面循环到嵌套循环
决策空间	人写死技能链 (无链即拒绝)	人穷举 14 种原子化组合 + 六类输入分类, PY 确定性查表	决策空间形式不同, 来源相同: 都是人事先钉死的
编排动作的承担者	人 (在系统外定义链)	LLM (在系统内驱动每一步)	最关键的一行, 见第四章

迁移的因果链是单向的: 因为编排对象从"脚本单元"变成了"有内部流程的智能体", 编排器就再也不能像 Orchestrator 那样把对象当黑盒脚本整段执行——智能体有自己的状态、自己的 API、自己的内部流程, 调用它之前需要先想清楚"这一步该调谁、输入怎么拼、返回怎么接"。于是原本由人承担的编排动作 (选路、拼参、判进度), 开始往系统内迁移: Orchestrator 时代这些动作发生在人的大脑里 (人工编排), silprespec 时代这些动作被搬进了系统 (LLM 承担执行)。

这一步迁移就是本篇文章的观察起点。它带来的直接后果, 是 silprespec 在外观上长成了一个"会自己编排的 agent"——下一章拆这个表象。

三、表象逼近 ReAct: silprespec 的"自主"长什么样

先承认表象。silprespec 的运行时外观与一个 ReAct agent 高度相似:

- **每步 LLM 在场:** Mapper 读用户任务与 ProgressMap, 决定"这一步选哪种组合"——对应 ReAct 的 Thought;
- **会调工具:** Executor 由 LLM 填空生成工具输入, POST 给 rag-assistant / structured-writer 这类智能体, 拿回结果继续——对应 ReAct 的 Action / Observation;
- **有循环:** can_accept 判据不过就回到循环起点, Adapter 负责把上一步输出适配成下一步输入, serial 层遍历子任务——对应 ReAct 的循环思考-行动。这里的循环不是让 LLM 空转一圈: 每次回到起点都是一次新的决策 (重新选组合、重新拼输入), 逐句发生、逐句生效;

- **有进度记忆**：ProgressMap 记录每步状态、组合标签、工具标签——对应 ReAct 的上下文记忆。

把这四样拍成截图给一个没读过实现的人看，他大概率会说"这就是一个自主 agent"。**"够了 ReAct 的样子"这句话是成立的，而且不是偶然**——设计上它就是要让 LLM 承担编排动作，动作承担者的拟人化必然带来外观的拟人化。

但表象到这里就结束了。把镜头拉进实现层，四样东西的**内核全部不是 ReAct**：

ReAct 的样子	silprespec 的实现	差异的本质
Thought: LLM 自由推理下一步	Mapper 在 14 种组合里选编号，编号进 PY 查表校验，失败回退 combo[0]	LLM 的"想"发生在枚举结果集内部，且被确定性查表拦截
Action: LLM 自由决定调用什么	组合定义里工具与输入槽位由人预先钉死，LLM 只填空	动作空间封闭，不存在未定义的调用分支
循环：无界直到任务完成	三层循环每一层都有边界：retry 上限、can_accept 直通判据、serial 子任务数	循环有界且边界全部是人事先写的规则
记忆：自由拼接上下文	ProgressMap 只记录固定槽位（每步状态/组合/工具），LLM 无权改写	记忆是结构化账本，不是自由发挥的上下文

母书 04/05 移除 ReAct 的理由在这里依然成立，一个字节都没被推翻：**LLM 的路径决策不可复现**。ReAct 的危险不在于"有循环、有工具调用、LLM 在场"这些可见特征——这些特征 silprespec 全都复刻了；ReAct 的危险在于**把路径决策权真的交给了 LLM**（下一步做什么、调用什么、何时终止，全部由概率决定）。silprespec 复刻了特征、没有复刻权力。它把 ReAct 的表皮穿在了枚举内核外面。

于是问题来了：如果内核根本没变，为什么还要大费周章让 LLM 承担编排动作？直接像 Orchestrator 那样人工编排不好吗？答案在第四章——这正是 z 轴的意义。

四、"圈"的 z 轴：人工编排到 LLM 自主编排的表象

进入 z 轴之前，先把三组说法对齐——它们不是文字洁癖，而是理解本篇的关键（提出语境见引用 [11]）：

- **loop 一律说"循环"**。本文不写英文 loop：循环就是 ReAct 那个"思考-行动"的往复结构，说循环即指它；第二章迁移表里的"三层嵌套有界循环"说的是实现内的嵌套结构，与 z 轴无关——z 轴不是把嵌套立体化成"套圈"，那是平面内的循环嵌套，不是轴上的叠加。
- **不说"自由路径决策"，说"自主编排"**。路径根本不自由：能选的下一步只有枚举结果集里那几种（14 组合查表）。"自主"修饰的是动作承担者的表象——LLM 以拟人姿态驱动每一步——不是决策

空间的开合。自由路径决策这个说法把表象误读成了权力，自主编排不会。

- **不说"填槽范式"，说"填槽即决策（逐句）"**。LLM 每填一个槽位（选组合编号、拼工具输入、判进度），都是一次决策——尽管它发生在圈内、被查表与边界约束。填槽不是机械填空，是逐句的、被约束的决策；循环的每一圈都产生这样的决策，不是让 LLM 空转一圈。这也是本篇把 silprespec 称为"表象自主"而非"假自主"的原因：圈内的每一句都在决策，只是决策权从未出圈。

对齐说法后，再立下 z 轴的正解：**z 轴上的移动是同一个圈整体平移——圈还是那个圈（圆心真决策、周长枚举、边界回退终止全在人），只是沿拟人自主轴从"人的桌面"上移一格坐标，落到"LLM 承担动作表象"的高度**。不是轴上另画一个圈，更是一圈套一圈往上叠——套圈是平面内循环嵌套的误读；z 轴只有一个圈，位置不同而已（提出语境的原始表述见引用 [11]）。

把两代编排器放到同一条轴上观察。这条轴不是"编排对象"（技能 → 智能体，那是平面上的内容迁移，第二章已讲），而是**编排动作的承担者在拟人尺度上的位置**——谁在扮演"编排者"这个角色、看起来像谁在编排。姑且称它为 z 轴：下端是机器按人写死的脚本执行（编排动作完全在系统外的人手里），上端是机器像人一样自主编排（编排动作在系统内，由 LLM 以思考-行动-循环的姿态承担）。

Orchestrator 的圈画在 z 轴的下端。它的编排是纯粹人工编排：技能链、每步参数模板、分支与回退，全部由人写在配置与代码里；运行时 LLM 只做前后两头，中间没有任何编排动作发生在系统内。这个"圈"（决定跑什么、按什么顺序跑、失败怎么办的那个控制闭环）**甚至不在系统里**——它在人的大脑里，人画好圈，机器沿着圈跑。

silprespec 的圈沿 z 轴上行。它的编排动作被搬进了系统：LLM 读任务、选组合、拼输入、判直通、定下一跳——系统内部长出了一个"会编排的角色"，以拟人的姿态承担了原本由人承担的编排动作。于是这个圈在 z 轴上往**接近人的方向**抬升了一段：机器不再只是"被编排"，它开始"表现得像在编排"。这就是"圈有了 z 轴坐标"的含义——同一个圈，从"人工编排"（圈在系统外，由人直接持有）移动到了"LLM 自主编排的表象"（圈进入系统，由 LLM 以拟人方式执行）。

但它是同一个圈。z 轴上行的只是"动作承担者的表象"，不是"决策权的归属"。圈的三样东西——圆心、周长、边界——全部没有移动：

- **圆心（真决策）在人**：14 种组合是人穷举写死的（combo_registry 在代码里）、六类输入分类是人定的、serial/adapter/retry 的触发规则是人写的。LLM 的"自主编排"全部发生在圈内：选编号 → 查表验证 → 填槽位。
- **周长（动作空间）封闭**：LLM 能"想"的下一步只有枚举结果集里那几种，不存在圈外的路径。
- **边界（终止与回退）由人钉死**：retry 上限、can_accept 判据、失败回退 combo[0]，全部是确定性规则，不依赖 LLM 自律。

所以"表象逼近 ReAct"与"内核仍是枚举"在 z 轴上是**同时为真**的两句话：可见特征（LLM 在场、循环、调工具）把圈抬到了 ReAct 的可见高度，控制边界（枚举、查表、人定规则）把圈钉在原地。z 轴上行是有界的——它逼近了 ReAct 的外观，但没有越过那条临界线：**决策是否让渡**。ReAct 的圈是 LLM 自己画的

(路径无界、决策真交给 LLM)；silprespec 的圈是人画好、LLM 在圈内走。前者 z 无上限，后者 z 有上限——上限就是"人的桌面"，圈从未离开过它。

这个观察顺带给出了一个比特征清单更可靠的判据：**判断一个系统是不是"真 agent 化"，不看它有没有循环、有没有工具调用、LLM 是否每步在场——这些特征 silprespec 全有；要看编排的圈画在 z 轴的哪一侧：决策让渡了没有。**特征可以复刻，决策让渡不可伪装——伪装出来的（silprespec 这类）就是编排器长得像 agent，真让渡的才是 agent 该有的样子。母书 08a 把 Orchestrator 称为"伪 agent"，论断依据的正是分工先于名分：先看决策在哪一侧，再谈它像不像。

五、这条变化线在母书中的位置

母书（已完结，不再增篇）的方法论正文里，这条变化线散落在两侧、没有被连起来：

母书篇目	覆盖的命题	与本篇的关系
母书 04 《技能流水线编排范式》	技能编排：LLM 当胶水，路径决策不可复现 → 移除 ReAct	Orchestrator 的方法论源头（技能侧）
母书 05 《链驱动 Pipeline 编排范式》	编排智能体的方法论沉淀：链驱动、固化 vs 调用	Orchestrator 的母篇；本篇的编排侧上游
母书 08 《回到当下——有限决策范式的三体印证》	决策在人、LLM 填空；让 LLM 做判断题而非决策题	silprespec 的总命题（决策在人）
母书 08a 《前置规范大于后置验证》	最小尝试次数、分工与监督、伪 agent 名分论	silprespec"14 组合事前穷举"的范式源头
母书 08b 《填空的边界》	空的收束、U 型注意力、验证的分工	silprespec 的 Mapper/Composer 填空执行
母书 08c 《槽位的减法》	三种场景的幻觉治理路径	silprespec 输入槽位的收束设计
母书 09b 《配置推动的穷举一致性》	统一推动点位	14 种组合由配置统一驱动

母书 04/05 站在技能编排这一侧批判 ReAct；母书 08 系站在智能体协作这一侧立有限决策。但**"从编排技能跨到编排智能体"这一步本身，母书没有任何一篇以它为命题——04/05 与 08 系之间的编排水位差是空的。**这正是本篇在架构册存在的位置：它不新增方法论（母书已完结），而是把母书两侧的命题在编排层

做一次合流印证——silprespec 是母书 08 系"有限决策范式"长到编排层的实例：事前穷举（08a 前置规范）、LLM 只填空（08b 填空的边界）、槽位收束（08c 槽位的减法）、组合由配置驱动（09b 配置推动），外加母书 05 的编排方法论，五者在一套系统里同时成立，且互为约束。

由此本篇文章与母书的对应关系也清楚了：**架构 08 = 母书 05（编排方法论） × 母书 08 系（有限决策/前置规范）在"编排对象迁移"命题上的实证篇**。母书若出后续版本，本篇沉淀的 z 轴判据（看决策让渡，不看特征清单）是可供收录的方法论候选。

六、结语：变的是对象与通道，不变的是枚举内核与决策在人

从 Orchestrator 到 silprespec，三句话可以收束整条变化线：

- 1. **迁移的是编排对象与调用通道**：subprocess 跑技能脚本 → HTTP POST 调智能体 API；连带 LLM 的在场位置从两头挪到全程、循环从平面到嵌套——这些都是对象换了之后被迫跟着换的。
- 2. **表象沿 z 轴上行**：编排动作的承担者从"系统外的人"内化为"系统内以拟人姿态驱动的 LLM"，圈在接近人的方向上抬升——它足够 ReAct 的样子。
- 3. **内核原地不动**：枚举空间封闭、查表验证、回退规则、终止边界全部由人钉死；LLM 的"编排"是圈内填空，真决策依旧在人——它绝不是 ReAct。

变化线的终点不是"下一个编排器"，而是一个可以反复使用的判据：**看一个系统是否真的把决策交给了 LLM，不要数它有几个循环、调不调工具、LLM 在不在场——要看编排的圈画在 z 轴的哪一侧**。特征会一代比一代更像 ReAct，但"决策在人"这条线只要还在，它就还是编排器；哪天这条线被跨过，才是真正的新物种。Orchestrator 与 silprespec 都还站在线的这一侧——这也是它们共同标注实验性的原因：方向已经探明，端到端的验证仍在进行，若实验收敛，本篇结论将随两篇系统文档一起更新。

编号	引用	出处
[1]	Orchestrator 架构文档（v2.8.1）：链驱动技能编排器，人工编排 Pipeline，LLM 只做两头，subprocess 确定性执行	架构册 架构 06（仓库根 orchestrator-architecture.md）
[2]	silprespec-orchestrator 架构文档（v0.1.0）：前置规范编排器，14 种穷举原子化组合，PY 确定性查表，LLM 填空执行	架构册 架构 07（仓库根 silprespec-orchestrator-architecture.md）
[3]	母书 05：编排智能体（Orchestrator）构建的方法论沉淀——链驱动 Pipeline 编排范式	母书《我思故我写》
[4]	母书 04：重型技能构建的方法论沉淀——技能流水线编排范式	母书《我思故我写》

编号	引用	出处
[5]	母书 08：智能体协作构建的方法论沉淀——回到当下：有限决策范式的三体印证	母书《我思故我写》
[6]	母书 08a：前置规范大于后置验证——最小尝试次数的分工与监督范式	母书《我思故我写》
[7]	母书 08b：填空的边界——空的收束、U 型注意力与验证的分工	母书《我思故我写》
[8]	母书 08c：槽位的减法——三种场景的幻觉治理路径与四个论断	母书《我思故我写》
[9]	母书 09b：配置推动的穷举一致性——统一推动点位与误差的哲学	母书《我思故我写》
[10]	架构册导读：第 I/II 代定位、实验性声明、"母书如出后续版本将收录方法论结论"	架构册 book/frontmatter/00_导读.md
[11]	原始对话：z 轴正解与三组术语的提出语境——"同一个圈沿轴往上，圈还是那个圈，位置不同了"（非轴上套圈）；loop→循环、自由路径决策→自主编排、填槽范式→填槽即决策（逐句）；循环每圈都是决策、非让 LLM 空转一圈	作者与 LLM 的结构辨析对话（2026-09，本篇第四章词汇对照与 z 轴正解的原始出处）

说明：[1][2] 为本篇的两代实证标本（系统文档，可核验实现细节）；[3]-[9] 为母书方法论引用（本篇"决策在人/有限枚举"立论的母书依据）；[10] 为架构册导读中的代际定位与实验性声明的出处；[11] 为 z 轴正解与三组术语的原始对话（loop→循环、自由路径决策→自主编排、填槽范式→填槽即决策（逐句），"圈还是那个圈、沿轴位置不同"）。无外部文献。

延伸阅读

- **架构 06 • Orchestrator 架构文档**：第 I 代标本的系统文档——本篇文章"人工编排、LLM 只做两头"的论断以其实现为实证。
- **架构 07 • silprespec-orchestrator 架构文档**：第 II 代标本的系统文档——"14 组合穷举、三层有界循环、LLM 只填空"的机制细节见其正文。
- **架构册导读**：两代定位（第 I/II 代）与实验性声明的原始出处；"两代编排器调的对象不同"的引子

论述。

- **母书 05（链驱动 Pipeline 编排范式）**：编排方法论侧的上游——本篇文章的编排论述承接其"链驱动、固化 vs 调用"命题。
- **母书 08/08a/08b/08c/09b**：有限决策与前置规范系列——本篇文章"决策在人、枚举封闭、z 轴不越过决策让渡线"的立论依据。

10 | 施工层级——画地图定位、功能解耦与统一各级推动点位

一、引子：蠢问题不发生在结果上，主要来自缺失过程控制

这个病不挑场景，它在一类项目的三个阶段反复出现。

立项的时候，架构决策散落在一次次对话里——为什么这么分模块、哪条公式绑定哪个功能，口头上都定过，白纸上什么都没有；下一次动工，先得考古。动工的时候，缺方案设计这一层——需求直接落到 LLM 手里开干，影响面没有人在动工之前画出来，"先分析什么也不要改"没有变成流程，只变成事后教训。施工的时候，不知道动过了什么——改了哪几处、哪几处必须同步、现在走到哪一步，全靠记忆；项目越大、活得越久，这份记忆越不可靠。维护只是三阶段的延续：改一个功能，心里要把关联模块全部过一遍才敢动手；改完跑全量回归，绿了也不敢确认真的绿。

先把病根说准（以下为经验刻画）。这类"蠢问题"**不发生在结果上**——结果错了测试能抓到一部分，抓不到的那部分也有明确的复现路径可以追。真正的损耗发生在**过程上**：这次改动牵动了谁、影响面画在哪里、改完之后哪几个点位必须同步——这些问题在动手之前没有答案，动手之后没有人核对。它还有两个让常规手段失灵的特点：

- **不一定方便设计 AB 测**。对照实验要求两次施工只有一处不同，而过程失控恰恰是"不知道动了多少处"——变量都数不出来，对照从何谈起。
- **测试能体现的不一定有意义**。测试绿只说明被测路径没坏，不说明改动只发生在被测路径上。

所以主要问题来自**缺失过程控制**——而且是覆盖立项、设计、施工、维护全程的过程控制，这正是母书 08a"前置规范大于后置验证"的全场景形态 [3]：不只写稿与维护，影响面在每一个环节都要封堵在事情发生之前。本篇把这套过程控制立成三根支柱：**画地图定位**（成品之外，一张动态多文件的地图）、**具体功能层的解耦**（成品之中，执行侧）、**统一各级推动点位**（成品之中，调度侧）。三根支柱合起来是一句话：**架构上完全解耦，耦合层完全剥离**。

二、画地图定位：一张动态的、多文件的地图

本章结论先行：画地图是施工层级的第一根支柱，也是三根支柱里唯一**落在成品之外**的一根——它不是一个系统模块，是成品之前、指导工作的那部分；形态是动态的、多文件的，回报是三方对比下的进度管理。

地图的起点：一份施工公约

地图一开始可能只是一份**施工公约**。它至少装着五样东西（经验刻画）：

要素	内容
要实现的功能	这个项目最终要做什么
文件结构	谁放在哪、谁调用谁
目标倾向	优先保什么、可以牺牲什么
技术落地位置	某些功能明确使用哪一条公式，其中各代数都代的是什么
施工偏好	LLM 运行纪律——可回溯、不冒进，是"形式"上的保证

"技术落地位置"的典型是 podcast-maker 的**单位同源**条款：一切字数账走同一把尺（有效字：汉字 1、标点 0.5、西文词 1.5），"不同源的数不许相减"——配额权重用有效字、装箱折 token 用原始字符数，两把尺各管一件事且永不混用 [2]。这条公约一经写成，所有涉及字数账的改动都有一个可以回去查的锚点：谁要是想把 token 折算数混进配额，不用吵，公约说了不许。structured-writer 的 detail 三要素同理：「」引用违规片段、对照、方向，30-120 字，逐字可定位 [1]——检测输出长什么样、长度的量从哪来，都是公约条目。

结构性布局也是公约的一部分：podcast-maker 五步管线的划分、四处唯一定义处的选址、"写作阶段不判句长"的边界声明 [2]——这些是立项期就要立的约，它们决定后面所有施工在哪条轨道上跑。公约缺位的项目，每一次动工都在重新发明架构。

地图的生长：每一次改动方案都落地进来

地图不是一次性文件——它是**动态的、多文件的**，后期每一次改动方案都可以完整落地进地图。

纪律是（经验刻画）：改动点位超过三处，先出方案落盘，确认后一口气推到底——方案里枚举全部改动点位、每处的验证方式、回退路径 [1]。这条纪律在 LLM 协作项目里有一个硬理由：LLM 的默认工作方式是阻力最小路径——没有方案约束时它会抄近路，而抄出来的近路就是过程失控的第一来源。

标准样本是 podcast-maker 接入同步工具链时的一次决策：源目录扫描出五万六千多个文件，全量同步等于灾难；方案把决策写成"仓库 tracked 全集 $\cap$ 扫描清单"，allow 集从五万多收敛到 220 个文件 [2]。这个样本里有两处要点：其一，决策不散落在命令行参数与临时文件里——必经统一入口落盘，事后可回溯；其二，敏感决策内置断言，发现值不在预期集合即中止交人工——**方案在写自己的时候就规定了怎么验证自己**。

方案落地之后，地图与**更新日志**互相作为参照 [1][2]：方案说这次要动哪几处，日志说这几处最后动成了什么样。两者合起来才是"动过了什么"的完整答案。

地图的回报：施工现场 × 施工规划 × 施工记录

地图真正的回报是进度管理，而进度管理收敛为一个动作：**三方对比**。

一方	说的是什么
施工规划	地图说应该是什么样
施工记录	更新日志说动过了什么
施工现场	代码与产物实际是什么样

三方一致，进度可信；有出入，先对账再动刀。这个机制的实证就是本册自身的一次对账：podcast-maker 架构册从 v0.39.0 漂到 v1.4.0，17 个版本的架构级变更靠逐版核对 CHANGELOG 补齐——**漂移被发现的代价是一次对账，而不是一次事故**，这就是地图在起作用 [2]。

掌控还有自动化形态：structured-writer 的版本门禁把"三端版本一致"从人工对账变成机器对账，每次发布自动核验 [1]；podcast-maker 的产能预检在排图之前把"计划期数撑不撑得起"算清楚，撑不起就报最多可排 N 期 [2]——进度问题在方案期暴露，不在竣工后发现。

地图在成品之外：地图不进成品，成品里是它的印痕

到这里可以回答一个容易走偏的问题：既然地图这么重要，为什么看一个已经完工的项目，找不到那张地图？

因为**地图是过程性文档，不进成品**。施工公约与改动方案未必都留在项目目录里；成品里能看到的，是这张地图被执行之后留下的**结构结果**——解耦与统一各级推动点位（第三、四章）。这构成一种**溯源式印证**：看到成品里结构分成两层，能反推它按这套规划方式施工过；但要看到规划本身，得回到成品之外——施工公约、改动方案、更新日志、原始对话。

一句话：**证据在成品里，规划在成品外**。说一个智能体"里面有个规划模块"，是把成品之外的指导性文件错当成了成品之内的系统构件。

三、功能层解耦：每个功能的迭代只属于它自己

本章结论先行：第二根支柱落在成品的执行侧，管"动坏了只坏一处"——解耦之后，每个功能的迭代只属于它自己。

先用一个比喻把目标形态立起来（提出语境见 [7]）：以简化的人体作比，从功能上看，**心脏是泵血的、肺是气体交换的，两者功能层面完全解耦**——心脏不替肺换气，肺不替心脏泵血。脏器之间的耦合不发生在器官本身，耦合发生在副交感神经；统一的推动点位则全在中枢神经。器官管功能，副交感管耦合，中枢管调度——这样修起来就很清楚。

工程上的对应判据是（经验刻画）：**解耦之后，每个功能的迭代只属于它自己**。一个功能可以落后、可以有缺陷、可以是整个系统里最短的那块木板——但它不是系统崩溃的节点，因为它的失败被关在自己的一层里。

podcast-maker 的五步管线是这个判据的正面样板：素材、排图、脚本、合成、出片五步，每一步都有独立的产物落盘、独立的判据、独立的失败方式 [2]——排图失败不产生半份脚本，脚本失败不产生半截音频，失败半径被产物的落盘边界物理地切开了。结构化写作侧同构：出货、检查、门禁三段串行，各自成环、各用各的预算、互不侵占对方的轮次 [1]——一段的失败不烧另一段的额度。

解耦还有一条容易被忽略的推论：**功能层不越界替别的层把关**。podcast-maker 把这条写进了边界声明：写作阶段不判句长——句长是全篇一把尺的量，归门禁判且两个方向都有处方；写作阶段替它把关，代价是把"一条坏句"放大成"整段停摆" [2]。越界把关看起来是"多一层保险"，实际是把别层的失败模式引进自己的层：本层的轮次预算被外层的判据烧掉，失败半径反而扩大。

反例值得单独记，因为它是两套系统事故链里最贵的一段。结构化写作的章级检测曾长期污染章内检测：章内六检的输出里混进了章级问题（"无具体对话"被标成章级 HARD），修复引擎拿着章级问题去章内找落点，找不到就反复空转 [1]。它的病根不是哪个功能写错了，是**两个功能层的判据互相串了通道**——A 层的产物流进了 B 层的输入。修复遵循的正是解耦判据：章内检测只产章内问题（sub 非法直接丢弃），章级判定归全文完结层，通道分开之后两个功能各自迭代互不牵动。

经验刻画：判断解耦是否成立，不看模块画得是否好看，看一句话——**这个功能的迭代，会不会改变另一个功能的行为**。

四、统一各级推动点位：耦合只发生在这一层

本章结论先行：第三根支柱落在成品的调度侧——**统一的推动点位是功能的耦合层与多功能调度层，它不涉及具体功能实现；耦合只发生在这一层**。母书 09b 的命题是"配置推动把偏差钉在可指认的位置上" [4]；三根支柱合起来那句话的后半句"耦合层完全剥离"，说的就是这里：耦合层从功能层里被整个剥出来，只此一处。

唯一定义处：一个真值只有一份

podcast-maker 把这条判据做成了四处"唯一定义处"：`layout.py` 定义项目内全部路径、`config_manager.py` 定义全部配置点位、`duration_model.py` 定义字数与时长唯一的换算系数、

`bins.py` 定义 ffmpeg/ffprobe 唯一的定位——“任何一处要拼路径、读配置或做换算，都从这几个地方取；两处各写一份，就是同一件事有两个真值来源；而这类错误最难查，因为两边看起来都对” [2]。这句话值得展开：唯一定义处防的不是“写错”，是“写对两次”——两份各自都对、但随时间漂移成两把尺，才是最难查的一类错误。v0.47.0 的 ffmpeg 定位事故是标准判例：四个模块各写一遍定位、且都只搜 PATH，一键安装把二进制装进项目 `bin/` 之后四处一个都找不到——安装等于没装 [2]。

全量枚举：一个落点都不漏

唯一定义处与全量枚举是一枚硬币的两面：前者保证一个真值只有一份，后者保证一份真值的所有消费点都被钉住。结构化写作的版本门禁 `check_version.py` 曾只枚举了三个校验点（`__init__`、CHANGELOG、pyproject），README 头部的版本戳不在枚举里——于是三次 bump，戳停在旧版本，门禁照样 PASS [1]。这个判例把推动点位的性质暴露得很干净：**点位体系是一个封闭枚举，枚举漏掉一个落点，偏差就在那个落点过**；修复不是加强检查力度，是把 README 头部补进枚举（有则必须相等、无则忽略、标签形式泛化中英文），让门禁重新覆盖全部落点。

判据与处方调同一个函数

耦合层内部还有一条精度更高的纪律：**判据与处方调同一个函数**。podcast-maker 的并句可行域由 `absorb_span` 单点计算，提示词里写给模型的具体区间与门禁校验的区间出自同一处，“修了 A 撞破 B”的死循环在结构上不可能发生 [2]；音色体检同理，点名修复的候选验收与漂移检测共用同一组三判据，15 个耳标样本 15/15 分离 [2]。反向判例是结构化写作的推理审核：规则路径（“维度不适用必须 PASS”）写在 prompt 里，模型在该走这条规则时抄了示例——**写在文本里的纪律，模型可以不执行；函数只有一份，谁都得调它** [1]。统一推动点位的尽头是：能收成函数的纪律，不留在文字里。

通信规范：功能层与调度层唯一的交互面

解耦与点位如何协作？答案在通信规范上——**功能层与调度层唯一的交互，只存在于通信规范**。这给出了一个排障判据（经验刻画）：单功能全部跑通的情况下，整功能崩溃，只可能是调度有问题——找通信规范、调度顺序、衍生孤点就可以，不必进功能层。

结构化写作的拼合事故是这个判据的教科书判例：前端发 POST、`/api/outputs/merge` 的 handler 从诞生起就用 POST 语义读请求体，**功能层的每一件东西都是对的**——但路由注册把它放进了 GET 表，POST 表里没有这条路由，分发器找不到，返回通用 404。拼合功能自引入起从未成功过，直到被用户第一次点到 [1]。这个判例的三段证据值得抄下来：功能层证据全绿（handler 语义正确）、调用方证据全绿（前端发的就是 POST）、系统证据全红（拼合从未成功）——**证据分布本身就在指认故障层**：两层各自正确而整体不通，故障只能在两者之间的通信规范上。修复一行：把路由挪进 POST 表。

podcast-maker 的进度条事故是同一判据的调度顺序变体：写稿、检查、门禁各段功能全部正常，但进度预算表只认检查与门禁两段的日志前缀——最耗时的写稿段一格刻度都没有，实测轨迹只有四跳、全挤

在收尾几十秒里，观感是"这条进度条从来没动过" [2]。功能没有一个错，错在调度层的预算分配漏了一段。顺带值得记下的是，**进度可见性本身就归调度层管**：五段预算表修齐之后，进度掌控才有了物理载体。两件事合起来给出这条纪律的完整形状：**功能层的证据与调度层的证据必须分开拿**——先确认单功能跑通（单元级），再确认注册、预算、顺序、路由正确（系统级）；两层证据混在一起拿，就会出现"测试全绿但功能就是不好使"的悬案。

五、总览：一张地图在成品之外，两个结构层在成品之中

三根支柱叠起来，是一个 AI 协作项目的完整施工层级。它分成两半——**过程侧（成品之外）与结构侧（成品之中）**。

【成品之外 · 过程侧】画地图定位

施工公约：要实现的功能 · 文件结构 · 目标倾向 · 技术落地位置 · 施工偏好



每一次改动方案完整落地进地图 ↔ 更新日志互为参照



施工现场 × 施工规划 × 施工记录 → 三方对比 = 施工进度管理

【成品之中 · 结构侧】架构本体

执行层（具体功能层解耦）



（每个功能的迭代只属于它自己；最短的木板 ≠ 系统崩溃的节点）



调度层（统一各级推动点位）



（功能的耦合层 + 多功能调度层，不涉及具体功能实现）



单功能全跑通而整体崩溃 → 查通信规范 / 调度顺序 / 衍生孤点

三条关系把两半串成一个整体：其一，**地图给结构侧立约**——施工公约管住每一次字数账改动（单位同源），改动方案在动工之前枚举要动的点位。其二，**结构侧两层各管一个失败维度**——解耦管失败半径，点位管耦合落点。其三，**两条排障路径各归各层**——三方对账有出入，查地图；单功能全绿而整体崩溃，只查调度层的通信规范与调度顺序。

架构册各篇系统文档与论述篇各自实现过结构侧两层的局部，但没有一篇把它当命题。这里把它们收进一张**只含结构两层**的对照表：

架构篇	执行层（功能解耦）	调度层（统一各级推动点位）
01 skill-standardization	六模式职责分离 (audit/create/update/refactor/bump/readonly)	审计入口唯一、R-01~R-26 规则体系自身入审计
02 semantic-split	三管线协同、正则先行	5W2H 提取与约束标注统一
04 rag-assistant	检索 / 重排 / 回答分域，Ranker 后插件点位隔离	前置分类两级收敛、top-N 多 KB 路由
05 structured-writer	章级 / 章内通道分离、六检独立	check_version 四点校验、配置统一源
06 podcast-maker	五步管线独立判据、三段互不侵占	四处唯一定义处、唯一出口六类设定
09 编排器演进	技能层 / 编排器层 / 智能体层各管各的	14 组合穷举查表 (combo_registry 唯一定义处)

规划侧不在表内。它是成品之外的那一半——施工公约与改动方案未必都留在项目目录里，只能靠溯源看到；表内能看到的，是这些规划被执行之后留下的结构结果。所以这张表的读法是**溯源式印证**：不是"这些系统里各自有个规划"，而是"这些系统的结构，反证了它们按这套规划方式施工过"。

与母书的关系也在这张表里显形：08a 立了"前置规范大于后置验证" [3]，09b 立了"统一推动点位" [4]——本篇不是新增方法论，是把这两个命题在三根支柱上做一次合流印证：08a 长成画地图（影响面在动工之前落盘），09b 长成调度层（偏差钉在可指认的位置）。架构 09 在编排层做过一次合流（有限决策长到编排层 [5]），本篇是第二次：过程纪律长到全流程。

六、结语：逻辑和纪律是给 LLM 的，不靠它的八爪鱼推理

三句话收束整条治理线：

- 1. **画地图定位管"知道自己在动什么"**——一张动态的、多文件的地图，从一份施工公约长起，每一次改动方案都完整落地进来，与更新日志互为参照；施工现场、施工规划、施工记录三方对比，就是进度管理本身。
- 2. **功能层解耦管"动坏了只坏一处"**——每个功能的迭代只属于它自己，最短的木板可以慢慢换，但它不是系统崩溃的节点。
- 3. **统一各级推动点位管"要动几处才动得齐、出事了往哪查"**——耦合只发生在这一层，真值只有一份、落点全部枚举、判据与处方调同一函数；单功能全绿而整体崩溃时，只查这一层的通信规范与调度顺序。

由此可以得到一个可复用的判据，替代"测试覆盖率"这类结果侧指标：**判断一个项目管得住管不住，看三件事——地图在不在动（公约有没有立、方案有没有落盘、三方能不能对账）、功能层的失败半径有多大、耦合是不是只发生在统一推动层。**三件齐备，"越改越不敢动"不会出现；缺任何一件，相应环节就是冒险。

最后回到这条线的源头。上述全部纪律有一个共同的承担者：它们都是**人写给 LLM 的逻辑和纪律**，而不是指望 LLM 在生成时自己推理出来。LLM 的推理像八爪鱼，触手多、覆盖广，但每条触手都没有骨头——公约、方案、记录、边界、点位就是骨头：LLM 负责在圈内填空，圈由人维护。这与架构 09 的运行判据同源 [5]——决策在人；本篇补上过程侧的另一半：**纪律也在人**。逻辑和纪律一直是**我们**给 LLM 的——给了，它才是可靠的施工者；不给，它只是概率的影子。

引用与出处说明

来源声明：[1][2] 为本篇的两套实证标本（架构册系统文档，可核验实现细节与事故记录）；[3][4] 为母书方法论引用（本篇"前置规范"与"统一推动点位"立论的母书依据）；[5] 为架构册演进收束篇（"决策在人"判据的出处）；[6] 为架构册导读；[7] 为三根支柱与比喻的原始对话。无外部文献。

编号	引用	出处
[1]	structured-writer 架构文档（v3.1.14）： detail 三要素与引文回验、任务地图与决策落盘纪律、check_version 四点校验与 README 戳漂移、章级/章内通道分离、拼合路由 GET/POST 错位判例、few-shot 示例照抄事故	架构册 架构 05（仓库根 structured-writer-architecture.md）

编号	引用	出处
[2]	podcast-maker 架构文档 (v1.4.0): 单位同源公约、五步管线独立判据、四处唯一定义处、唯一出口六类设定、absorb_span 判据与处方同函数、音色体检三判据合验、进度预算五段、产能预检与计划期数、同步 allow 集决策 (tracked 交集)、CHANGELOG 体例收紧	架构册 架构 06 (仓库根 podcast-maker-architecture.md)
[3]	母书 08a: 前置规范大于后置验证——最小尝试次数的分工与监督范式	母书《我思故我写》
[4]	母书 09b: 配置推动的穷举一致性——统一推动点位与误差的哲学	母书《我思故我写》
[5]	架构 09: 技能编排器到 agent 编排器——编排对象迁移与"圈"的 z 轴 (决策让渡判据)	架构册 演进收束篇 (仓库根 技能编排器到 agent编排器——编排对象迁移与圈的z轴.md)
[6]	架构册导读: 方法论 ↔ 架构映射、八套系统的三层关系总览	架构册 book/frontmatter/00_导读.md
[7]	原始对话: 施工层级三根支柱 (画地图定位 / 具体功能层的解耦 / 统一各级推动点位) 与心脏、肺、副交感神经、中枢神经比喻的提出语境——"蠢问题并不发生在结果上, 并不一定方便设计 AB 测, 测试能体现的也不一定有意义, 主要问题来自于缺失过程控制"; "画地图提供的就是施工现场、施工规划、施工记录三对比下的施工进度管理"; "架构上的完全解耦与耦合层的完全剥离"; "逻辑和纪律一直是给 LLM 的, 而不是仅依靠 LLM 自己的八爪鱼推理"	作者与 LLM 的结构辨析对话 (2026-10, 本篇三根支柱与比喻的原始出处)

说明: [1][2] 为项目实证 (本地项目架构文档, 随代码演进); [3]-[6] 为系列自引 (母书与架构册篇目); [7] 为原始对话。正文中工程判据多为经验刻画 (L4, 挂 [1][2] 的具体判例与版本号); "地图在成品之外, 成品里是它的印痕""故障层指认"等方法为本文推导 (L2, 论证链在第二、四章)。

延伸阅读

- **架构 05 • structured-writer 架构文档**：detail 三要素、引文回验、版本门禁与拼合判例的实现细节——本篇执行层与通信规范判据的实证来源。
- **架构 06 • podcast-maker 架构文档**：五步管线、四处唯一定义处、唯一出口、产能预检与音色体检的系统文档——本篇执行层与调度层判据的实证来源。
- **架构 09 • 技能编排器到 agent 编排器**：运行侧的同源判据（决策让渡、圈沿 z 轴上行）——本篇过程侧纪律的运行侧对应物。
- **母书 08a / 09b**：前置规范与统一推动点位的方法论原文——本篇三根支柱里"画地图"与"推动点位"两根的母书依据。

结语：结构是方法论的落点

这本册子收录了什么路

从架构 01 的 skill-standardization（标准化审计引擎），到架构 09/10 的两篇收束——这本册子走完了一条从原则到结构的弧线：

硬约束架构（审计引擎） → 语义拆分（需求到执行块） → 历时估算（WBS 与 EVM）

→ 知识库智能体（RAG 自治） → 结构化写作（五元组与引用管道） → 播客成片（判与写分离的五步管线）

→ 编排器（从技能编排到 agent 编排的收束） → 施工层级（画地图、解耦、点位的过程收束）

回头看，八套系统做的其实是同一件事：**把母书的原则落成可以运行的结构。**

- 递归自举的元验证，落成 skill-standardization 的三级分层与门禁
- 槽位的减法，落成 semantic-split 的三管线与 5W2H 正则
- 前置规范大于后置验证，落成 activity-duration-estimation 的估算方法约束
- 穷举的宿命，落成 rag-assistant 的路由、精排与语义免疫
- 原文锚定与配置驱动，落成 structured-writer 的五元组模板与引用管道
- 判与写分离、唯一定义处，落成 podcast-maker 的五步管线与四处真值源
- 编排器的两代迭代是运行侧的收束——编排对象从技能迁移到 agent，"圈"的 z 轴随之抬升
- 画地图定位、功能层解耦与统一各级推动点位是过程侧的收束——从立项到维护，管得住靠施工层级，不靠更多测试

原则不落地就只是格言。**结构是方法论的落点**——这本册子的全部结论可以压缩成一句话：

母书答"为什么"，本册答"怎么做"：每一条方法论原则，都必须能指到一个真实运行的结构；指不出来的，回炉。

架构文档的特殊性

本册与母书、排版册有一点根本不同：**它随代码走，不随书走**。工具每迭代一个版本，文档就可能要改——所以各篇版本号跟随对应工具的真实版本，册子只保证"成册时刻文档与代码一致"，不承诺永远正确。

这不是缺陷，是这类文档的本分：结构服务于运行中的系统，系统在动，文档就得跟着动。母书完结封版，本册永远是一份"此刻的快照"。

最后

读这本册子的正确方式，是拿它对着仓库读：每篇架构文档对应一个真实目录、一套真实配置、一条真实链路。**结构的价值不在纸面上成立，在运行中成立**——能跑、能改、能审计，才算落了地。

—— wUwproject，2026 年 10 月

附录 A 统一术语表

本表收拢本书正文（七套系统文档 + 演进收束篇）共用的概念。**承自母书的术语定义与母书附录 A 一字不差**（不另造定义）；架构册特有的工程术语单列，并注明使用范围。

一、承自母书的术语（定义见母书附录 A）

术语	一句话口径	母书出处	本册使用篇
技能（Skill）	带元数据声明、可被标准校验的工具包	母书第 I 部	全部
智能体（Agent）	由链驱动架构编排的 LLM 执行系统	母书第 II 部	架构 04/05/07
链驱动执行（Chain-Driven）	链是主体，LLM 只做两头，中间确定性执行	母书 05	架构 06
有限决策范式（Limited Decision）	LLM 只填空，不决策——决策权归链、架构与人	母书 08	全部
前置规范（Pre-emptive Specification）	前置规范 > 后置验证——错误空间封堵在生成时刻	母书 08a	架构 07 主线
空（Slot） / 槽位	LLM 填空的槽位，内容轴不可收束到点	母书 08b	架构 04/05/07
槽位减法（Slot Subtraction）	槽位是减法不是加法——子集接受语义损失换幻觉收敛	母书 08c	架构 05/07
穷举（Enumeration）	用枚举保证稳定；穷举是统计本质的定价	母书 09	架构 04/05/07
推动点位（Driver Point）	配置推动把偏差钉在可指认的位置上	母书 09b	架构 05

二、架构册特有术语

术语	定义	使用篇
可执行原子 (Executable Atom)	技能拆解出的可执行单元 (CLI/脚本/子步骤) ——有主脚本入口的技能才可被编排	架构 06
可执行过滤 (Executable Filter)	纯提示词 skill (无主脚本入口) 不进入 Pipeline 池——编排对象必须是真执行的	架构 06
组合 (Combo)	silprespec-orchestrator 的前置规范单元: 生成 (LLM) → 后处理 (PY) → 校验 (PY) → 观测 (PY) 四段骨架, 当前原子库下穷举 14 种	架构 07
combo_registry	14 种组合的声明注册表 (COMBOS: list[ComboSpec]), PY 查表驱动, 不存在第 15 种	架构 07
PY 门禁	Python 确定性校验/后处理层的统称——"达标"与否的裁判是代码, 不是模型自评	架构 04/05/07
exec_recipe	组合的执行配方: generate / postprocess / validate / retry / observe 五元声明	架构 07 (源承 silprespec-emulator)
ProgressMap (进度地图)	贯穿编排全程的进度对象——子任务分解到执行汇总的每一步状态	架构 07
KB 签名 (KB Signature)	rag-assistant 的知识库关键词签名 (四分法采样 + jieba 候选 + BCE 排序) ——路由的精排依据	架构 04

术语	定义	使用篇
查询类型 (Query Type / 场景)	rag-assistant 的三槽位取值约束模板 (label + example + rules) —— 问题命中哪个场景就按哪个场景的规则填槽；内置 4 类 (fact/compare/opposition/analysis) + 用户自定义同 key 覆盖，配置级变更	架构 04
前置分类两级收敛	无定义穷举 → 三槽位骨架 (归类, 系统锁定) → 查询类型场景 (场景, 用户配置) → 有限枚举；归类切维度、场景约束值域，两级正交	架构 04
Ranker 后 插件点位 (before_response)	input_return 插件的执行时机——完整检索管道 (含 rerank) 完成后、LLM 综合回答前：只消费管道产出 (附加注入不改写)、失败透传、五道防线复用；引擎管道内部无插件点位，主体变更必须架构级，用点位隔离代替试错	架构 04
evidence 原文 原文锚定	每个 entity/attr 必须提供原文出处 (JSON 字典, key 与槽位写法精确一致、value 为原文单连续子串)，经当前消息/上一轮问答三源校验 + NLI verify 语义二次判断	架构 04
top-N 多 KB 路由	route_query 收集所有过阈值 KB 按分降序取前 N 个并查 (v2.3.0), <code>router.top_n</code> / <code>classify_threshold</code>	架构 04
vendor 自 包含	第三方库嵌入 <code>vendor/</code> 目录 (bs4/pypdfium2/markdownify…), 零外部 pip 安装也可运行	架构 04
配置统一源 (Config Unification)	散落各文件的常量收拢为单一模块 (如 structured-writer 的 <code>novel/nover_config.py</code>) ——推动点位唯一化	架构 05
三智能体注册 (Tool Registry)	silprespec-orchestrator 的 <code>tool_registry.py</code> : ToolSpec/FieldSpec/ExampleSpec 声明可调用的智能体及其输入槽位	架构 07
填空域 (Fill Domain)	一个 LLM 在一次协作中负责填充的边界——编排器/检索/写作各域不重叠不冲突	架构 05/06/07

术语	定义	使用篇
编排对象迁移 (Orchestration Object Migration)	编排器调用对象从技能 (subprocess 跑脚本) 迁到智能体 (HTTP POST 调 API) ——对象形态变了, 通道/LLM 在场位置/循环形态跟着换	架构 08
拟人 z 轴 (Z-Axis)	编排动作承担者在拟人尺度上的位置: 下端人工编排 (圈在系统外), 上端 LLM 拟人自主表象—— 同一个圈整圈上移 (圈还是那个圈, 位置不同), 圆心 (真决策) 在人, 非轴上套圈	架构 08
自主编排 (Autonomous Orchestration)	编排动作承担者的表象口径——LLM 以拟人姿态驱动每一步, 但路径不自由 (枚举内选择), 不说"自由路径决策"	架构 08
填槽即决策 (Slot-Filling as Decision)	LLM 每填一个槽位都是一次被约束的决策, 逐句发生逐句生效——循环的每一圈都产生决策, 非让 LLM 空转一圈	架构 08
决策让渡 (Decision Delegation)	路径/边界/回退的真决策是否交给 LLM——判断"长得像 ReAct 的编排器"还是"真 agent"的判据, 不看特征清单看决策归属	架构 09
施工公约 (Construction Covenant)	画地图的起点——一份动态、多文件施工地图的首个形态: 功能与公式间的硬绑定 (这个功能使用哪条公式、各代数是什么) 与施工偏好 (LLM 运行纪律: 可回溯、不冒进, 是"形式"上的保证); 后续每一次改动方案完整落地进地图, 与更新日志互为参照	架构 10
施工层级 (Construction Hierarchy)	三根支柱: 画地图定位 (成品之外——动态多文件的地图, 施工现场/施工规划/施工记录三方对比即进度管理)、 具体功能层解耦 (成品之中执行侧——每个功能的迭代只属于它自己)、 统一各级推动点位 (成品之中调度侧——功能的耦合层与多功能调度层, 耦合只发生在这一层); 合起来 = 架构上完全解耦、耦合层完全剥离	架构 10

术语	定义	使用篇
唯一定义处 (Single Definition Point)	同一真值只允许一个出口（路径/配置/换算/二进制定位各归一处）—— 两处各写一份就是两个真值来源，"两边看起来都对"的错误最难查	架构 06/10
引文回验 (Quote Verification)	检测输出 detail 中的「」引文必须在对应正文逐字存在，找不到即伪造 审核丢弃——前置规范范式：不依赖模型自觉	架构 05
三判据合验 (Triple-Criterion Verification)	音色体检的点名判定：四维总分、暗淡子分、身份下限任一命中即点名； 修复候选的验收与漂移检测共用同一把尺	架构 06

附录 B 运行速查

八个系统的运行方式完全不同：**技能**由宿主平台加载（无独立进程，跨平台分发）；**编排器与智能体**是独立程序（main.py 起进程、占端口、带 Web UI）。下表全部为真实代码核实（2026-10-03 更新）。

一、技能（宿主平台加载，无端口）

系统	位置	版本	运行方式
skill-standardization	maby_skills/skill-standardization/	2.10 3.0	宿主平台按 SKILL.md 触发 (audit/create/update/refactor/bump/readonly 六模式)
semantic-split	maby_skills/semantic-split/	3.1.1	宿主平台按 SKILL.md 触发（语义拆分 → 结构化规划单元）
activity-duration-estimation	maby_skills/activity-duration-estimation/	1.11.7	宿主平台按 SKILL.md 触发（WBS 分解 + 三点估算/蒙特卡洛）

技能发布于 maby_skills 仓库（ / 为任意宿主平台安装后的目录）；技能的"运行"= 宿主平台读取 SKILL.md 后按声明调用 scripts/ 脚本；也可被编排器（orchestrator）以 subprocess 直接跑脚本。

二、编排器（独立进程 + Web UI）

系统	位置	版本	运行方式
orchestrator（第 I 代）	maby_agent/Orchestrator/	2.8.1	python main.py （Web UI 端口 8788，支持 --port auto；LLM 后端 --backend custom --base-url ...）

系统	位置	版本	运行方式
silprespec-orchestrator（第II代，实验）	独立仓库 silprespec-orchestrator/	0.1.0	<code>python main.py --web</code> （Web UI 端口 8789）；依赖外部 LLM 后端（LM Studio / Ollama / OpenAI 兼容 API）

三、智能体（独立 HTTP 服务，maby_agent 仓库）

系统	位置	版本	运行方式
rag-assistant	maby_agent/rag-assistant/	2.4.1	<code>python main.py</code> （Web 界面 8765；外部 API <code>--api-port</code> 指定即启动，如 8767）
structured-writer	maby_agent/structured-writer/	3.1.14	<code>python main.py</code> （Web UI 8770；外部 API <code>--api-port</code> 指定，如 8777）
podcast-maker	maby_agent/podcast-maker/	1.4.0	<code>python main.py</code> （Web UI 8811， <code>--port</code> 可指定；本地语音服务 <code>tts_service/</code> 独立环境按需安装）

四、调用关系速记

- orchestrator —— subprocess ——> 技能（可执行过滤后的技能池）
- silprespec-orchestrator —— HTTP ——> 智能体 API（rag-assistant / structured-writer）
- skill-standardization —— 审计 ——> 其他所有技能与智能体的文档规范（元层，不参与业务调用）

五、收束篇（架构 09/10）

架构 09/10 为纯论述篇（非系统），**无运行实体**——架构 09 把编排器两代的变化线（对象/通道迁移、表象 vs 内核、“圈”的 z 轴）立成专文，运行形态以架构 07（orchestrator）与架构 08（silprespec-orchestrator）为准；架构 10 把多套系统共同遵守的施工层级（画地图定位/功能层解耦/统一各级推动点位）立成专文，运行形态以架构 01-08 为准。

